

Algorithms

Lecture # 01

Dr. Sohail Aslam

Design and analysis of Algorithms

Design and Analysis of Algorithms
Sohail Aslam

Model of Computation

Model of Computation

- In order to say anything meaningful about our algorithms, it will be important for us to settle on a mathematical model of computation
- Ideally this model should be a reasonable abstraction of a standard generic single-processor machine.

Model of Computation

Model of Computation

about our algorithms, it will be important for us to settle on a mathematical model of computation

- Ideally this model should be a reasonable abstraction of a standard generic single-processor machine.
- We call this model a *random access machine* or *RAM*.

Preliminaries

Preliminaries

- *Course:* Design and Analysis of Algorithms
- *Prerequisites:* Data Structures, Discrete Mathematics
- *Textbook:* Introduction to Algorithms, Cormen, Leiserson, Rivest, Stein, 2nd edition, 2001 The MIT Press.

Preliminaries

Preliminaries

- *Textbook:* Introduction to Algorithms, Cormen, Leiserson, Rivest, Stein, 2nd edition, 2001 The MIT Press.
- Grading:
 - homework and projects
 - mid term exam
 - final

Origin of Word: ALGORITHM

Origin of word: Algorithm

Abu Ja'far Mohammad
ibn Musa al-Khowarizmi.

Origin of Word: ALGORITHM



Origin of Word: ALGORITHM



Origin of Word: ALGORITHM

Origin of word: ***Algorithm***

- His year of birth is not known exactly.
- Al-Khwarizmi parents migrated to a placesouth of Baghdad when he was a child.

Origin of Word: ALGORITHM

Origin of word: ***Algorithm***

- It has been established from his contributions that he flourished under Khalifah Al-Mamun at Baghdad during 813 to 833 C.E.
- Al-Khwarizmi died around 840 C.E.

Origin of Word: ALGORITHM



Origin of Word: ALGORITHM

*al Kitab al-mukhatar
fi hisab al-jabr wa'l-
muqabalah*

Origin of Word: ALGORITHM

Origin of word: *Algorithm*

- It is from the titles of these writings and his name that the words **algebra** and **algorithm** are derived.
- As a result of his work, al-Khwarizmi is regarded as the most outstanding mathematician of his time

ALGORITHM informal definition

Algorithm: Informal Definition

- An algorithm is any well-defined computational procedure that takes some values, or set of values, as input and produces some value, or set of values, as output.
- An algorithm is thus a sequence of computational steps that transform the input into output.

ALGORITHM, programming

Algorithms, Programming

- A good understanding of algorithms is essential for *programming*.
- However, unlike a program, an algorithm is a *mathematical entity*

ALGORITHM, programming

Algorithms, Programming

- Algorithm is independent of a specific programming language, machine, or compiler.
- Algorithm design is about mathematical theory behind the design of good programs.

Why Study?

Why Study Algorithms

- This course is all about how to design good algorithms.
- **CS301** dealt with how to design good data structures.
- This is not really an independent issue because the fastest algorithms are fast because they use fast data structures, and vice versa.

Why Study?

Why Study Algorithms

- The fact is that many of the courses in computer science deal with efficient algorithms and data structures.

Why Study?

Why Study Algorithms

They apply these to various applications:

- compilers,
- operating systems,
- databases,
- artificial intelligence,
- computer graphics and vision,
- networks, etc.

Why Study?

Why Study Algorithms

- A good understanding of algorithm design is a *central element* to a good understanding of computer science and computer programming.

Course outline

Course Outline

This course will consist of four major sections.

1. Mathematical tools necessary for the analysis of algorithms. This will focus on asymptotics, summations, recurrences.

Course outline

Course Outline

-
2. Sorting: this will focus on different strategies for sorting, and use this problem as a case-study in different techniques for designing and analyzing algorithms.

Course outline

Course Outline

3. Collection of various algorithmic problems and solution techniques: Dynamic programming, greedy strategy, graphs.

Course outline

Course Outline

4. Introduction to the theory of NP-completeness.
NP-complete problem are those for which no efficient algorithms are known, but no one knows for sure whether efficient solutions might exist.

Criteria for analyzing Algorithms

Criterion for Analyzing Algorithms

- In order to design good algorithms, we must first agree on the *criterion* for measuring algorithms.
- The emphasis is on the design of efficient algorithm

Criteria for analyzing Algorithms

Criterion for Analyzing Algorithms

We will measure algorithms in terms of the amount of computational resources that the algorithm requires:

- running time
- memory

Model of Computation

Model of Computation

- We want the analysis to be as independent as possible of the variations in machine, operating system, compiler, or programming language.
- Unlike programs, algorithms are to be understood primarily by people and not machines.

Model of Computation

Model of Computation

- Thus gives us quite a bit of flexibility in how we present our algorithms, and many low-level details may be omitted

Model of Computation

Model of Computation

- Ideally this model should be a reasonable abstraction of a standard generic single-processor machine.
- We call this model a *random access machine* or *RAM*.

Random Access Machine

Random Access Machine

- A RAM is an idealized machine with an infinitely large random-access memory.
- Instructions are executed one-by-one (there is no parallelism).
- Each instruction involves performing some basic operation on two values in the machine's memory

Random Access Machine

- A RAM is an idealized machine with an infinitely large random-access memory.
- Instructions are executed one-by-one (there is no parallelism).
- Each instruction involves performing some basic operation on two values in the machine's memory

Basic RAM Operations

Basic operations include

- assigning a value to a variable,
- computing any basic arithmetic operation (+, -, $\times$ and division) on integer values of any size
- performing any comparison or boolean operations
- accessing an element of an array (e.g. $A[10]$)

RAM Model

- We assume that each basic operation takes the same constant time to execute.
- The RAM model does a good job of describing the computational power of most modern (nonparallel) machines.

RAM Model

Basic operations in C++ that are assumed to take up same amount of CPU time.

- `x = y;`
- `z = a + b;` `z = a - b;`
- `z = a * b;` `z = a / b;`
- `z = w[10];`
- `(x <= y)`

Example: 2-dimension maxima

Let us do an example that illustrates how we analyze algorithms.

- Suppose you want to buy a car.
- You want to pick the fastest car.
- But fast cars are expensive; you want the cheapest.
- You cannot decide which is more important: speed or price.

Example: 2-dimension maxima

- Definitely *do not* want a car if there is another that is both faster and cheaper.
- We say that the fast, cheap car *dominates* the slow, expensive car relative to your selection criteria.
- So, given a collection of cars, we want to list those cars that are not dominated by any other.

Example: 2-dimension maxima

Here is how we might model this as a formal problem.

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to be *dominated* by point q if $p.x \leq q.x$ and $p.y \leq q.y$.

Example: 2-dimension maxima

- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be *maximal* if it is not dominated by any other point in P .

Example: 2-dimension maxima

The car selection problem can be modelled this way:

For each car we associate (x, y) pair where

- x is the speed of the car and
- y is the negation of the price.

Example: 2-dimension maxima

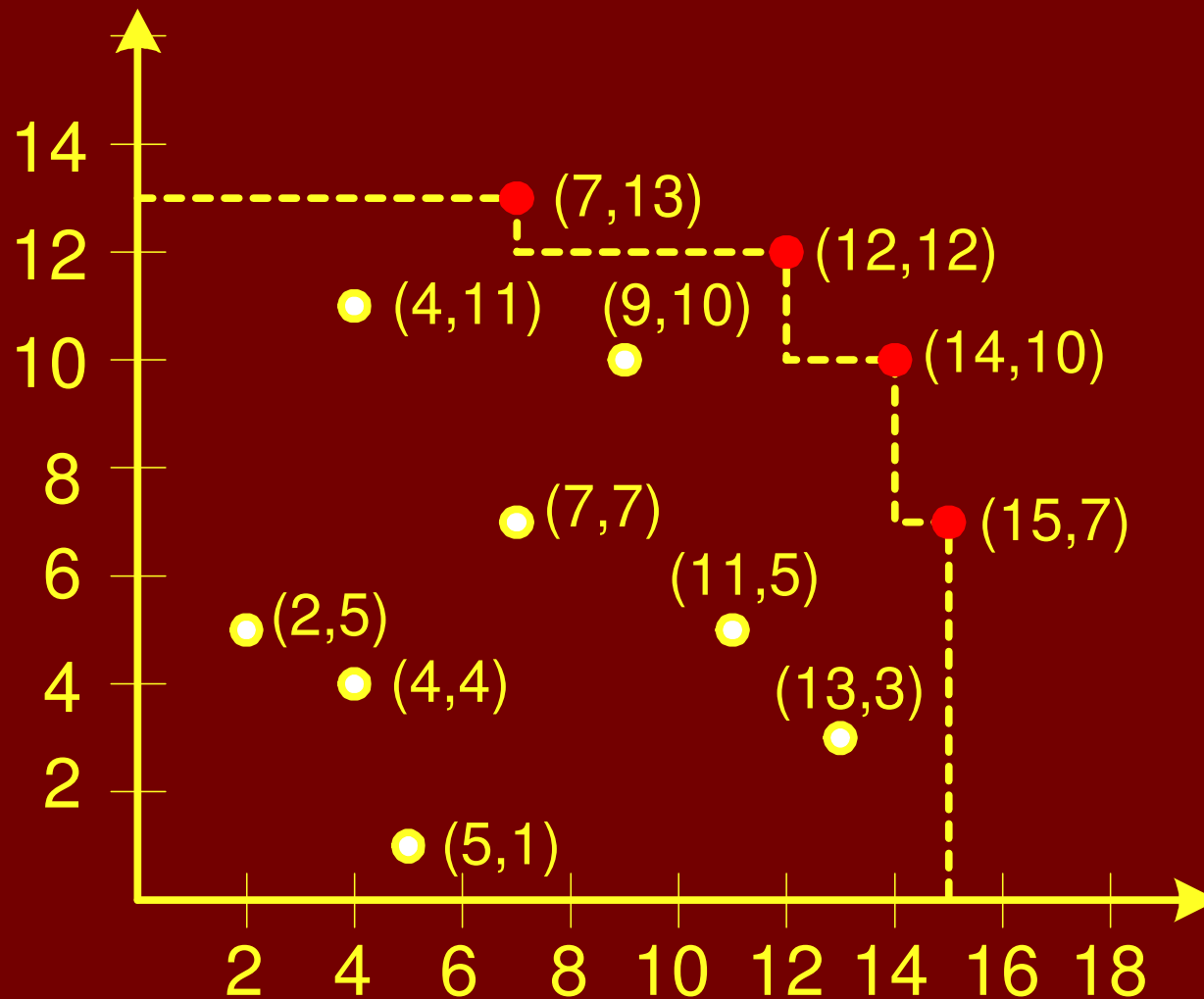
- High y value means a cheap car and low y means expensive car.
- Think of y as the money left in your pocket after you have paid for the car.
- Maximal points correspond to the fastest and cheapest cars.

Example: 2-dimension maxima

2-dimensional Maxima:

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space,
- output the set of maximal points of P ,
- i.e., those points p_i such that p_i is not dominated by any other point of P .

Example: 2-dimension maxima



Example: 2-dimension maxima

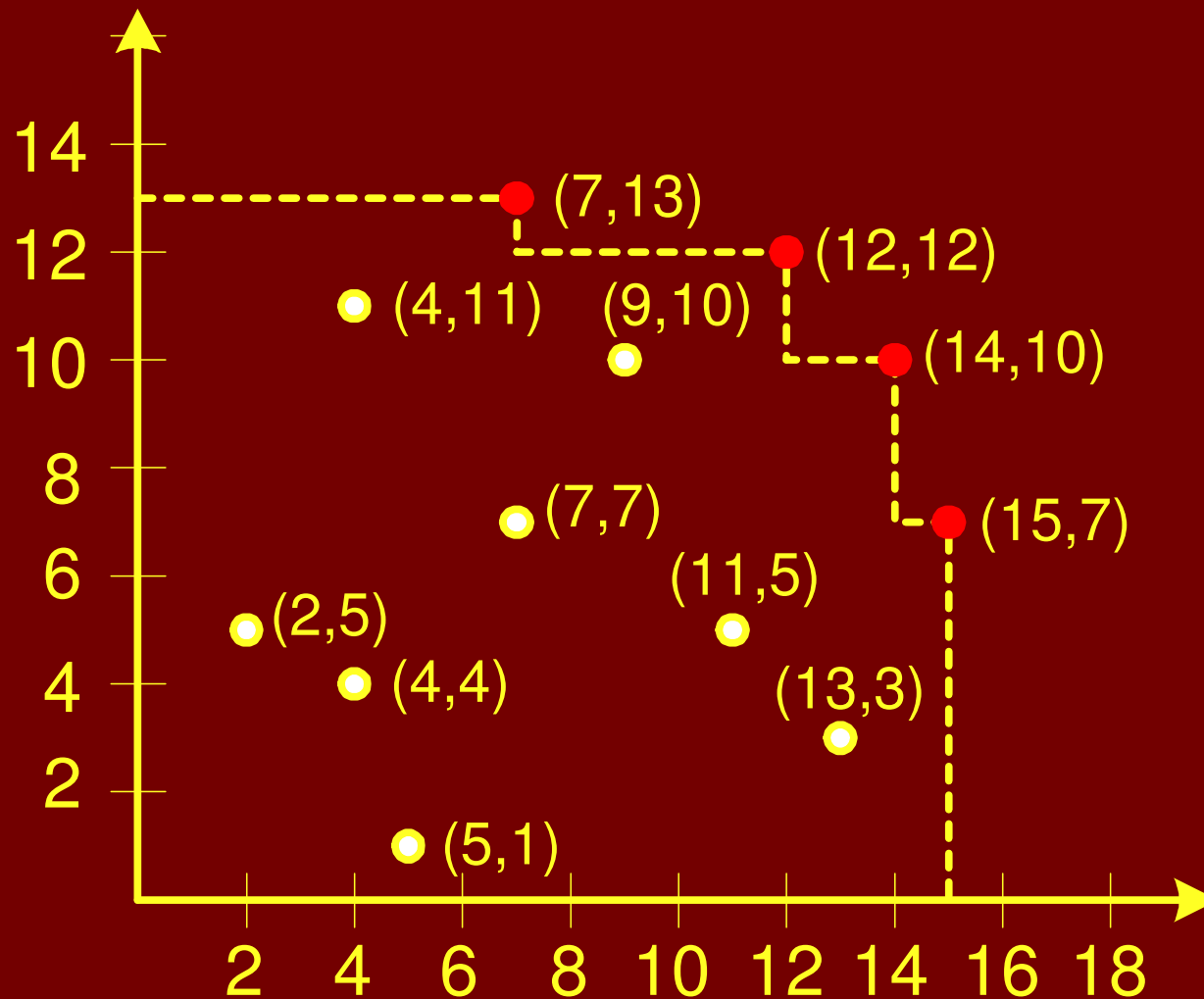
- Our description of the problem is at a fairly mathematical level.
- We have intentionally not discussed how the points are represented.
- We have not discussed any input or output formats.
- We have avoided programming and other software issues.

Example: 2-dimension maxima

2-dimensional Maxima:

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space,
- output the set of maximal points of P ,
- i.e., those points p_i such that p_i is not dominated by any other point of P .

Example: 2-dimension maxima



Example: 2-dimension maxima

- Our description of the problem is at a fairly mathematical level.
- We have intentionally not discussed how the points are represented.
- We have not discussed any input or output formats.
- We have avoided programming and other software issues.

Example: 2-dimension maxima

Brute-Force Algorithm:

- Let $P = \{p_1, p_2, \dots, p_n\}$ be the initial set of points.
- For each point p_i , test it against all other points p_j .
- If p_i is not dominated by any other point, then output it.

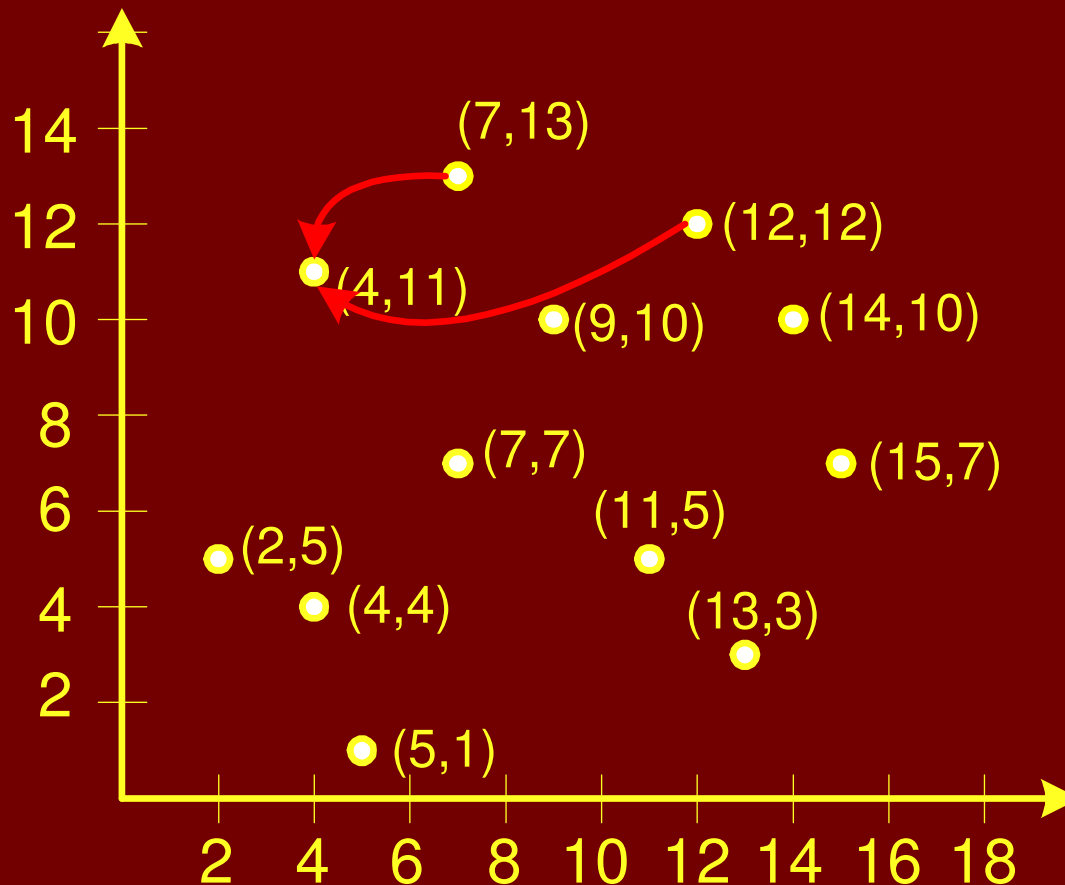
Example: 2-dimension maxima

MAXIMA(int n , Point $P[1 \dots n]$)

```
1  for  $i \leftarrow 1$  to  $n$ 
2  do maximal  $\leftarrow$  true
3    for  $j \leftarrow 1$  to  $n$ 
4    do
5      if  $(i \neq j)$  and  $(P[i].x \leq P[j].x)$  and  $(P[i].y \leq P[j].y)$ 
6      then maximal  $\leftarrow$  false; break
7  if (maximal = true)
8    then output  $P[i]$ 
```

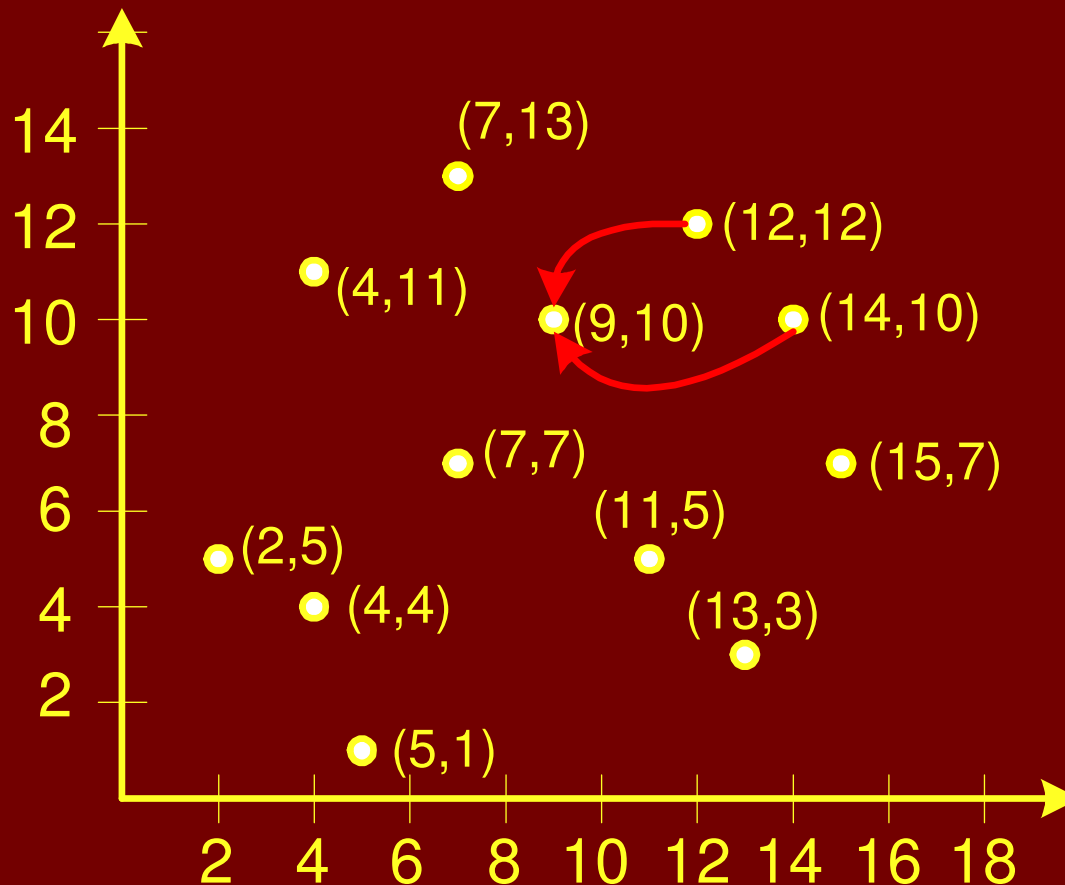
Example: 2-dimension maxima

Points that dominate (4, 11)



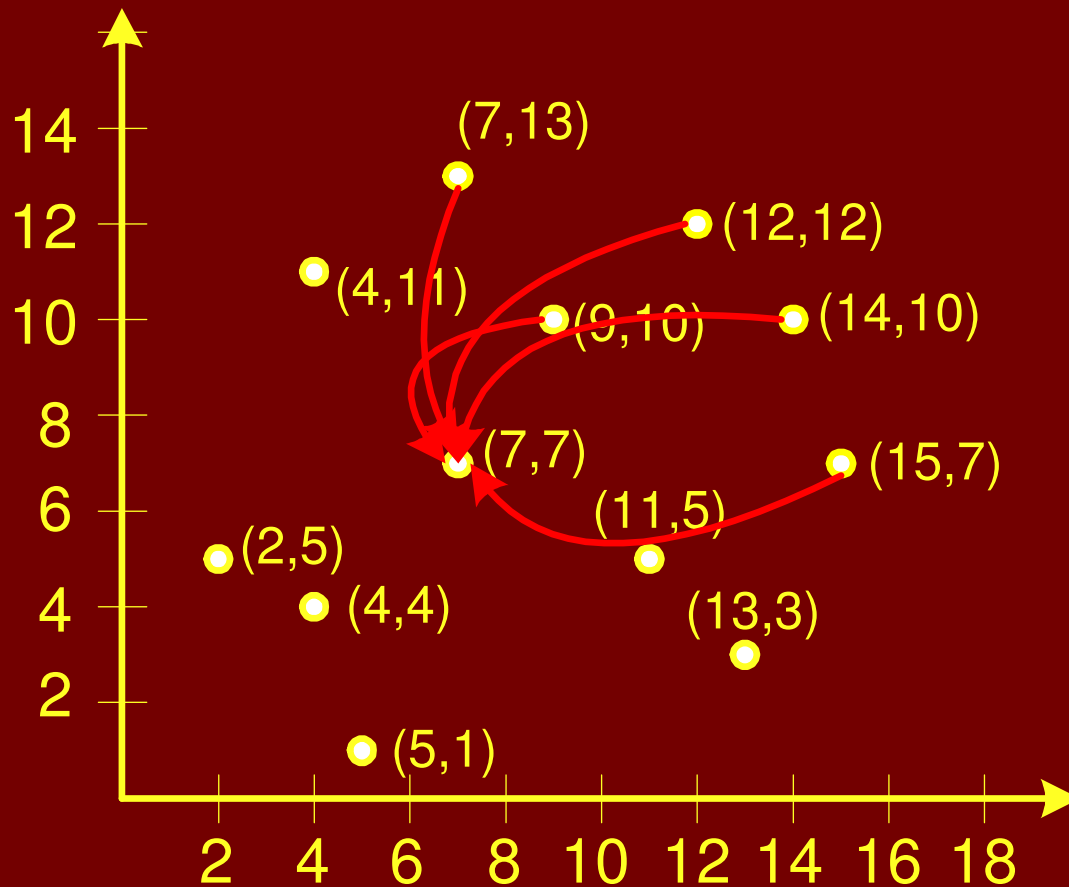
Example: 2-dimension maxima

Points that dominate (9, 10)



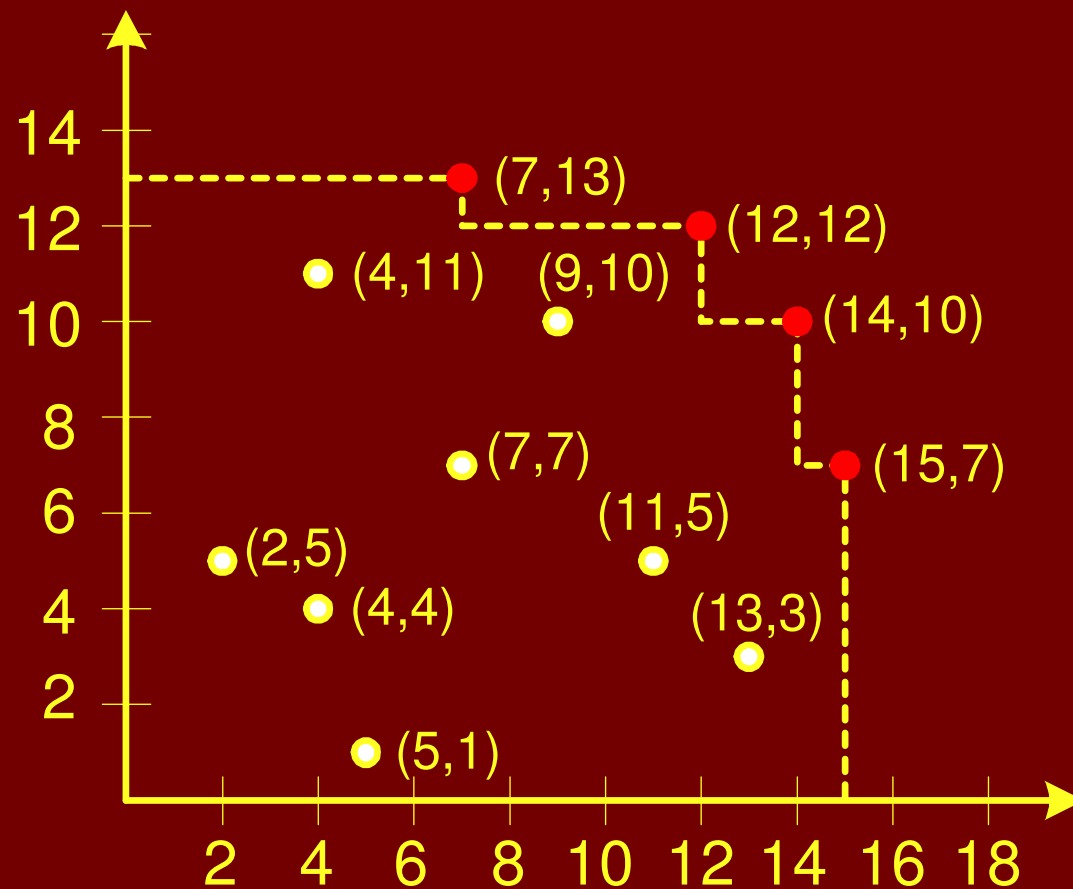
Example: 2-dimension maxima

Points that dominate $(7, 7)$



Example: 2-dimension maxima

The maximal points



Running Time Analysis

Running Time Analysis

- The main purpose of our mathematical analysis will be measuring the execution time.
- We will also be concerned about the space (memory) required by the algorithm.

Running Time Analysis

- The running time of an implementation of the algorithm would depend upon
 - the speed of the computer,
 - programming language,
 - optimization by the compiler etc.
- Although important, we will ignore these technological issues in our analysis.

Analysis of 2-d Maxima

To measure the running time of the brute-force 2-d maxima algorithm, we could

- count the number of steps of the pseudo code that are executed
- or, count the number of times an element of P is accessed
- or, the number of comparisons that are performed.

Running Time Analysis

- The running time depends upon the input size, e.g. n
- Different inputs of the same size may result in different running time.
- For example, breaking out of the inner loop in the brute-force algorithm depends not only on the input size of P but also the structure of the input.

Running Time Analysis: Criteria

Two criteria for measuring running time:

1. Worst-case time:

- Worst-case time is the maximum running time over all (legal) inputs of size n .

Running Time Analysis: Criteria

Worst-case time:

- Let I denote an input instance,
- let $|I|$ denote its length, and
- let $T(I)$ denote the running time of the algorithm on input I .
- Then

$$T_{\text{worst}}(n) = \max_{|I|=n} T(I)$$

Running Time Analysis: Criteria

2. Average-case time

- Average-case time is the average running time over all inputs of size n .

Running Time Analysis: Criteria

2. Average-case time

- Let $p(I)$ denote the probability of seeing this input.
- The average-case time is the weighted sum of running times with weights being the probabilities:

$$T_{\text{avg}}(n) = \sum_{|I|=n} p(I)T(I)$$

Running Time Analysis

- We will almost always work with worst-case time.
- Average-case time is more difficult to compute; it is difficult to specify probability distribution on inputs.
- Worst-case time will specify an upper limit on the running time.

Analysis of 2-dimension maxima

Analysis of the brute-force maxima algorithm.

- Input size is n
- We will count number of times that any element of P is accessed.

Analysis of 2-dimension maxima

MAXIMA(int n, Point P[1...n])

1 for i ← 1 to n *n times*

2 do maximal ← true

3 for j ← 1 to n *n times*

4 do

5 if $(i \neq j) \& (P[i].x \leq P[j].x) \& (P[i].y \leq P[j].y)$ *4 accesses*

6 then maximal ← false break

7 if maximal

8 then output P[i].x, P[i].y *2 accesses*

Analysis of 2-dimension maxima

- The outer loop runs n times
- For each i iteration, the inner loop runs n time.
- P is accessed four times in the *if* statement.
- The output statement accesses P 2 times.
- In the worst case, every point is maximal so every point is output.

Analysis of 2-dimension maxima

MAXIMA(int n, Point P[1...n])

1 for i ← 1 to n *n times*

2 do maximal ← true

3 for j ← 1 to n *n times*

4 do

5 if $(i \neq j) \& (P[i].x \leq P[j].x) \& (P[i].y \leq P[j].y)$ *4 accesses*

6 then maximal ← false break

7 if maximal

8 then output P[i].x, P[i].y *2 accesses*

Analysis of 2-dimension maxima

- The outer loop runs n times
- For each i iteration, the inner loop runs n time.
- P is accessed four times in the *if* statement.
- The output statement accesses P 2 times.
- In the worst case, every point is maximal so every point is output.

Analysis of 2-dimension maxima

Worst-case running time:

Pair of nested summations, one for i-loop and the other for the j-loop

$$T(n) = \sum_{i=1}^n \left(2 + \sum_{j=1}^n 4 \right)$$

Analysis of 2-dimension maxima

Worst-case running time:

$$T(n) = \sum_{i=1}^n \left(2 + \sum_{j=1}^n 4 \right)$$

$(\sum_{j=1}^n 4) = 4n$, and so

$$\begin{aligned} T(n) &= \sum_{i=1}^n (4n + 2) \\ &= (4n + 2)n = 4n^2 + 2n \end{aligned}$$

Analysis of 2-dimension maxima

- For small values of n , any algorithm is fast enough.
- What happens when n gets large?
- Running time does become an issue.
- When n is large, n^2 term will be much larger than the n term and will dominate the running time.

Analysis of 2-dimension maxima

- We will say that the worst-case running time is $\Theta(n^2)$.
- This is called the *asymptotic growth rate* of the function.
- We will discuss this Θ -notation more formally later.

Summations

- The analysis involved computing a summation.
- Summation should be familiar but let us review a bit here.

Summations

- Given a finite sequence of values $a_1, a_2, \dots, a_n$,
- their sum $a_1 + a_2 + \dots + a_n$ is expressed in *summation notation* as

$$\sum_{i=1}^n a_i$$

- If $n = 0$, then the sum is additive identity, 0.

Summations

Some facts about summation:

- If c is a constant

$$\sum_{i=1}^n c a_i = c \sum_{i=1}^n a_i$$

and

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

Summations

Some important summations that should be committed to memory.

Arithmetic series:

$$\begin{aligned}\sum_{i=1}^n i &= 1 + 2 + \dots + n \\ &= \frac{n(n+1)}{2} = \Theta(n^2)\end{aligned}$$

Summations

Quadratic series:

$$\begin{aligned}\sum_{i=1}^n i^2 &= 1 + 4 + 9 + \dots + n^2 \\ &= \frac{2n^3 + 3n^2 + n}{6} = \Theta(n^3)\end{aligned}$$

Summations

Geometric series:

$$\sum_{i=1}^n x^i = 1 + x + x^2 + \dots + x^n$$
$$= \frac{x^{(n+1)} - 1}{x - 1} = \Theta(n^2)$$

If $0 < x < 1$ then this is $\Theta(1)$, and if $x > 1$, then this is $\Theta(x^n)$.

Summations

Harmonic series: For $n \geq 0$

$$\begin{aligned} H_n &= \sum_{i=1}^n \frac{1}{i} \\ &= 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n \\ &= \Theta(\ln n) \end{aligned}$$

A Harder Example

NESTED-LOOPS()

```
1  for  $i \leftarrow 1$  to  $n$ 
2  do
3      for  $j \leftarrow 1$  to  $2i$ 
4      do  $k = j \dots$ 
5          while ( $k \geq 0$ )
6          do  $k = k - 1 \dots$ 
```

A Harder Example

```
NESTED-LOOPS()  
1  for  $i \leftarrow 1$  to  $n$   
2  do  
3      for  $j \leftarrow 1$  to  $2i$   
4      do  $k = j \dots$   
5          while ( $k \geq 0$ )  
6          do  $k = k - 1 \dots$ 
```

How do we analyze the running time of an algorithm that has complex nested loop?

A Harder Example

```
NESTED-LOOPS()  
1  for  $i \leftarrow 1$  to  $n$   
2  do  
3      for  $j \leftarrow 1$  to  $2i$   
4      do  $k = j \dots$   
5          while ( $k \geq 0$ )  
6          do  $k = k - 1 \dots$ 
```

The answer is we write out the loops as summations and then solve the summations.

A Harder Example

```
NESTED-LOOPS()  
1  for  $i \leftarrow 1$  to  $n$   
2  do  
3      for  $j \leftarrow 1$  to  $2i$   
4      do  $k = j \dots$   
5          while ( $k \geq 0$ )  
6          do  $k = k - 1 \dots$ 
```

To convert loops into summations, we work from inside-out.

A Harder Example

NESTED-LOOPS()

```
1  for  $i \leftarrow 1$  to  $n$ 
2  do for  $j \leftarrow 1$  to  $2i$ 
3      do  $k = j$ 
4          while ( $k \geq 0$ ) ◀
5          do  $k = k - 1$ 
```

- Consider the *inner most while* loop.
- It is executed for $k = j, j - 1, j - 2, \dots, 0$.

A Harder Example

NESTED-LOOPS()

```
1  for i ← 1 to n
2  do for j ← 1 to 2i
3      do k = j
4          while (k ≥ 0) ◀
5          do k = k - 1
```

- Time spent inside the while loop is constant.
- Let $I()$ be the time spent in the while loop. Thus

$$I(j) = \sum_{k=0}^j 1 = j + 1$$

A Harder Example

NESTED-LOOPS()

1 for $i \leftarrow 1$ to n

2 do for $j \leftarrow 1$ to $2i$ 

3 do $k = j$

4 while ($k \geq 0$)

5 do $k = k - 1$

- Consider the *middle for* loop.
- It's running time is determined by i .

A Harder Example

NESTED-LOOPS()

1 for $i \leftarrow 1$ to n

2 do for $j \leftarrow 1$ to $2i$ 

3 do $k = j$

4 while ($k \geq 0$)

5 do $k = k - 1$

- Let $M()$ be the time spent in the for loop:

$$M(i) = \sum_{j=1}^{2i} I(j)$$

A Harder Example

$$\begin{aligned}M(i) &= \sum_{j=1}^{2i} I(j) = \sum_{j=1}^{2i} (j + 1) \\&= \sum_{j=1}^{2i} j + \sum_{j=1}^{2i} 1 \\&= \frac{2i(2i + 1)}{2} + 2i \\&= 2i^2 + 3i\end{aligned}$$

A Harder Example

NESTED-LOOPS()

```
1  for i ← 1 to n
2  do for j ← 1 to 2i
3      do k = j
4          while (k ≥ 0)
5              do k = k - 1
```



- Finally the *outer-most for* loop.
- Let $T()$ be running time of the entire algorithm:

$$T(n) = \sum_{i=1}^n M(i)$$

A Harder Example

$$\begin{aligned}T(n) &= \sum_{i=1}^n M(i) = \sum_{i=1}^n (2i^2 + 3i) \\&= \sum_{i=1}^n 2i^2 + \sum_{i=1}^n 3i \\&= 2 \frac{2n^3 + 3n^2 + n}{6} + 3 \frac{n(n+1)}{2} \\&= \frac{4n^3 + 15n^2 + 11n}{6} \\&= \Theta(n^3)\end{aligned}$$

2-dimension Maxima Revisited

Recall the 2-d maxima problem:

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to *dominated* by point q if $p.x \leq q.x$ and $p.y \leq q.y$.

2-dimension Maxima Revisited

- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be *maximal* if it is not dominated by any other point in P .
- The problem is to output all the maximal points of P .

2-dimension Maxima Revisited

- We introduced a brute-force algorithm that ran in $\Theta(n^2)$ time.
- It operated by comparing *all pairs* of points.
- Is there an approach that is significantly better?

2-dimension Maxima Revisited

- The problem with the brute-force algorithm is that it uses no intelligence in *pruning* out decisions.
- For example, once we know that a point p_i is dominated by another point p_j , we do not need to use p_i for eliminating other points.

2-dimension Maxima Revisited

- This follows from the fact that dominance relation is *transitive*.
- If p_j dominates p_i and p_i dominates p_h then p_j also dominates p_h ; p_i is not needed.

Plane-sweep Algorithm

Plane-sweep Algorithm

- We will sweep a vertical line across the 2-d plane from left to right.
- As we sweep, we will build a structure holding the maximal points lying to the left of the sweep line.

Plane-sweep Algorithm

Plane-sweep Algorithm

- When the sweep line reaches the rightmost point of P , we will have the complete set of maximal points.
- This approach of solving geometric problems is called the *plane sweep*.

Plane-sweep Algorithm

Plane-sweep Algorithm:

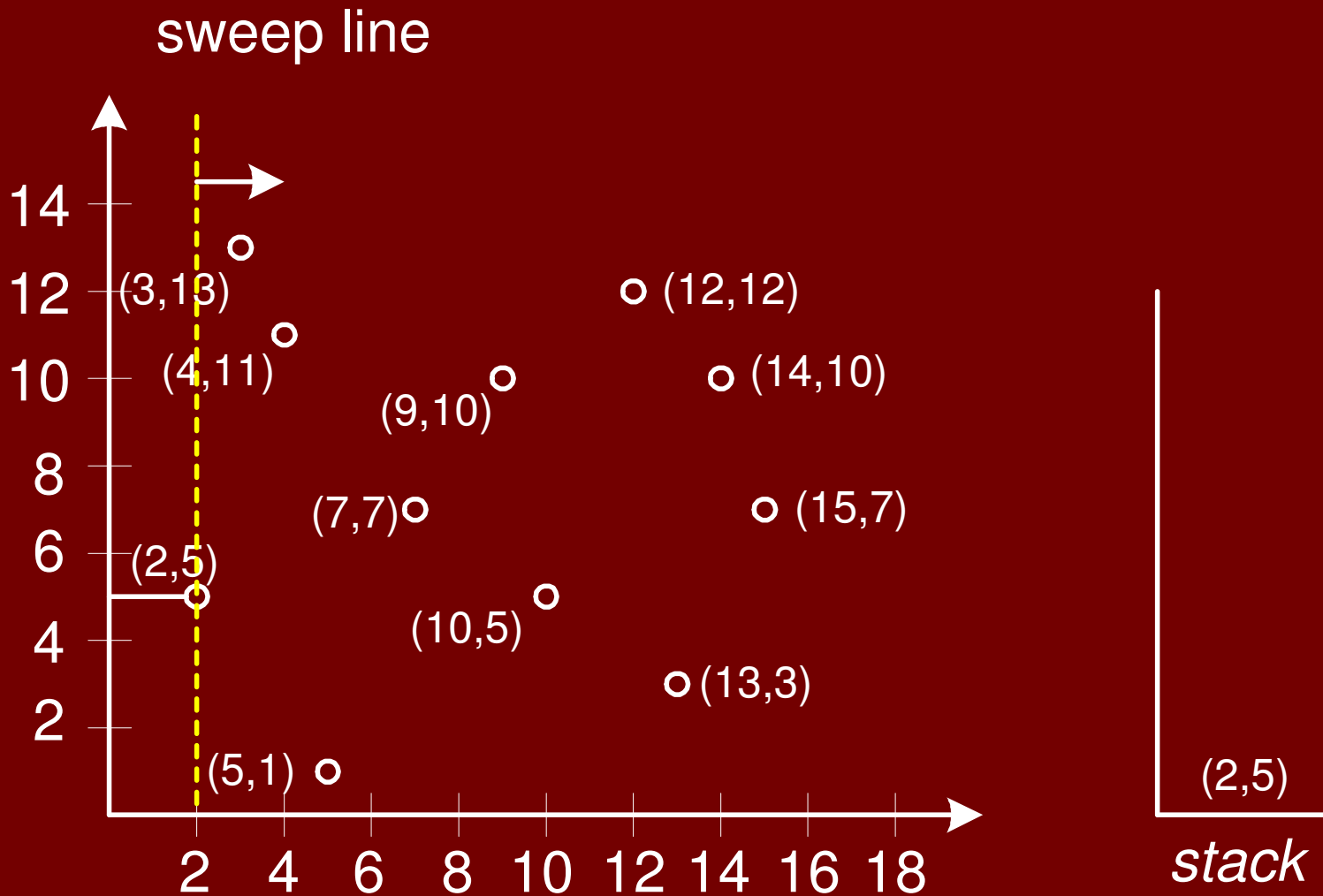
- Store the existing maximal points in a list
- The points that p_i dominates will appear at the end of the list because points are sorted by x -coordinate.
- Scan the list left to right.
- Every maximal point with y -coordinate less than p_i will be eliminated from computation.

Plane-sweep Algorithm

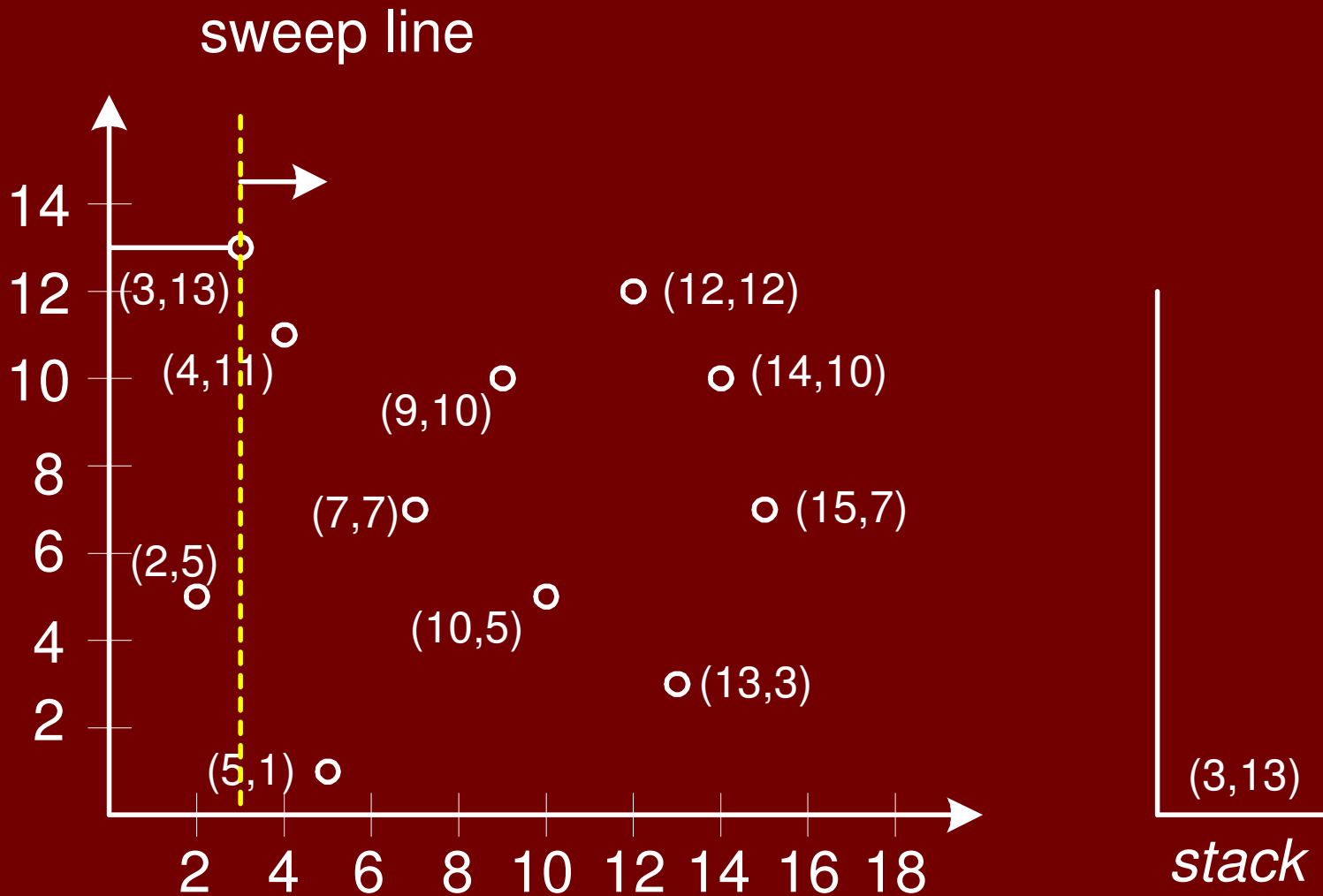
Plane-sweep Algorithm:

- We add maximal points onto the end of a list and delete from the end of the list.
- We can thus use a *stack* to store the maximal points.
- The point at the top of the stack will have the highest x -coordinate.

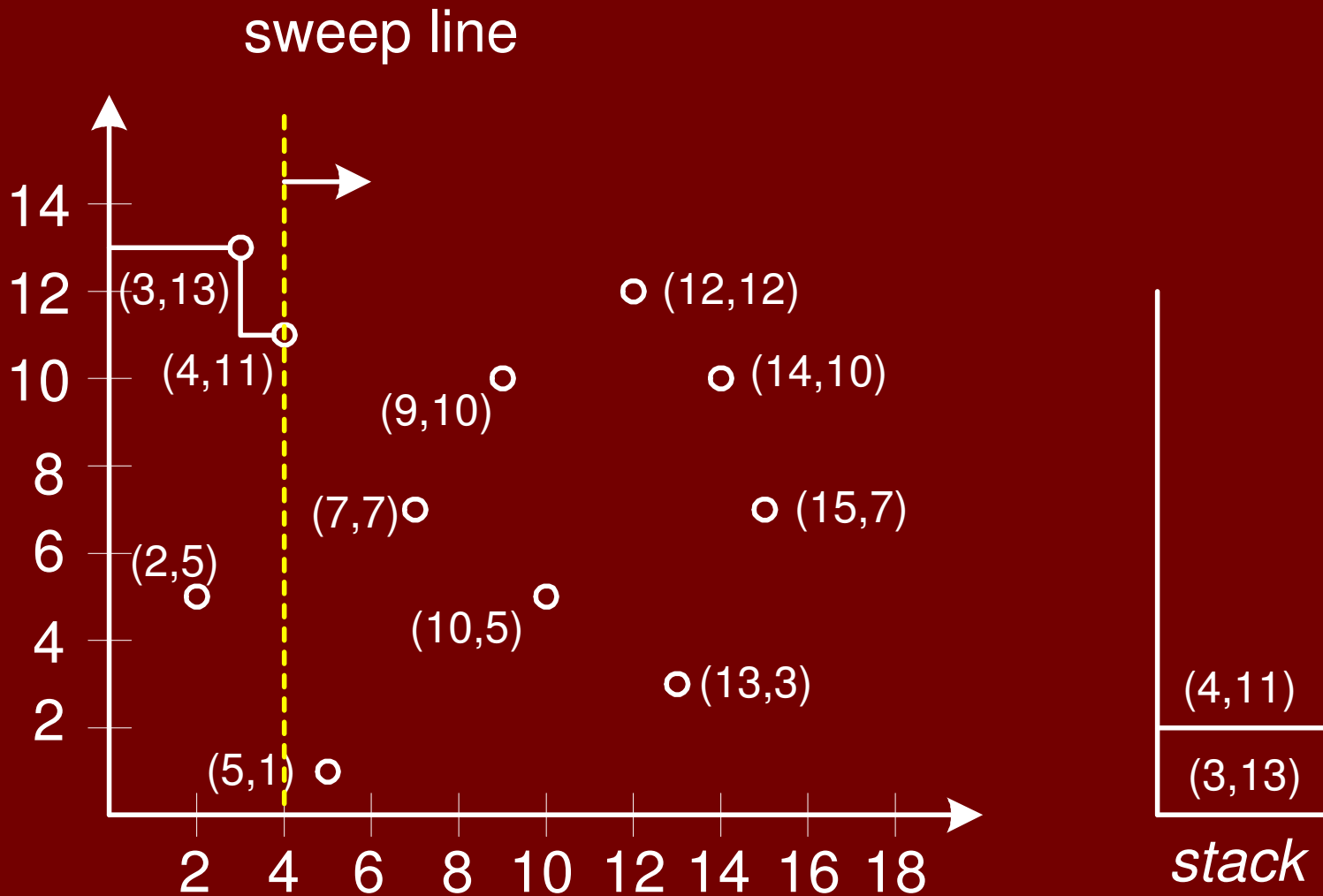
Plane-sweep Algorithm



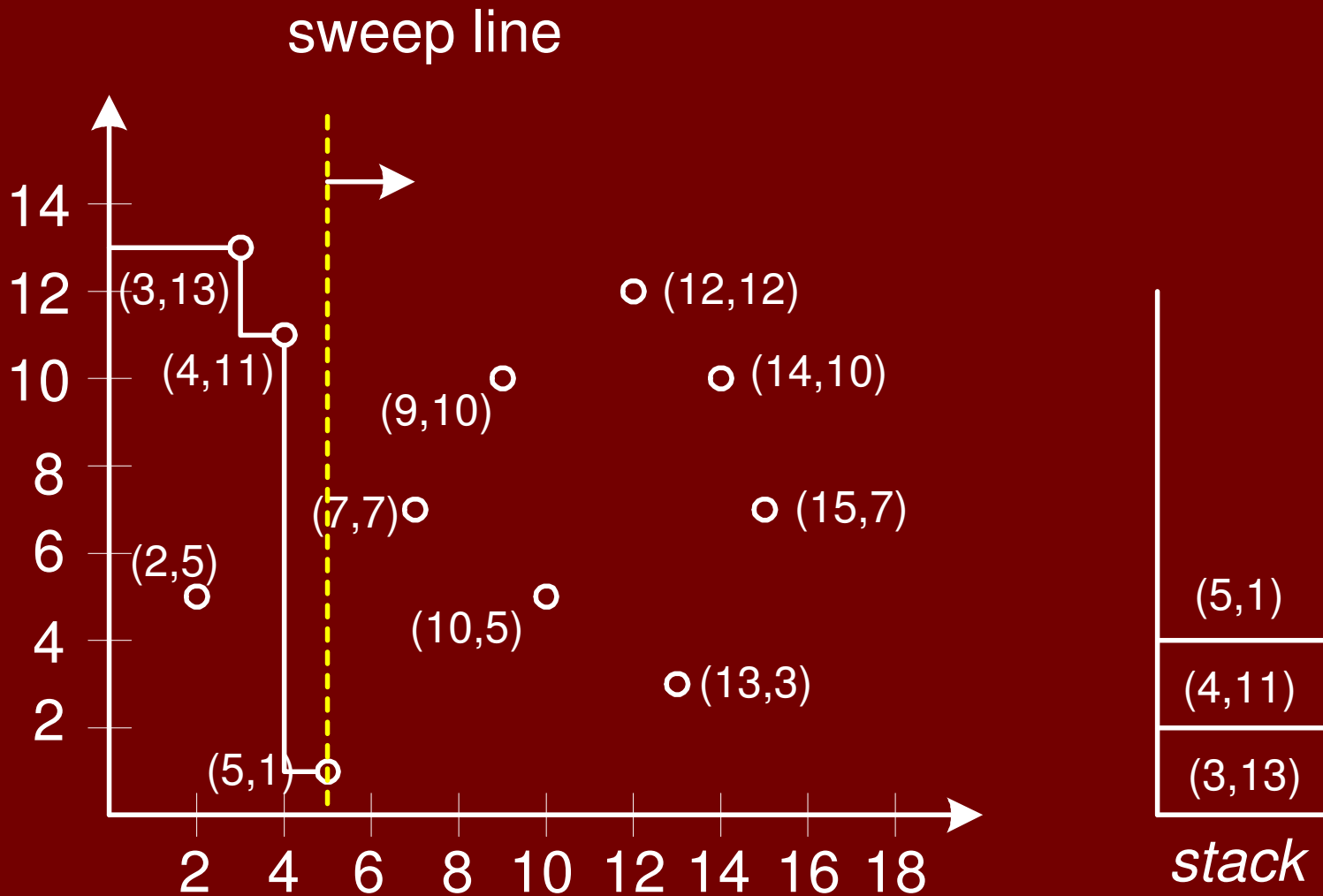
Plane-sweep Algorithm



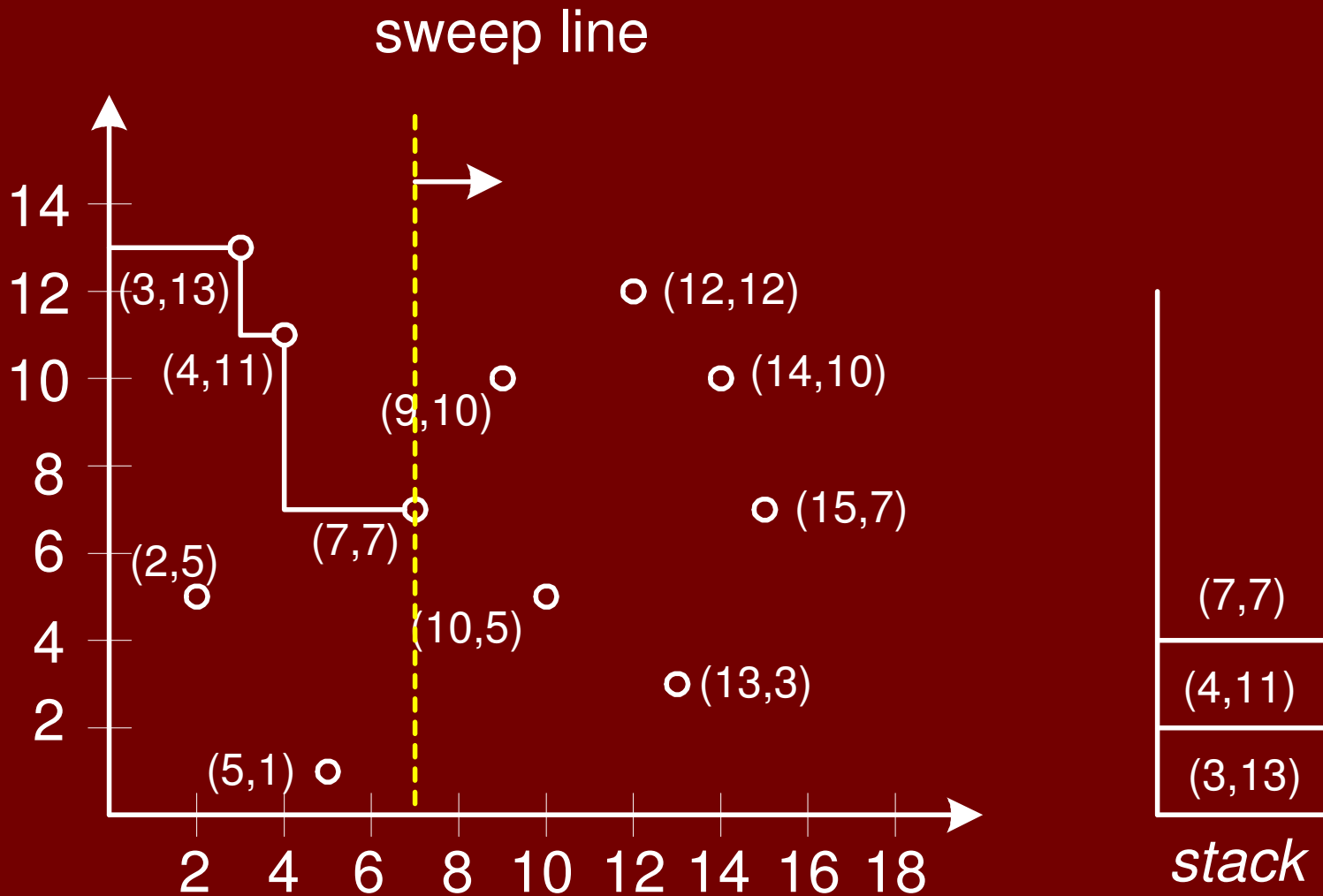
Plane-sweep Algorithm



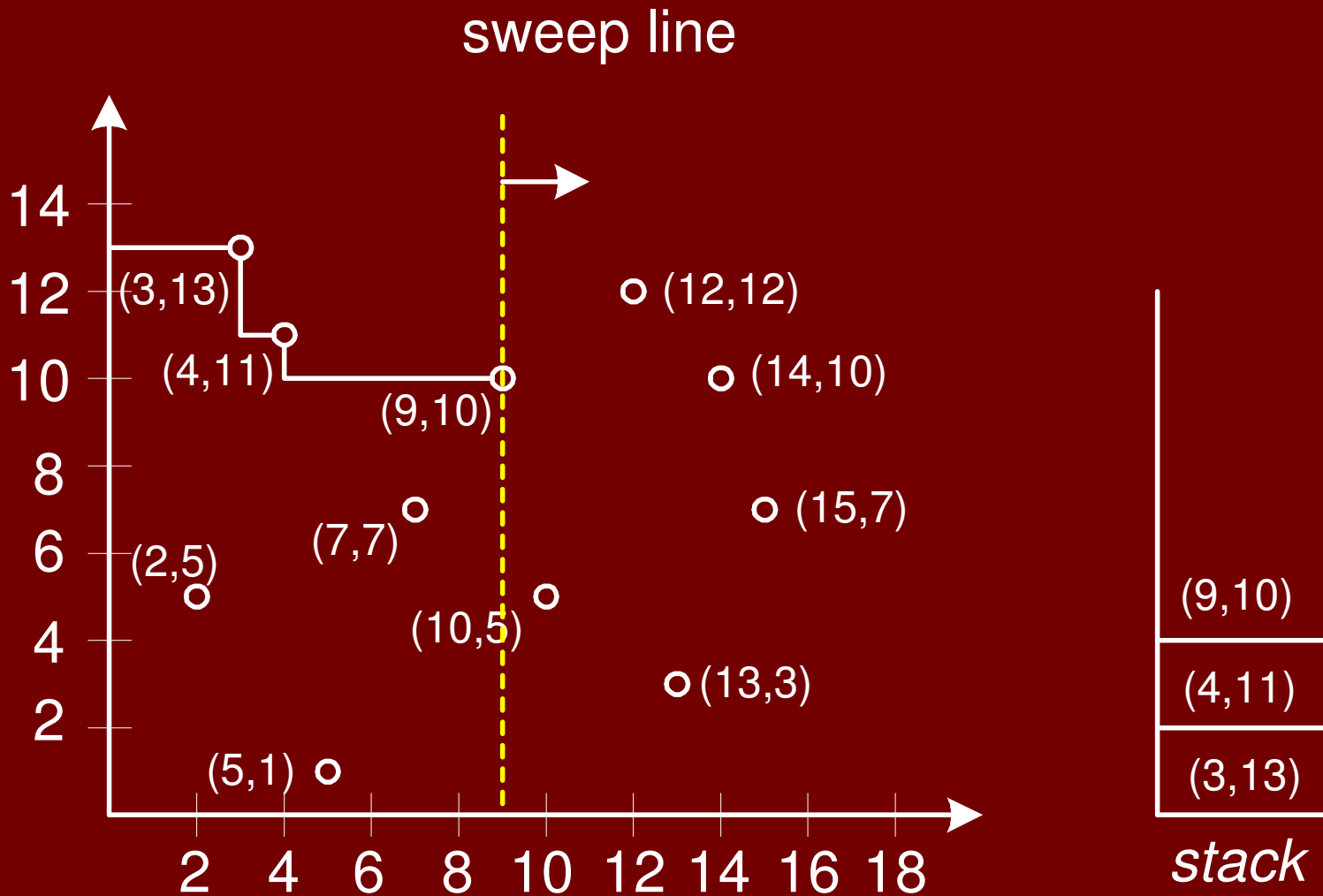
Plane-sweep Algorithm



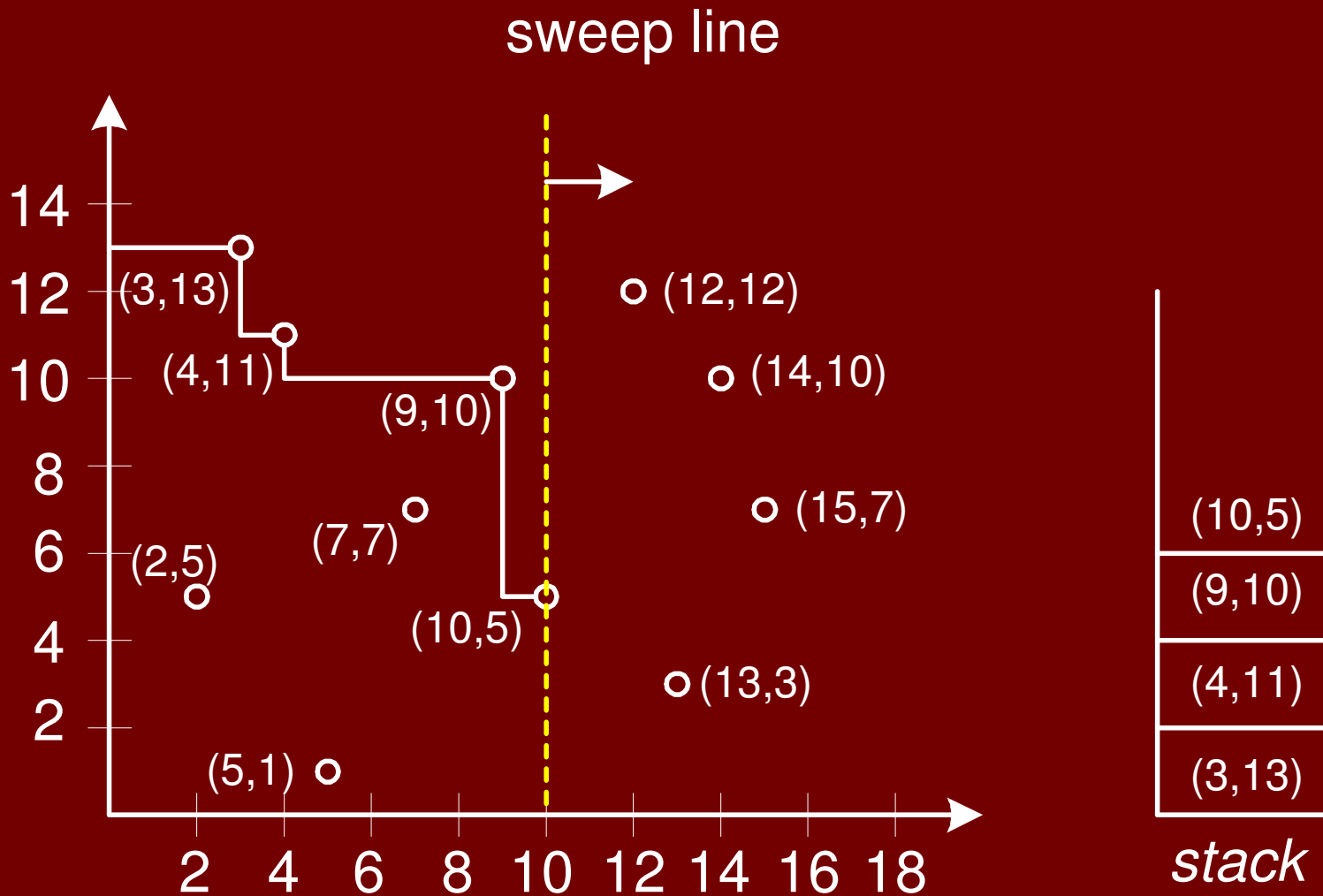
Plane-sweep Algorithm



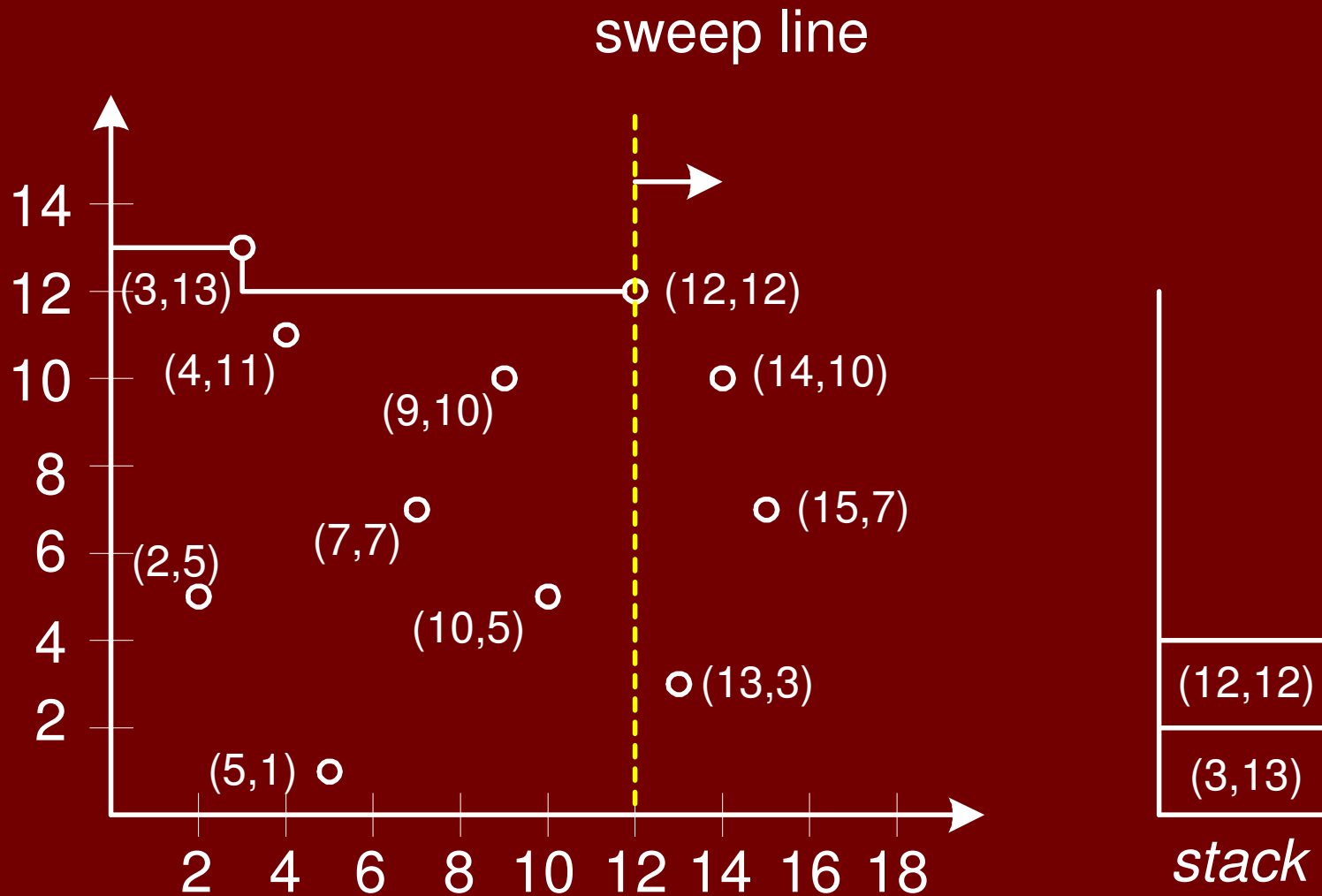
Plane-sweep Algorithm



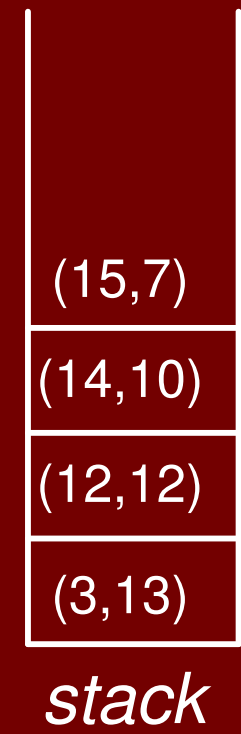
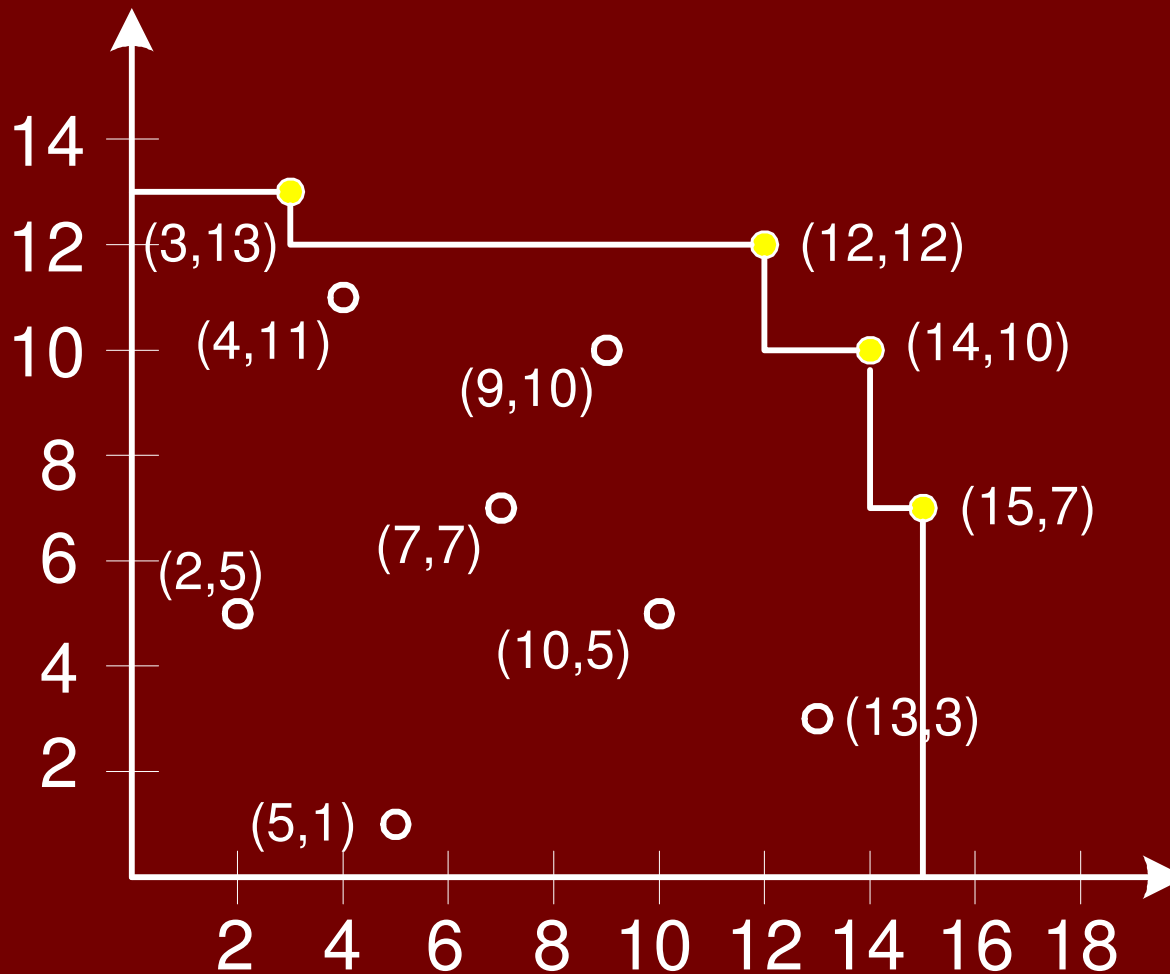
Plane-sweep Algorithm



Plane-sweep Algorithm



Plane-sweep Algorithm



Plane-sweep Algorithm

PLANE-SWEEP-MAXIMA(n , $P[1..n]$)

```
1  sort  $P$  in increasing order by  $x$ ;  
2  stack  $s$ ;  
3  for  $i \leftarrow 1$  to  $n$   
4  do  
5      while ( $s.\text{notEmpty}() \ \& \ s.\text{top}().y \leq P[i].y$ )  
6          do  $s.\text{pop}()$ ;  
7           $s.\text{push}(P[i])$ ;  
8  output the contents of stack  $s$ ;
```

Analysis of Plane-sweep Algorithm

- Sorting takes $\Theta(n \log n)$; we will show this later when we discuss sorting.
- The for loop executes n times.
- The inner loop (seemingly) could be iterated $(n - 1)$ times.
- It seems we still have an $n(n - 1)$ or $\Theta(n^2)$ algorithm.
- Got fooled by simple minded loop-counting.

Analysis of Plane-sweep Algorithm

- The while loop will not execute more n times over the entire course of the algorithm.
- Why is this?
- Observe that the total number of elements that can be pushed on the stack is n since we execute exactly one push each time during the outer for-loop.

Analysis of Plane-sweep Algorithm

- We pop an element off the stack each time we go through the inner while-loop.
- It is impossible to pop more elements than are ever pushed on the stack.
- Therefore, the inner while-loop cannot execute more than n times over the entire course of the algorithm. (*Make sure that you understand this*).

Analysis of Plane-sweep Algorithm

- The for-loop iterates n times and the inner while-loop also iterates n time for a total of $\Theta(n)$.
- Combined with the sorting, the runtime of entire plane-sweep algorithm is $\Theta(n \log n)$.

Comparison

- How much of an improvement is plane-sweep over brute-force?
- Ratio of running times:

$$\frac{n^2}{n \log n} = \frac{n}{\log n}$$

Comparison

n	$\log n$	$\frac{n}{\log n}$
100	7	15
1000	10	100
10000	13	752
100000	17	6021
1000000	20	50171

For $n = 1,000,000$, if plane-sweep takes 1 second, the brute-force will take about 14 hours!.

Asymptotic Notation

- You may be asking that we continue to use the notation $\Theta()$ but have never defined it.
- Let's remedy this now.
- Given any function $g(n)$, we define $\Theta(g(n))$ to be a set of functions that *asymptotically equivalent* to $g(n)$.

Asymptotic Notation

Formally

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$
 $\text{for all } n \geq n_0\}$

Asymptotic Notation

Formally

$$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \\ \text{for all } n \geq n_0\}$$

Your response at this point might be, “I am sorry that I asked!”

Asymptotic Notation

- This is written as “ $f(n) \in \Theta(g(n))$ ”
- That is, $f(n)$ and $g(n)$ are asymptotically equivalent.
- This means that they have essentially the same growth rates for large n .

Asymptotic Notation

For example, functions like

- $4n^2$,
- $(8n^2 + 2n - 3)$,
- $(n^2/5 + \sqrt{n} - 10 \log n)$
- $n(n - 3)$

are all asymptotically equivalent. As n becomes large, the dominant (fastest growing) term is some constant times n^2 .

Asymptotic Notation - Example

- Consider the function

$$f(n) = 8n^2 + 2n - 3$$

- Our informal rule of keeping the largest term and ignoring the constant suggests that $f(n) \in \Theta(n^2)$.
- Let's see why this bears out formally.

Asymptotic Notation - Example

We need to show two things for

$$f(n) = 8n^2 + 2n - 3$$

- **Lower bound:** $f(n) = 8n^2 + 2n - 3$ grows asymptotically at least as fast as n^2 ,
- **Upper bound:** $f(n)$ grows no faster asymptotically than n^2 ,

Asymptotic Notation - Example

Lower bound: $f(n)$ grows asymptotically at least as fast as n^2 .

- For this, need to show that there exist positive constants c_1 and n_0 , such that $f(n) \geq c_1 n^2$ for all $n \geq n_0$.
- Consider the reasoning

$$f(n) = 8n^2 + 2n - 3 \geq 8n^2 - 3 = 7n^2 + (n^2 - 3) \geq 7n^2$$

- Thus $c_1 = 7$.

Asymptotic Notation - Example

- We implicitly assumed that $2n \geq 0$ and $n^2 - 3 \geq 0$
- These are not true for all n but if $n \geq \sqrt{3}$, then both are true.
- So select $n_0 \geq \sqrt{3}$.
- We then have $f(n) \geq c_1 n^2$ for all $n \geq n_0$.

Asymptotic Notation - Example

Upper bound: $f(n)$ grows asymptotically no faster than n^2 .

- For this, we need to show that there exist positive constants c_2 and n_0 , such that $f(n) \leq c_2 n^2$ for all $n \geq n_0$.
- Consider the reasoning

$$f(n) = 8n^2 + 2n - 3 \leq 8n^2 + 2n \leq 8n^2 + 2n^2 = 10n^2$$

- Thus $c_2 = 10$.

Asymptotic Notation - Example

- We implicitly made the assumption that $2n \leq 2n^2$.
- This is not true for all n but it is true for all $n \geq 1$
- So select $n_0 \geq 1$.
- We thus have $f(n) \leq c_2 n^2$ for all $n \geq n_0$.

Asymptotic Notation - Example

- From lower bound we have $n_0 \geq \sqrt{3}$
- From upper bound we have $n_0 \geq 1$.
- Combining the two, we let n_0 be the larger of the two: $n_0 \geq \sqrt{3}$.

Asymptotic Notation - Example

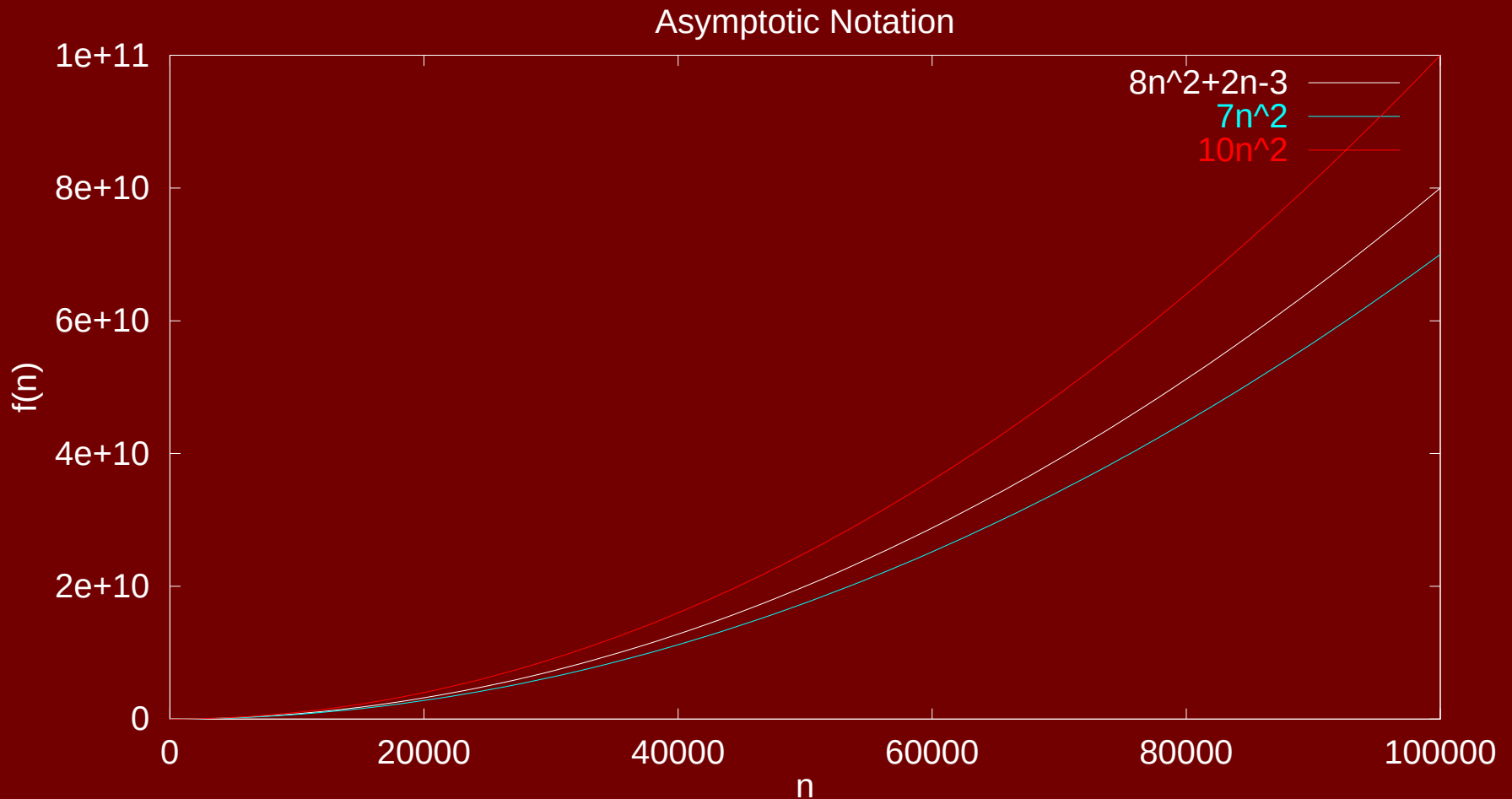
- In conclusion, if we let $c_1 = 7$, $c_2 = 10$ and $n_0 \geq \sqrt{3}$, we have

$$7n^2 \leq 8n^2 + 2n - 3 \leq 10n^2 \quad \text{for all } n \geq \sqrt{3}$$

- We have thus established

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{for all } n \geq n_0$$

Asymptotic Notation - Example



Asymptotic Notation - Example

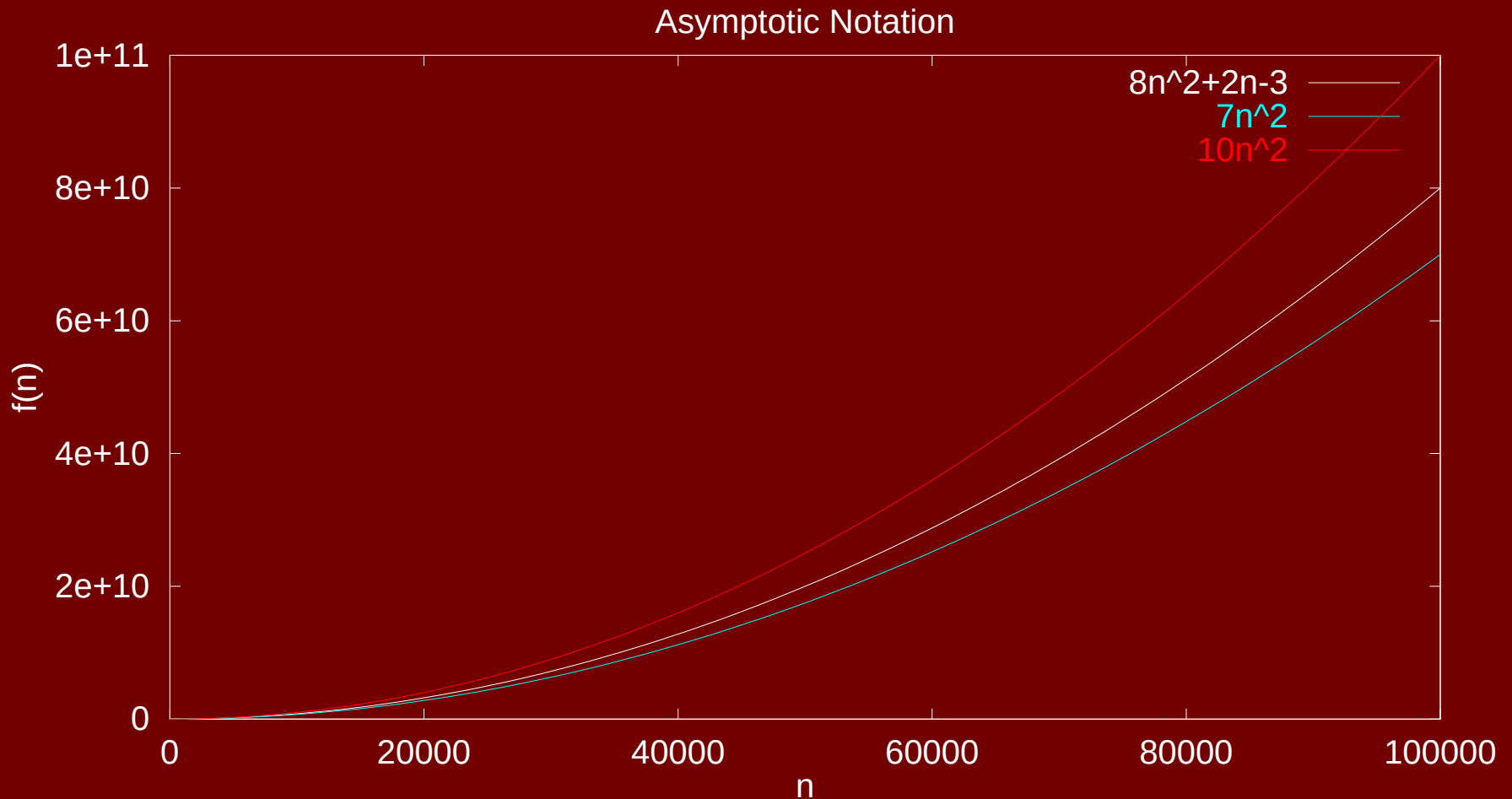
- We have established that $f(n) \in n^2$.
- Let's show why $f(n)$ is not in some other asymptotic class.
- First, let's show that $f(n) \notin \Theta(n)$.

Asymptotic Notation - Example

Show that $f(n) \notin \Theta(n)$.

- If this were true, we would have had to satisfy both the upper and lower bounds.
- The lower bound is satisfied because $f(n) = 8n^2 + 2n - 3$ does grow at least as fast asymptotically as n .
- But the upper bound is false. Upper bounds requires that there exist positive constants c_2 and n_0 such that $f(n) \leq c_2n$ for all $n \geq n_0$.

Asymptotic Notation - Example



Asymptotic Notation - Example

- We have established that $f(n) \in n^2$.
- Let's show why $f(n)$ is not in some other asymptotic class.
- First, let's show that $f(n) \notin \Theta(n)$.

Asymptotic Notation - Example

Show that $f(n) \notin \Theta(n)$.

- If this were true, we would have had to satisfy both the upper and lower bounds.
- The lower bound is satisfied because $f(n) = 8n^2 + 2n - 3$ does grow at least as fast asymptotically as n .
- But the upper bound is false. Upper bounds requires that there exist positive constants c_2 and n_0 such that $f(n) \leq c_2n$ for all $n \geq n_0$.

Asymptotic Notation - Example

- Informally we know that $f(n) = 8n^2 + 2n - 3$ will eventually exceed c_2n no matter how large we make c_2 .
- To see this, suppose we assume that constants c_2 and n_0 did exist such that $8n^2 + 2n - 3 \leq c_2n$ for all $n \geq n_0$
- Since this is true for all sufficiently large n then it must be true in the limit as n tends to infinity.

Asymptotic Notation - Example

- If we divide both sides by n , we have

$$\lim_{n \rightarrow \infty} \left(8n + 2 - \frac{3}{n} \right) \leq c_2.$$

- It is easy to see that in the limit, the left side tends to ∞ .
- So, no matter how large c_2 is, the statement is violated. Thus $f(n) \notin \Theta(n)$.

Asymptotic Notation - Example

Let's show that $f(n) \notin \Theta(n^3)$.

- The idea would be to show that the lower bound $f(n) \geq c_1 n^3$ for all $n \geq n_0$ is violated.
- (c_1 and n_0 are positive constants).
- Informally we know this to be true because any cubic function will overtake a quadratic.

Asymptotic Notation - Example

- If we divide both sides by n^3 :

$$\lim_{n \rightarrow \infty} \left(\frac{8}{n} + \frac{2}{n^2} - \frac{3}{n^3} \right) \geq c_1$$

- The left side tends to 0. The only way to satisfy this is to set $c_1 = 0$.
- But by hypothesis, c_1 is positive.
- This means that $f(n) \notin \Theta(n^3)$.

O-notation and Ω -notation

- The definition of Θ -notation relies on proving both a lower and upper asymptotic bound.
- Sometimes we are only interested in proving one bound or the other.
- The O -notation is used to state only the asymptotic *upper* bounds.
- The Ω -notation allows us to state only the asymptotic *lower* bounds.

O-notation (Big-O)

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n)$
 $\text{for all } n \geq n_0\}$

Ω -notation

$\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n)$
 $\text{for all } n \geq n_0\}$

Θ , O , Ω -notation

The three notations:

$$\Theta(g(n)) : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$O(g(n)) : 0 \leq f(n) \leq c g(n)$$

$$\Omega(g(n)) : 0 \leq c g(n) \leq f(n)$$

for all $n \geq n_0$

Limit Rule

These definitions suggest an alternative way of showing the asymptotic behavior.

Limit rule for Θ -notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c,$$

for some constant $c > 0$ (strictly positive but not infinity) then $f(n) \in \Theta(g(n))$.

Limit Rule

Limit rule for O-notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c,$$

for some constant $c \geq 0$ (nonnegative but not infinite) then $f(n) \in O(g(n))$.

Limit Rule

Limit rule for Ω -notation:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0,$$

(either a strictly positive constant or infinity) then
 $f(n) \in \Omega(g(n))$.

Asymptotic Intuition

Here is a list of common asymptotic running times:

- $\Theta(1)$: Constant time; can't beat it!
- $\Theta(\log n)$: Inserting into a balanced binary tree; time to find an item in a sorted array of length n using binary search.
- $\Theta(n)$: About the fastest that an algorithm can run.
- $\Theta(n \log n)$: Best sorting algorithms.

Asymptotic Intuition

- $\Theta(n^2)$, $\Theta(n^3)$: Polynomial time. These running times are acceptable when the exponent of n is small or n is not too large, e.g., $n \leq 1000$.
- $\Theta(2^n)$, $\Theta(3^n)$: Exponential time. Acceptable only if n is small, e.g., $n \leq 50$.
- $\Theta(n!)$, $\Theta(n^n)$: Acceptable only for really small n , e.g. $n \leq 20$.

Divide and Conquer Strategy

Divide and Conquer

The divide and conquer is a strategy employed to solve a large number of computational problems:

- **Divide:** the problem into a small number of pieces
- **Conquer:** solve each piece by applying divide and conquer to it *recursively*
- **Combine:** the pieces together into a global solution.

Merge Sort

- Divide and conquer strategy is applicable in a huge number of computational problems.
- The first example of divide and conquer algorithm we will discuss is a simple and efficient sorting procedure called *Merge Sort*.

Merge Sort

- We are given a sequence of n numbers A , which we will assume are stored in an array $A[1..n]$.
- The objective is to output a permutation of this sequence sorted in increasing order.
- This is normally done by permuting the elements within the array A .

Merge Sort

Here is how the merge sort algorithm works:

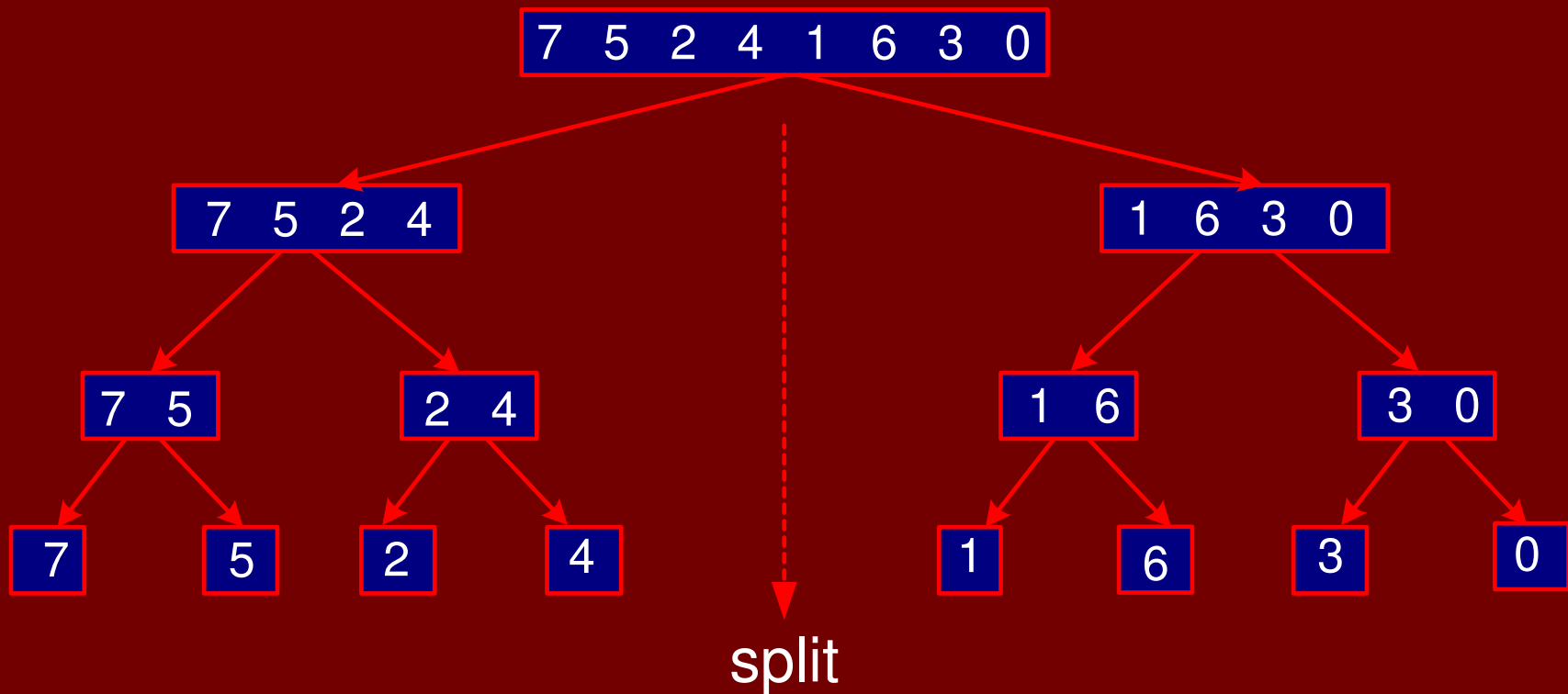
- Divide: split A down the middle into two subsequences, each of size roughly $n/2$
- Conquer: sort each subsequence by calling merge sort recursively on each.
- Combine: merge the two sorted subsequences into a single sorted list.

Merge Sort

- The dividing process ends when we have split the subsequences down to a single item.
- A sequence of length one is trivially sorted.
- The key operation is the combine stage which merges together two sorted lists into a single sorted list.
- Fortunately, the combining process is quite easy to implement.

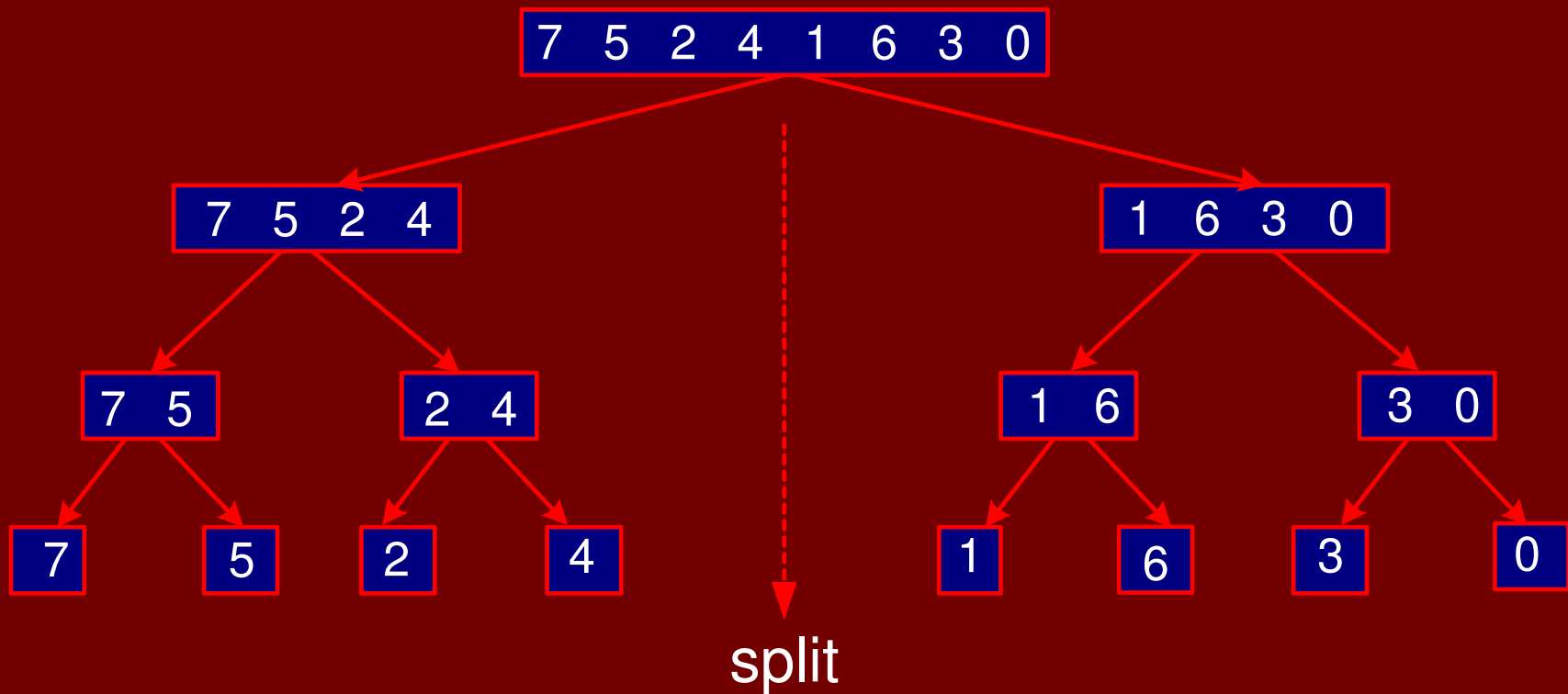
Merge sort

The divide phase: split the list top-down into smaller pieces



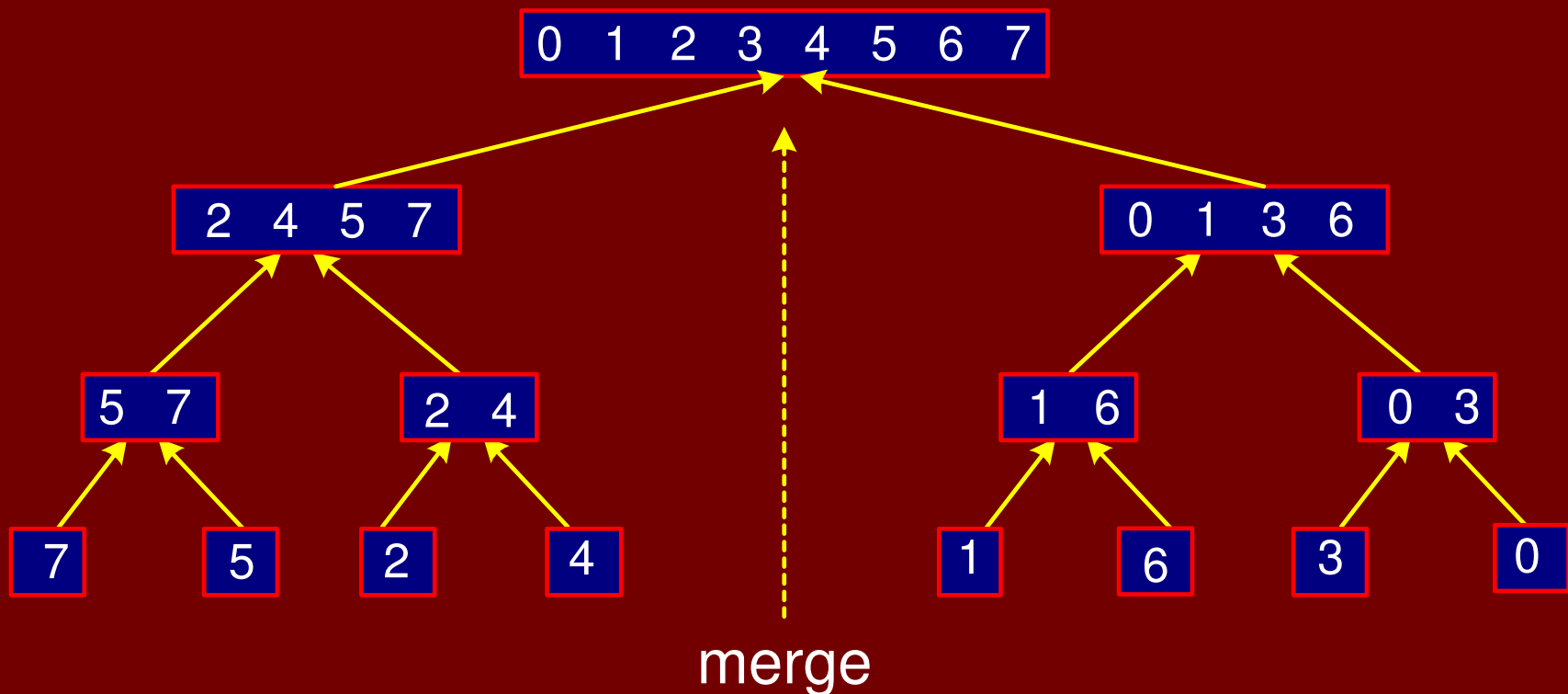
Merge sort

The divide phase: split the list top-down into smaller pieces



Merge sort

The conquer phase: merge the smaller lists bottom up into larger pieces



Merge Sort Algorithm

```
MERGE-SORT( array  $A$ , int  $p$ , int  $r$ )  
1  if ( $p < r$ )  
2    then  
3       $q \leftarrow (p + r)/2$   
4      MERGE-SORT( $A$ ,  $p$ ,  $q$ )      // sort  $A[p..q]$   
5      MERGE-SORT( $A$ ,  $q + 1$ ,  $r$ ) // sort  $A[q + 1..r]$   
6      MERGE( $A$ ,  $p$ ,  $q$ ,  $r$ )      // merge the two pieces
```

Merge Sort Algorithm

MERGE(array A , int p , int q int r)

```
1  int  $B[p..r]$ ; int  $i \leftarrow k \leftarrow p$ ; int  $j \leftarrow q + 1$ 
2  while ( $i \leq q$ ) and ( $j \leq r$ )
3  do if ( $A[i] \leq A[j]$ )
4      then  $B[k++ ] \leftarrow A[i++ ]$ 
5      else  $B[k++ ] \leftarrow A[j++ ]$ 
6  while ( $i \leq q$ )
7  do  $B[k++ ] \leftarrow A[i++ ]$ 
8  while ( $j \leq r$ )
9  do  $B[k++ ] \leftarrow A[j++ ]$ 
10 for  $i \leftarrow p$  to  $r$ 
11 do  $A[i] \leftarrow B[i]$ 
```


Analysis of Merge Sort

- First, consider the running time of the procedure $\text{merge}(A, p, q, r)$.
- Let $n = r - p + 1$ denote the total length of both left and right sub-arrays, i.e., sorted pieces.
- The merge procedure contains four loops, none nested in the other.
- Each loop can be executed at most n times.
- Thus running time of merge is $\Theta(n)$

Analysis of Merge Sort

- Let $T(n)$ denote the worst case running time of MergeSort on an array of length n .
- If we call MergeSort with an array containing a single item ($n = 1$) then the running time is constant.
- We can just write $T(n) = 1$, ignoring all constants.

Analysis of Merge Sort

- For $n > 1$, MergeSort splits into two halves, sorts the two and then merges them together.
- The left half is of size $\lceil n/2 \rceil$ and the right half is $\lfloor n/2 \rfloor$.
- How long does it take to sort elements in sub array of size $\lceil n/2 \rceil$?
- We do not know this but because $\lceil n/2 \rceil < n$ for $n > 1$, we can express this as $T(\lceil n/2 \rceil)$.

Analysis of Merge Sort

- Similarly the time taken to sort right sub array is expressed as $T(\lfloor n/2 \rfloor)$.
- In conclusion we have

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n & \text{otherwise} \end{cases}$$

- This is called *recurrence relation*, i.e., a recursively defined function.

Solving the Recurrence

Let's expand the terms.

$$T(1) = 1$$

$$T(2) = T(1) + T(1) + 2 = 1 + 1 + 2 = 4$$

$$T(3) = T(2) + T(1) + 3 = 4 + 1 + 3 = 8$$

$$T(4) = T(2) + T(2) + 4 = 8 + 8 + 4 = 12$$

$$T(5) = T(3) + T(2) + 5 = 8 + 4 + 5 = 17$$

...

Solving the Recurrence

...

$$T(8) = T(4) + T(4) + 8 = 12 + 12 + 8 = 32$$

...

$$T(16) = T(8) + T(8) + 16 = 32 + 32 + 16 = 80$$

...

$$T(32) = T(16) + T(16) + 32 = 80 + 80 + 32 = 192$$

Solving the Recurrence

What is the pattern here?

- Let's consider the ratios $T(n)/n$ for powers of 2:

$$T(1)/1 = 1 \qquad T(8)/8 = 4$$

$$T(2)/2 = 2 \qquad T(16)/16 = 5$$

$$T(4)/4 = 3 \qquad T(32)/32 = 6$$

- This suggests $T(n)/n = \log n + 1$
- Or, $T(n) = n \log n + n$ which is $\Theta(n \log n)$ (use the limit rule).

Solving the Recurrence

What is the pattern here?

- Let's consider the ratios $T(n)/n$ for powers of 2:

$$T(1)/1 = 1 \qquad T(8)/8 = 4$$

$$T(2)/2 = 2 \qquad T(16)/16 = 5$$

$$T(4)/4 = 3 \qquad T(32)/32 = 6$$

- This suggests $T(n)/n = \log n + 1$
- Or, $T(n) = n \log n + n$ which is $\Theta(n \log n)$ (use the limit rule).

Eliminating Floor and Ceiling

- Floor and ceilings are a pain to deal with.
- If n is assumed to be a power of 2 ($2^k = n$), this will simplify the recurrence to

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{otherwise} \end{cases}$$

The Iteration Method

- The iteration method turns the recurrence into a summation.
- Let's see how it works.

The Iteration Method

Let's expand the recurrence:

$$\begin{aligned}T(n) &= 2T(n/2) + n \\&= 2(2T(n/4) + n/2) + n \\&= 4T(n/4) + n + n \\&= 4(2T(n/8) + n/4) + n + n \\&= 8T(n/8) + n + n + n \\&= 8(2T(n/16) + n/8) + n + n + n \\&= 16T(n/16) + n + n + n + n\end{aligned}$$

The Iteration Method

If n is a power of 2 then let $n = 2^k$ or $k = \log n$.

$$T(n) = 2^k T(n/(2^k)) + \underbrace{(n + n + n + \cdots + n)}_{k \text{ times}}$$

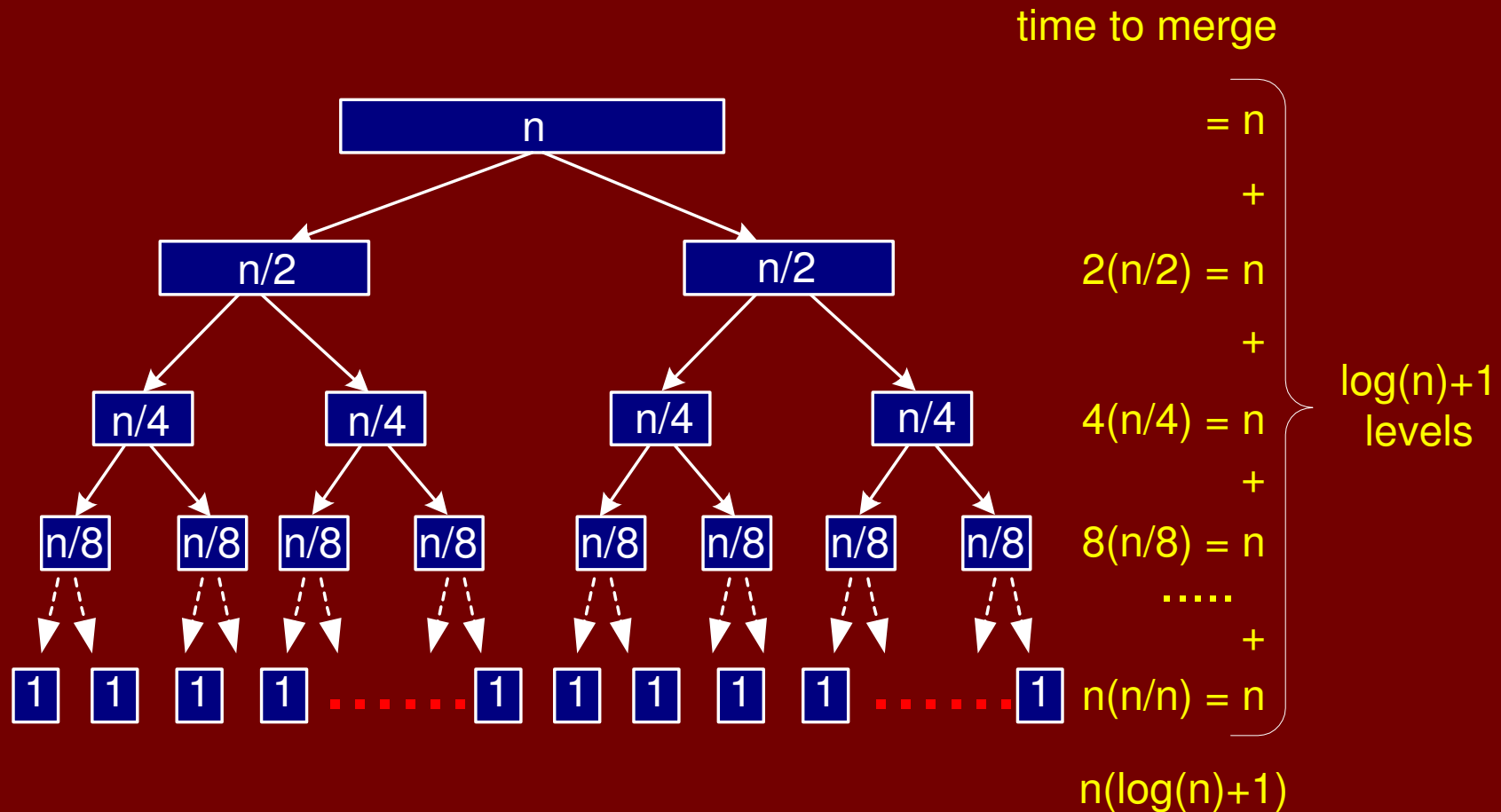
$$= 2^k T(n/(2^k)) + kn$$

$$= 2^{(\log n)} T(n/(2^{(\log n)})) + (\log n)n$$

$$= 2^{(\log n)} T(n/n) + (\log n)n$$

$$= nT(1) + n \log n = n + n \log n$$

Mergesort Recurrence Tree



A Messier Example

- The merge sort recurrence was too easy.
- Let's try a messier recurrence.

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 3T(n/4) + n & \text{otherwise} \end{cases}$$

- Assume n to be a power of 4, i.e., $n = 4^k$ and $k = \log_4 n$

The Iteration Method

$$\begin{aligned}T(n) &= 3T(n/4) + n \\&= 3(3T(n/16) + (n/4)) + n \\&= 9T(n/16) + 3(n/4) + n \\&= 27T(n/64) + 9(n/16) + 3(n/4) + n \\&= \dots\end{aligned}$$

The Iteration Method

$$T(n) = 3^k T\left(\frac{n}{4^k}\right) + 3^{k-1} (n/4^{k-1}) \\ + \dots + 9(n/16) + 3(n/4) + n$$

$$= 3^k T\left(\frac{n}{4^k}\right) + \sum_{i=0}^{k-1} \frac{3^i}{4^i} n$$

The Iteration Method

With $n = 4^k$ and $T(1) = 1$

$$\begin{aligned}T(n) &= 3^k T\left(\frac{n}{4^k}\right) + \sum_{i=0}^{k-1} \frac{3^i}{4^i} n \\&= 3^{\log_4 n} T(1) + \sum_{i=0}^{(\log_4 n)-1} \frac{3^i}{4^i} n \\&= n^{\log_4 3} + \sum_{i=0}^{(\log_4 n)-1} \frac{3^i}{4^i} n\end{aligned}$$

We used the formula $a^{\log_b n} = n^{\log_b a}$.

The Iteration Method

n remains constant throughout the sum and $3^i/4^i = (3/4)^i$; we thus have

$$T(n) = n^{\log_4 3} + n \sum_{i=0}^{(\log_4 n)-1} \left(\frac{3}{4}\right)^i$$

The sum is a geometric series; recall that for $x \neq 1$

$$\sum_{i=0}^m x^i = \frac{x^{m+1} - 1}{x - 1}$$

The Iteration Method

In this case $x = 3/4$ and $m = \log_4 n - 1$. We get

$$T(n) = n^{\log_4 3} + n \frac{(3/4)^{\log_4 n + 1} - 1}{(3/4) - 1}$$

Applying the log identity once more

$$\begin{aligned} (3/4)^{\log_4 n} &= n^{\log_4 (3/4)} = n^{\log_4 3 - \log_4 4} \\ &= n^{\log_4 3 - 1} = \frac{n^{\log_4 3}}{n} \end{aligned}$$

The Iteration Method

If we plug this back, we get

$$\begin{aligned}T(n) &= n^{\log_4 3} + n \frac{n^{\log_4 3} - 1}{(3/4) - 1} \\&= n^{\log_4 3} + \frac{n^{\log_4 3} - n}{-1/4} \\&= n^{\log_4 3} + 4(n - n^{\log_4 3}) \\&= 4n - 3n^{\log_4 3}\end{aligned}$$

The Iteration Method

With $\log_4 3 \approx 0.79$, we finally have the result!

$$T(n) = 4n - 3n^{\log_4 3} \approx 4n - 3n^{0.79} \in \Theta(n)$$

Selection Problem

- Suppose we are given a set of n numbers.
- Define the *rank* of an element to be one plus the number of elements that are smaller.
- Thus, the rank of an element is its final position if the set is sorted.
- The minimum is of rank 1 and the maximum is of rank n .

Medians and Selection

- Consider the set: {5, 7, 2, 10, 8, 15, 21, 37, 41}.
- The rank of each number is its position in the sorted order.

<i>position</i>	1	2	3	4	5	6	7	8	9
Number	2	5	7	8	10	15	21	37	41

- For example, rank of 8 is 4, one plus the number of elements smaller than 8 which is 3.

The Selection Problem

Given a set A of n distinct numbers and an integer k , $1 \leq k \leq n$, output the element of A of rank k .

Median

- Of particular interest in statistics is the *median*.
- If n is odd then the median is defined to be element of rank $(n + 1)/2$.
- When n is even, there are two choices: $n/2$ and $(n + 1)/2$.
- In statistics, it is common to return the average of the two elements.

Median

- Medians are useful as a measures of the *central tendency* of a set especially when the distribution of values is highly skewed.
- For example, the median income in a community is a more meaningful measure than average.
- Suppose 7 households have monthly incomes 5000, 7000, 2000, 10000, 8000, 15000 and 16000.

Median

- In sorted order, the incomes are 2000, 5000, 7000, 8000, 10000, 15000, 16000.
- The median income is 8000; median is element with rank 4: $(7 + 1)/2 = 4$.
- The average income is 9000.

Median

- Suppose the income 16000 goes up to 450,000.
- The median is still 8000 but the average goes up to 71,000.
- Clearly, the average is not a good representative of the majority income levels.

Medians and Selection

- The selection problem can be easily solved by simply sorting the numbers of A and returning $A[k]$.
- Sorting, however, requires $\Theta(n \log n)$ time.
- The question is: can we do better than that?
- In particular, is it possible to solve the selections problem in $\Theta(n)$ time?
- The answer is yes. However, the solution is far from obvious.

Medians and Selection

- We will use a *variation* of the divide and conquer strategy called the *sieve technique*.
- In divide and conquer, we break the problem into a small number of smaller subproblems which we solve recursively.

Medians and Selection

- In the selection problem, we are looking for an item.
- We will divide the problem into subproblems.
- However, we will discard those small subproblems for which we determine that they do not contain the desired answer (the *variation*).

Medians and Selection

Here is how the sieve technique will be applied to the selection problem.

- We will begin with the given array $A[1..n]$.
- We will pick an item from the array, called the *pivot element* which we will denote by x .
- We will talk about how an item is chosen as the pivot later; for now just think of it as a random element of A .

Medians and Selection

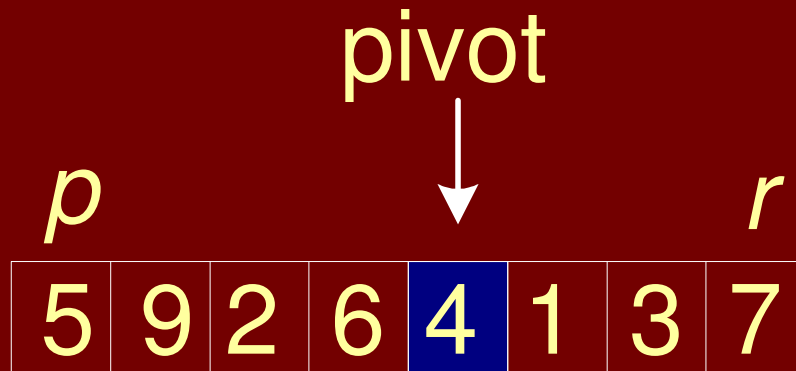
We partition A into three parts.

1. $A[q]$ contains the pivot element x ,
2. $A[1..q - 1]$ will contain all the elements that are less than x and
3. $A[q + 1..n]$ will contains all elements that are greater than x .

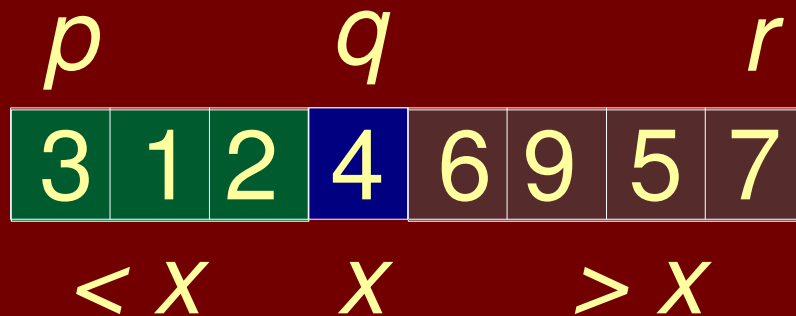
Within each sub array, the items may appear in any order.

Partitioning

$A[p..r]$ partitioned about the pivot x .



Before partitioning



After partitioning

Medians and Selection

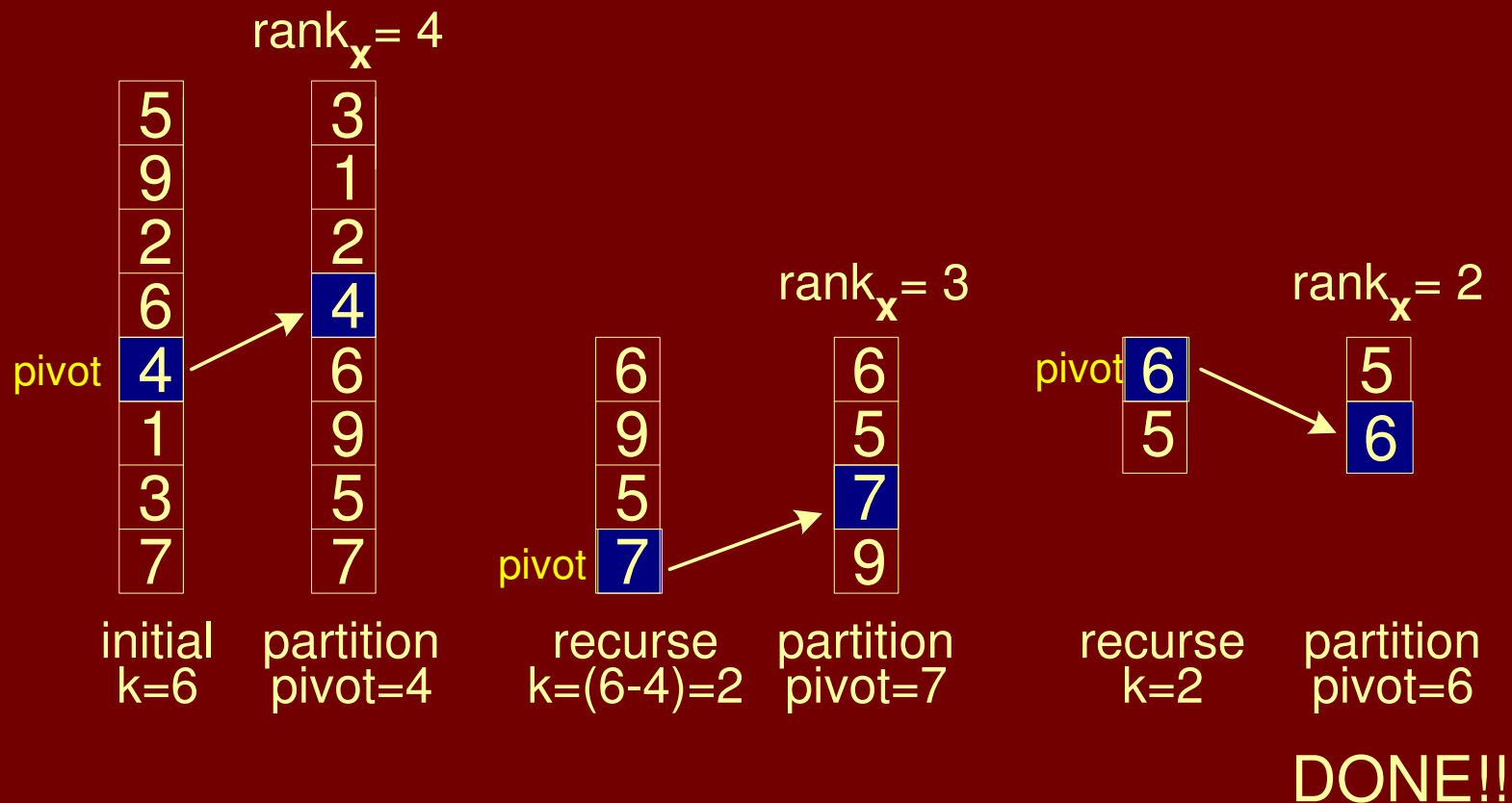
- The rank of the pivot x is $q - p + 1$ in $A[p..r]$.
- Let $\text{rank_}x = q - p + 1$.
- If $k = \text{rank_}x$ then the pivot is k^{th} smallest.
- If $k < \text{rank_}x$ then search $A[p..q - 1]$ recursively.
- If $k > \text{rank_}x$ then search $A[q + 1..r]$ recursively. Find element of rank $(k - q)$ because we eliminated q smaller elements in A .

Select Algorithm

```
SELECT( array  $A$ , int  $p$ , int  $r$ , int  $k$ )  
  1  if ( $p = r$ )  
  2    then return  $A[p]$   
  3  else  $x \leftarrow \text{CHOOSE\_PIVOT}(A, p, r)$   
  4     $q \leftarrow \text{PARTITION}(A, p, r, x)$   
  5     $\text{rank\_}x \leftarrow q - p + 1$   
  6    if  $k = \text{rank\_}x$   
  7      then return  $x$   
  8    if  $k < \text{rank\_}x$   
  9      then return SELECT( $A, p, q - 1, k$ )  
 10   else return SELECT( $A, q + 1, r, k - q$ )
```

Select Algorithm

Example: select the 6th smallest element of the set {5, 9, 2, 6, 4, 1, 3, 7}

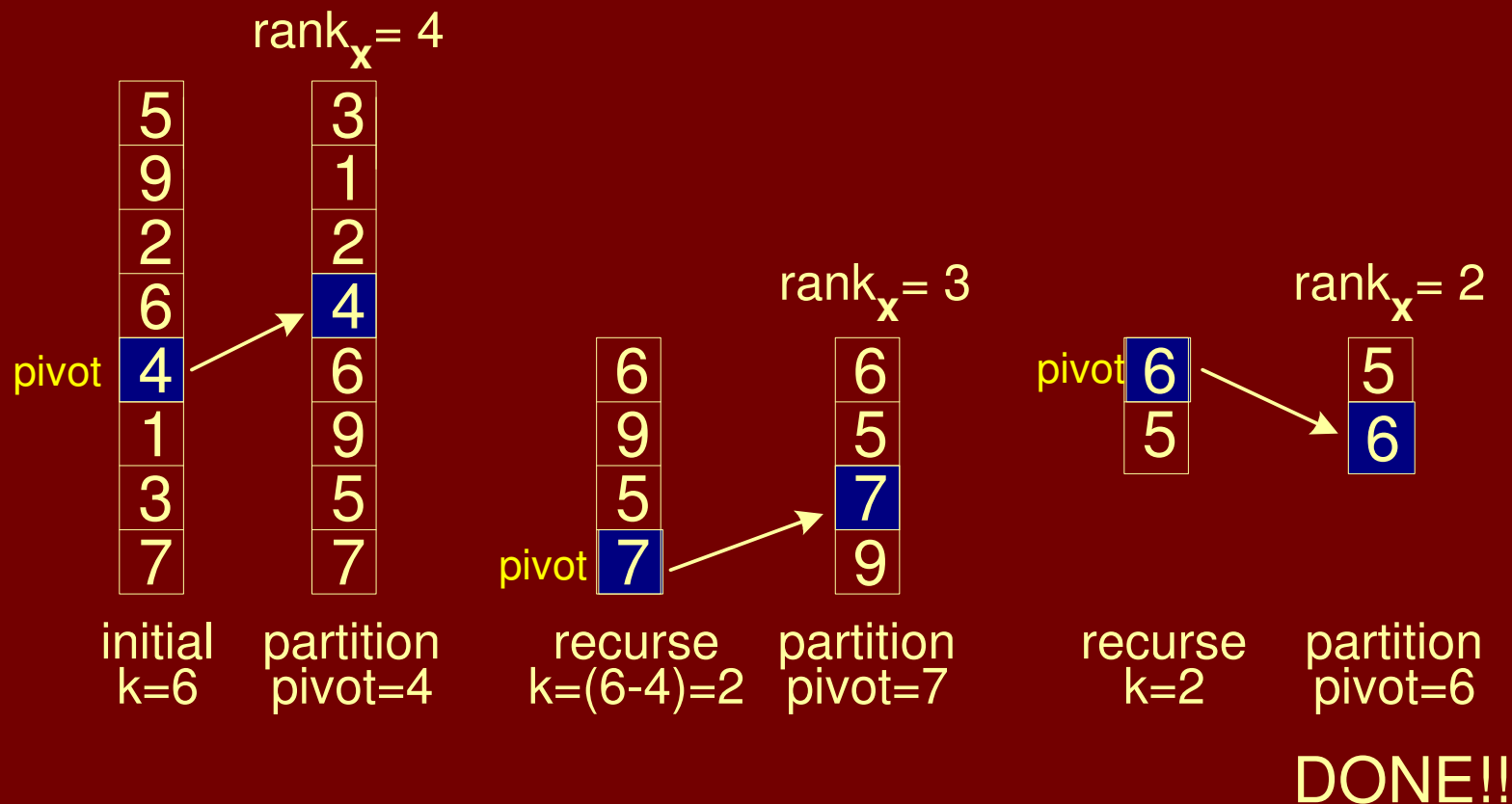


Select Algorithm

```
SELECT( array  $A$ , int  $p$ , int  $r$ , int  $k$ )  
1  if ( $p = r$ )  
2    then return  $A[p]$   
3  else  $x \leftarrow \text{CHOOSE\_PIVOT}(A, p, r)$   
4       $q \leftarrow \text{PARTITION}(A, p, r, x)$   
5       $\text{rank\_}x \leftarrow q - p + 1$   
6      if  $k = \text{rank\_}x$   
7        then return  $x$   
8      if  $k < \text{rank\_}x$   
9        then return SELECT( $A, p, q - 1, k$ )  
10     else return SELECT( $A, q + 1, r, k - q$ )
```

Select Algorithm

Example: select the 6th smallest element of the set {5, 9, 2, 6, 4, 1, 3, 7}



Medians and Selection

- We will discuss how to choose a pivot and the partitioning later.
- For the moment, we will assume that they both take $\Theta(n)$ time.
- How many elements do we eliminate in each time?

Medians and Selection

- If x is the largest or the smallest then we may only succeed in eliminating one element.

5, 9, 2, 6, 4, 1, 3, 7 pivot is 1

1, 5, 9, 2, 6, 4, 3, 7 after partition

- Ideally, x should have a rank that is neither too large or too small.

Analysis of Selection

- Suppose we are able to choose a pivot that causes exactly half of the array to be eliminated in each phase.
- This means that we recurse on the remaining $n/2$ elements.
- This leads to the following recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n/2) + n & \text{otherwise} \end{cases}$$

Analysis of Selection

If we expand this recurrence, we get

$$\begin{aligned} T(n) &= n + \frac{n}{2} + \frac{n}{4} + \dots \\ &\leq \sum_{i=0}^{\infty} \frac{n}{2^i} \\ &= n \sum_{i=0}^{\infty} \frac{1}{2^i} \end{aligned}$$

Analysis of Selection

- Recall the formula for infinite geometric series; for any $|c| < 1$,

$$\sum_{i=0}^{\infty} c^i = \frac{1}{1-c}$$

- Using this we have

$$T(n) \leq 2n \in \Theta(n)$$

Analysis of Selection

- Let's think about how we ended up with a $\Theta(n)$ algorithm for selection.
- Normally, a $\Theta(n)$ time algorithm would make a single or perhaps a constant number of passes of the data set.
- In this algorithm, we make a number of passes. In fact it could be as many as $\log n$.

Analysis of Selection

- However, because we eliminate a constant fraction of the array with each phase, we get the convergent geometric series in the analysis.
- This shows that the total running time is indeed *linear* in n .
- This lesson is well worth remembering.
- It is often possible to achieve linear running times in ways that you would not expect.



Sorting



Sorting

- For the next series of lectures, we will focus on sorting.
- There are a number of reasons for sorting. Here are a few important ones.
- Procedures for sorting are parts of many large software systems.
- Design of efficient sorting algorithms is necessary to achieve overall efficiency of these systems.

Sorting

- Sorting is well studied problem from the analysis point of view.
- Sorting is one of the few problems where provable lower bounds exist on how fast we can sort.
- In sorting, we are given an array $A[1..n]$ of n numbers
- We are to reorder these elements into increasing (or decreasing) order.

Sorting

- More generally, A is an array of objects and we sort them based on one of the attributes - the *key value*.
- The key value need not be a number. It can be any object from a *totally ordered domain*.
- Totally ordered domain means that for any two elements of the domain, x and y , either $x < y$, $x = y$ or $x > y$.

Slow Sorting Algorithms

Bubble sort

- Scan the array.
- Whenever two consecutive items are found that are out of order, swap them.
- Repeat until all consecutive items are in order.

Slow Sorting Algorithms

Insertion sort

- Assume that $A[1..i - 1]$ have already been sorted.
- Insert $A[i]$ into its proper position in this sub array.
- Create this position by shifting all larger elements to the right.

Slow Sorting Algorithms

Selection sort

- Assume that $A[1..i - 1]$ contain the $i - 1$ smallest elements in sorted order.
- Find the smallest element in $A[i..n]$
- Swap it with $A[i]$.

Slow Sorting Algorithms

- These algorithms are easy to implement.
- But they run in $\Theta(n^2)$ time in the worst case.

$n \log n$ **Sorting**

- We have already seen that Mergesort sorts an array of numbers in $\Theta(n \log n)$ time.
- We will study two others: *Heapsort* and *Quicksort*.

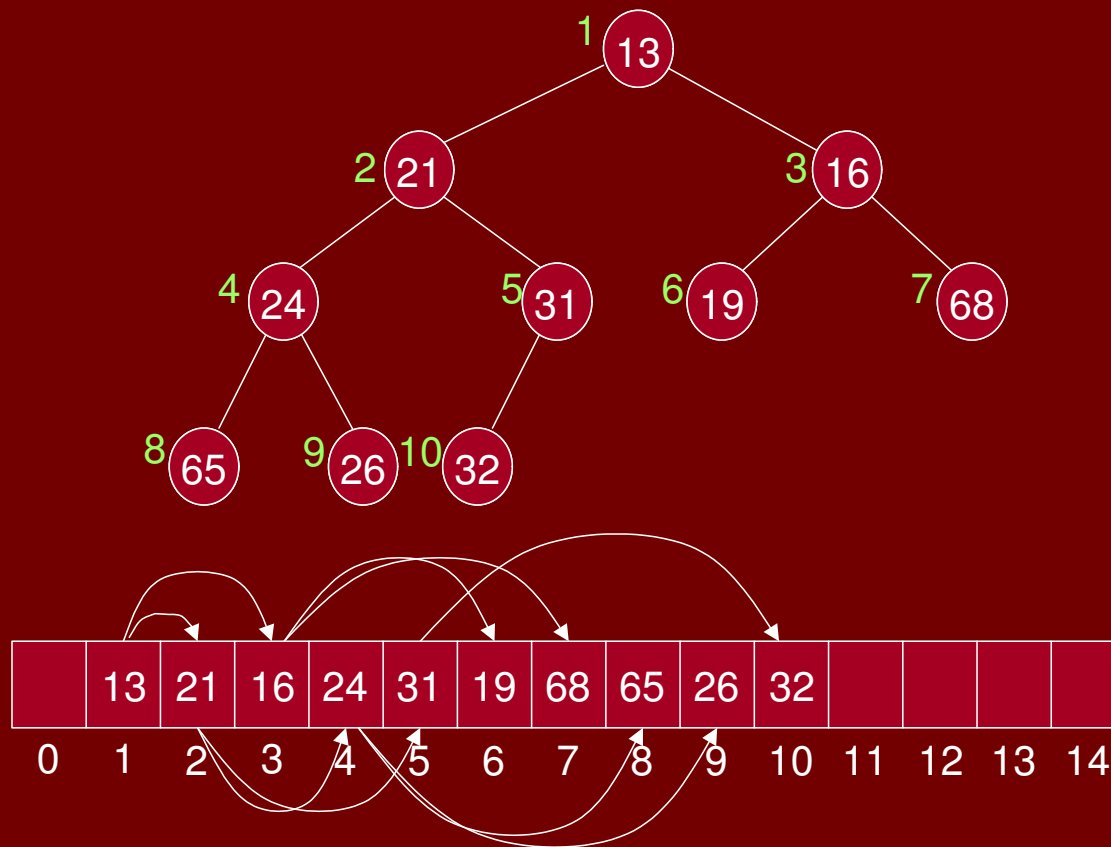
Heap

- A *heap* is a left-complete binary tree that conforms to the *heap order*.
- The heap order property: in a (min) heap, for every node X , the key in the parent is smaller than or equal to the key in X .
- In other words, the parent node has key smaller than or equal to both of its children nodes.

Heap

- Similarly, in a max heap, the parent has a key larger than or equal both of its children
- Thus the smallest key is in the root in a min heap; in the max heap, the largest is in the root.

Heap



Heap

- The number of nodes in a complete binary tree of height h is

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^h = \sum_{i=0}^h 2^i = 2^{h+1} - 1$$

- h in terms of n is

$$h = (\log(n + 1)) - 1 \approx \log n \in \Theta(\log n)$$

Heap

- One of the clever aspects of heaps is that they can be stored in arrays without using any pointers.
- This is due to the left-complete nature of the binary tree.
- We store the tree nodes in level-order traversal.

Heap

Access to nodes involves simple arithmetic operations:

- $\text{left}(i)$: returns $2i$, index of left child of node i .
- $\text{right}(i)$: returns $2i + 1$, the right child.
- $\text{parent}(i)$ returns $\lfloor i/2 \rfloor$, the parent of i .

The root is at position 1 of the array.

Heapsort Algorithm

- We build a max heap out of the given array of numbers $A[1..n]$.
- We repeatedly extract the the maximum item from the heap.
- Once the max item is removed, we are left with a hole at the root.
- To fix this, we will replace it with the last leaf in tree.
- But now the heap order will very likely be destroyed.
- We will apply a heapify procedure to the root to restore the heap.

Heap Sort

HEAPSORT(array A , int n)

1 BUILD-HEAP(A , n)

2 $m \leftarrow n$

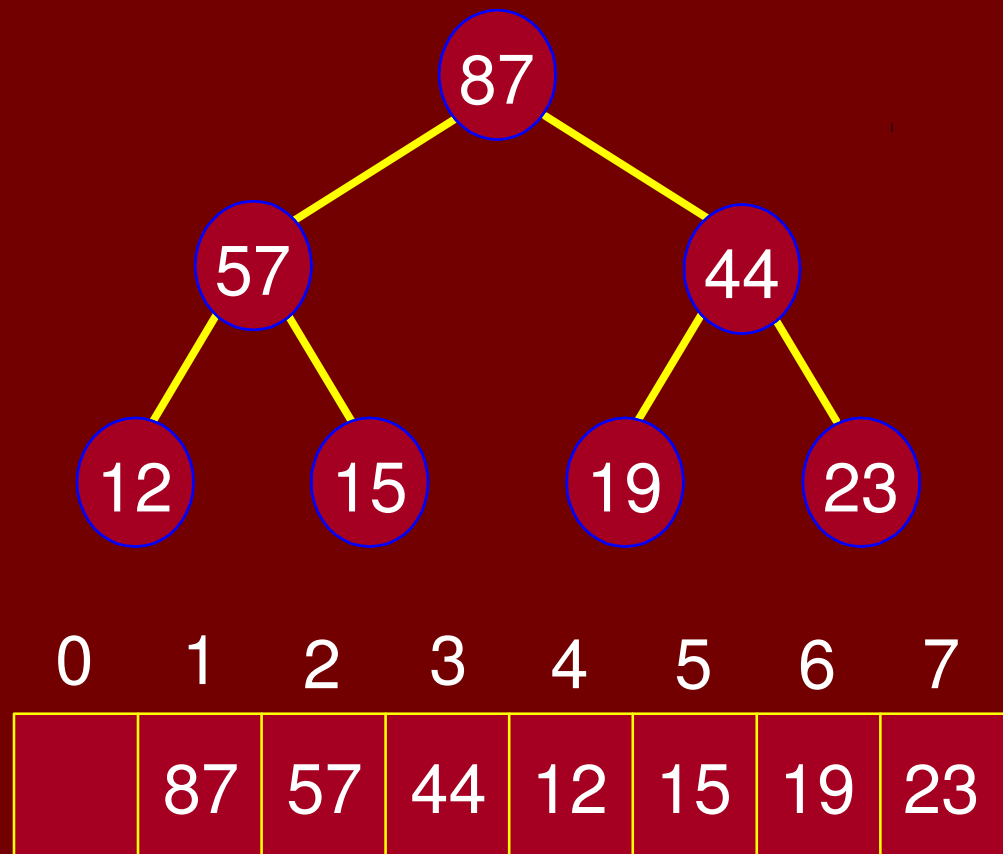
3 **while** ($m \geq 2$)

4 **do** SWAP($A[1]$, $A[m]$)

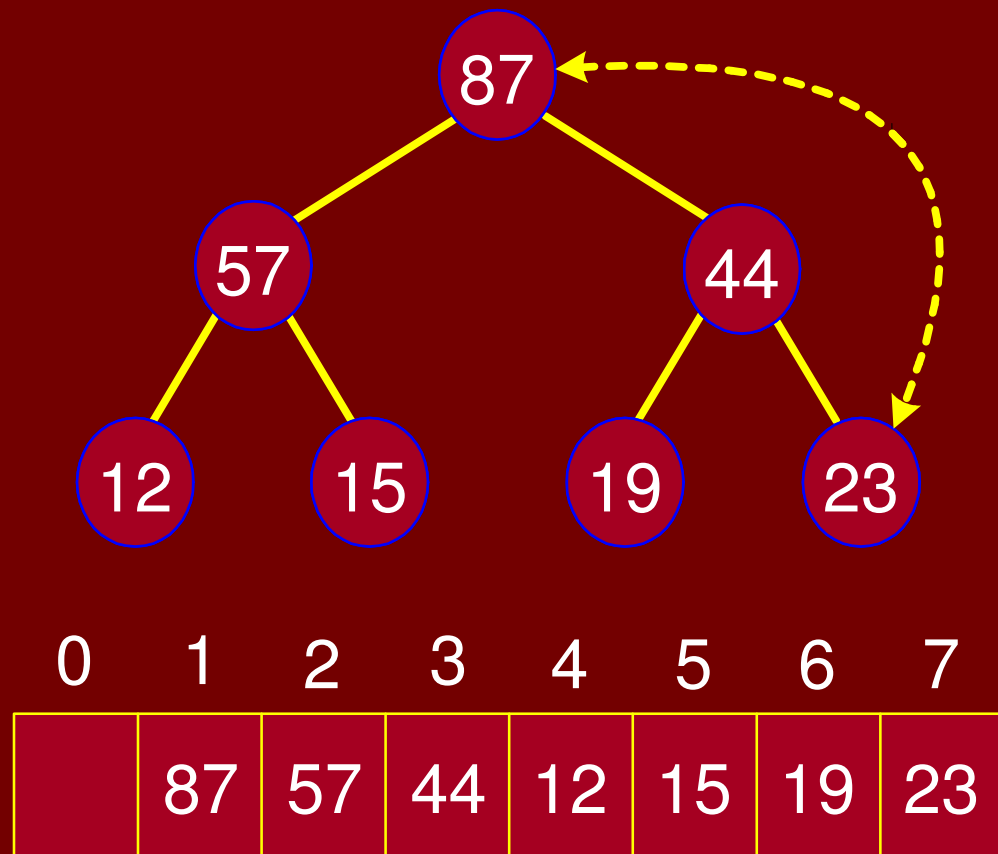
5 $m \leftarrow m - 1$

6 HEAPIFY(A , 1, m)

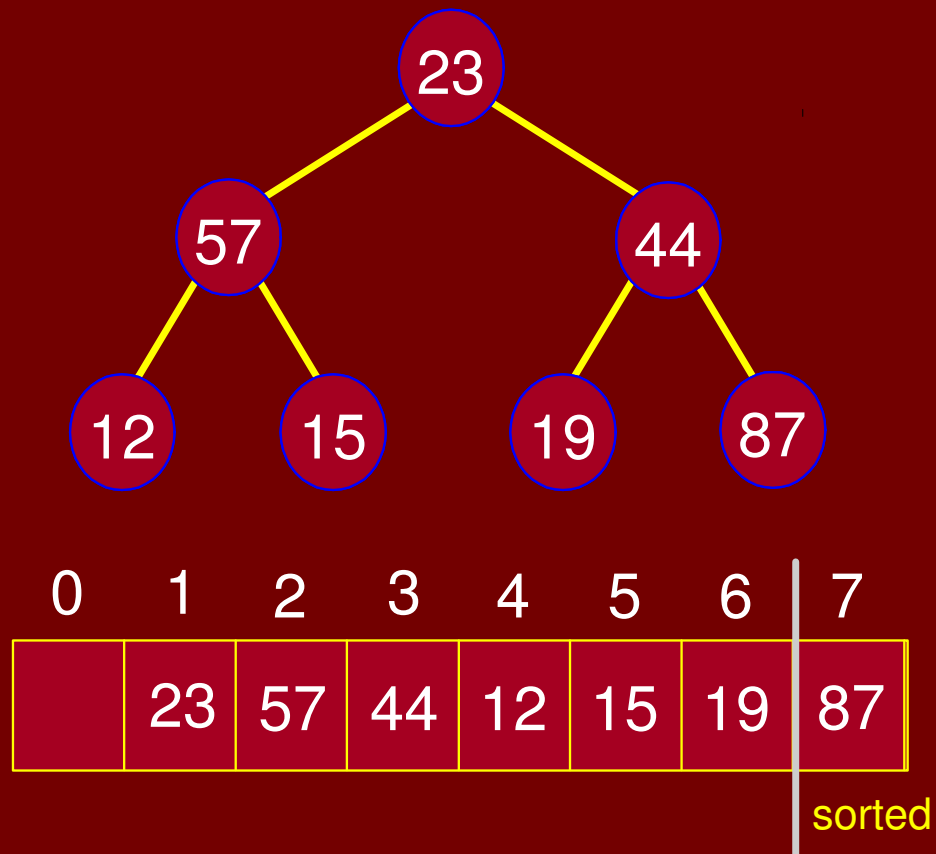
Heapsort Trace



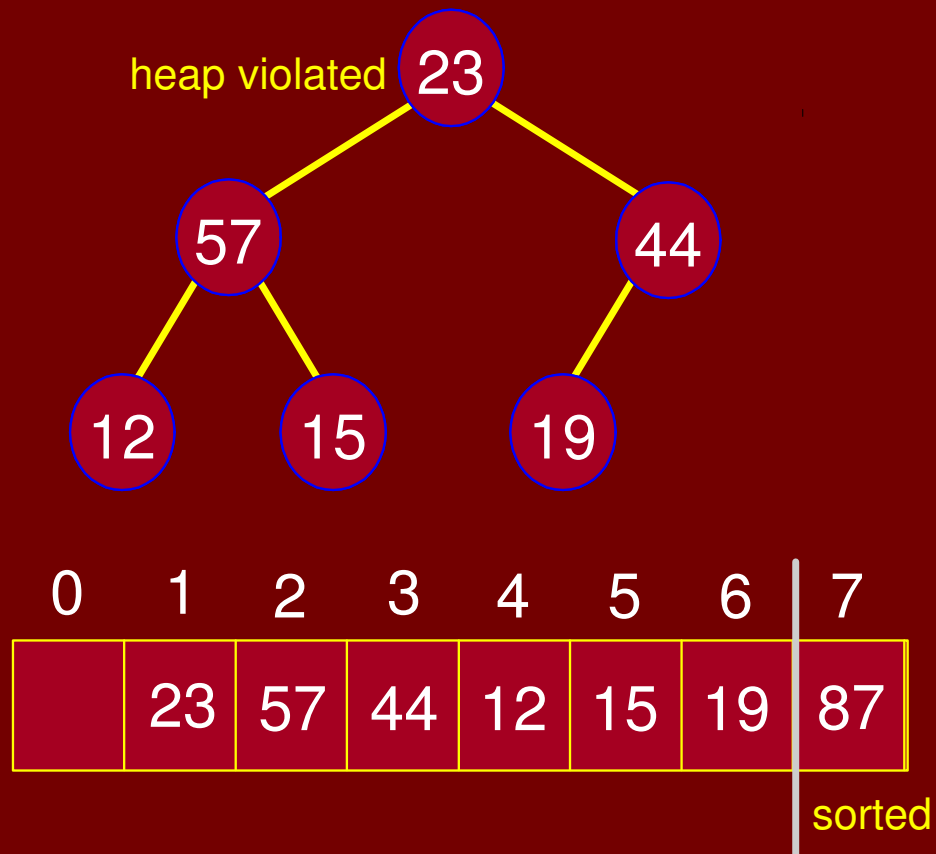
Heapsort Trace



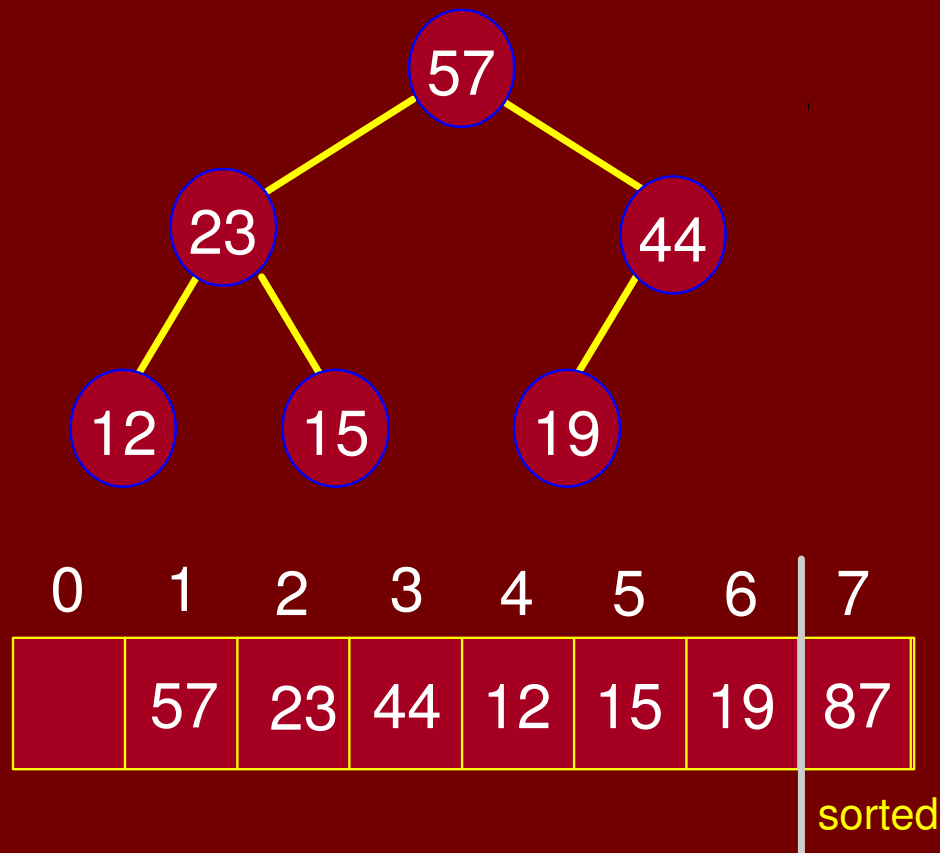
Heapsort Trace



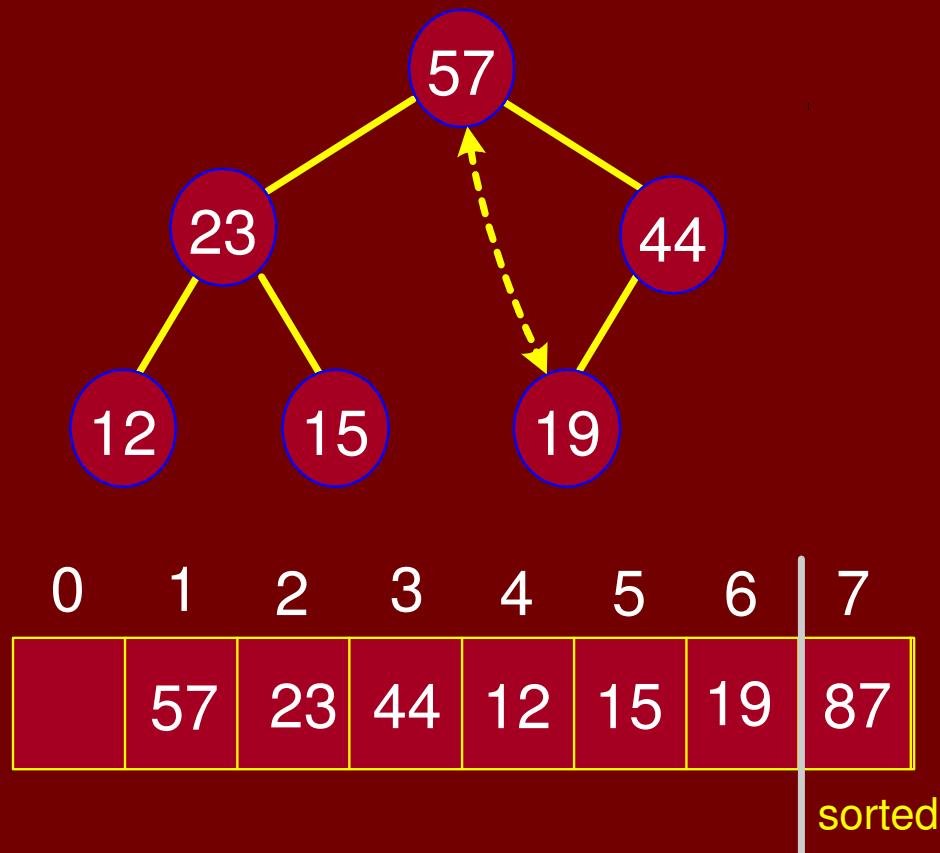
Heapsort Trace



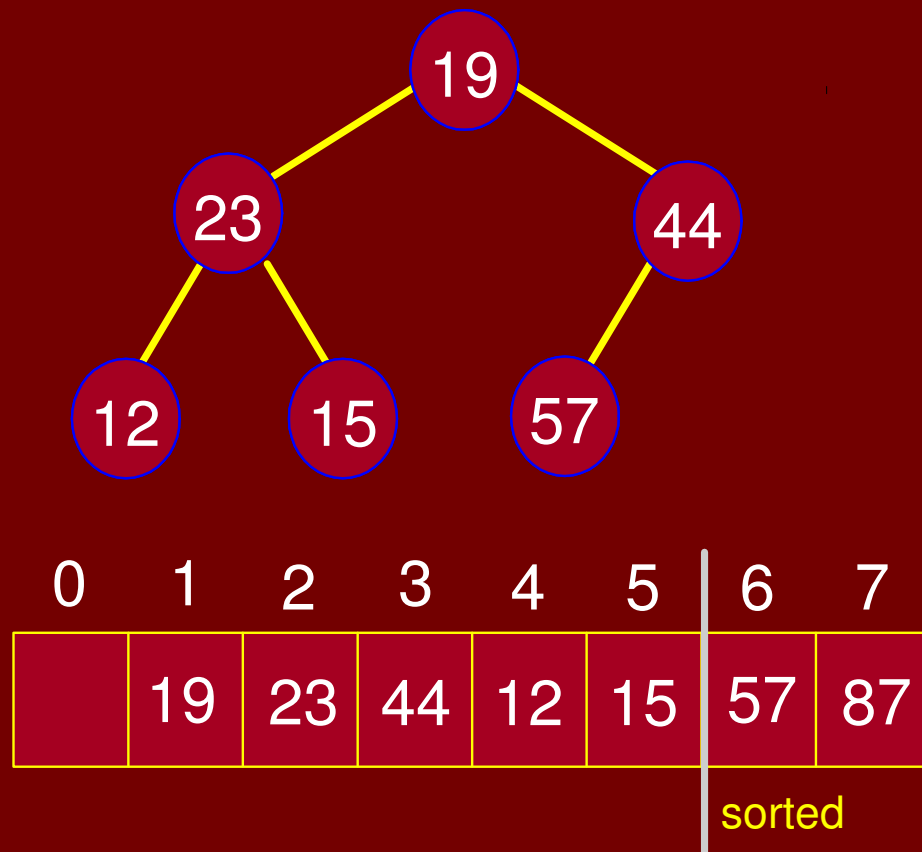
Heapsort Trace



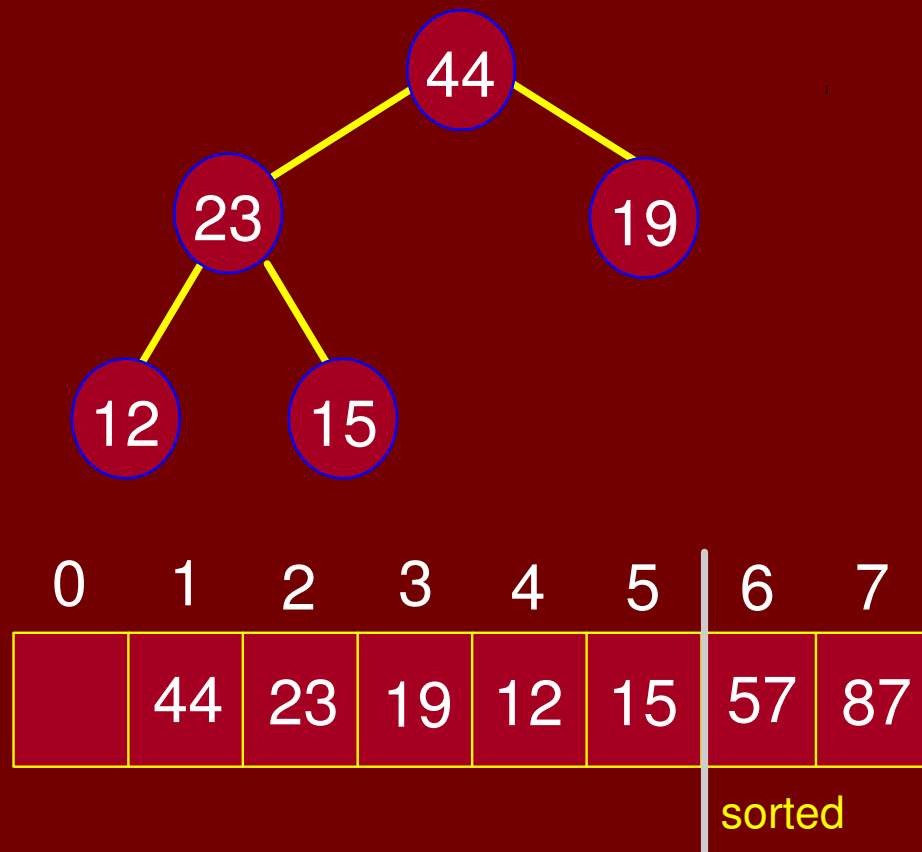
Heapsort Trace



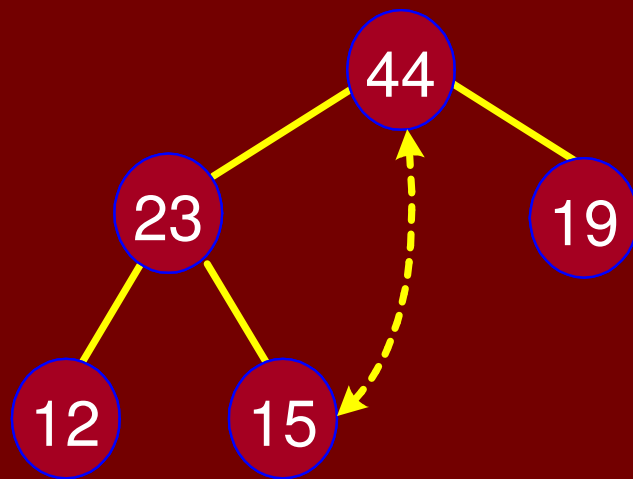
Heapsort Trace



Heapsort Trace



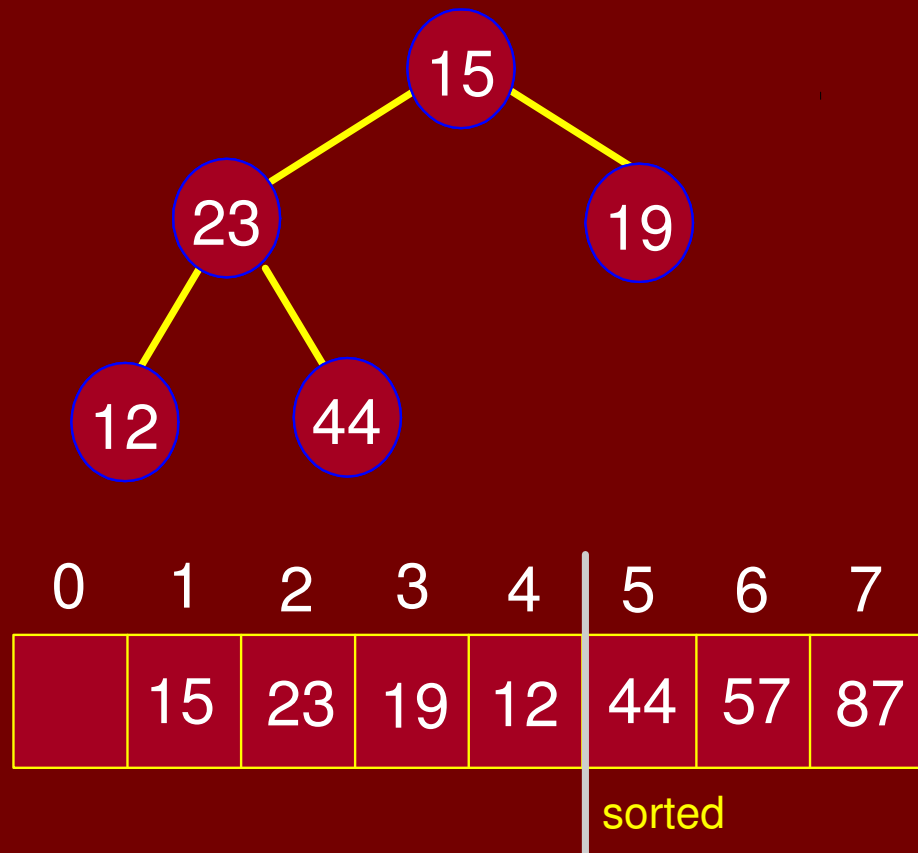
Heapsort Trace



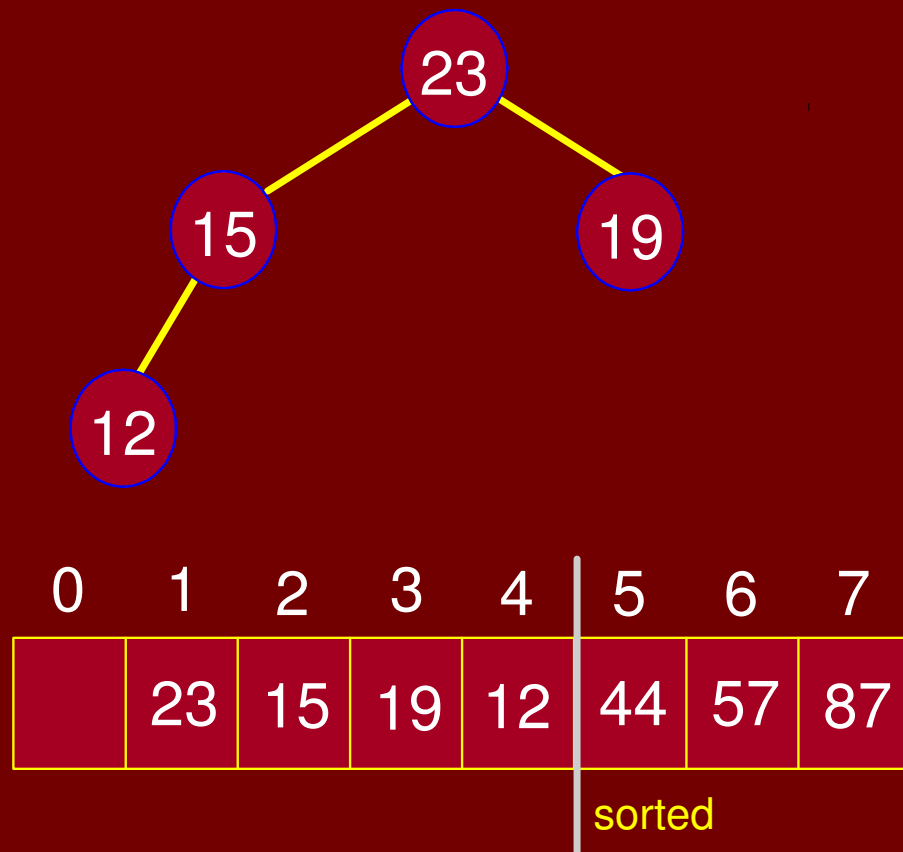
0	1	2	3	4	5	6	7
	44	23	19	12	15	57	87

sorted

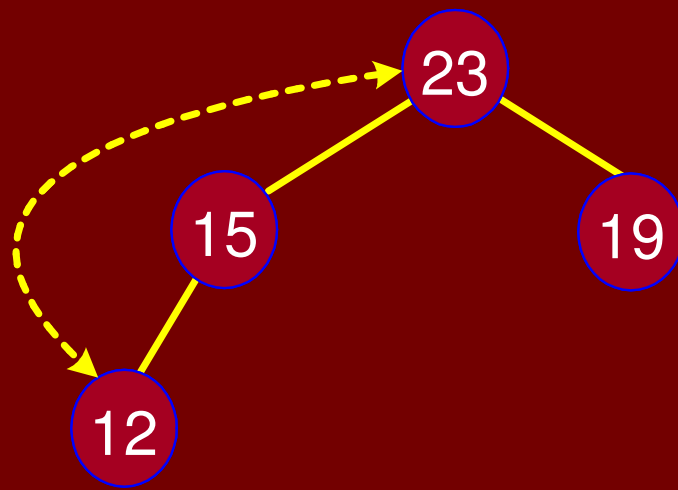
Heapsort Trace



Heapsort Trace



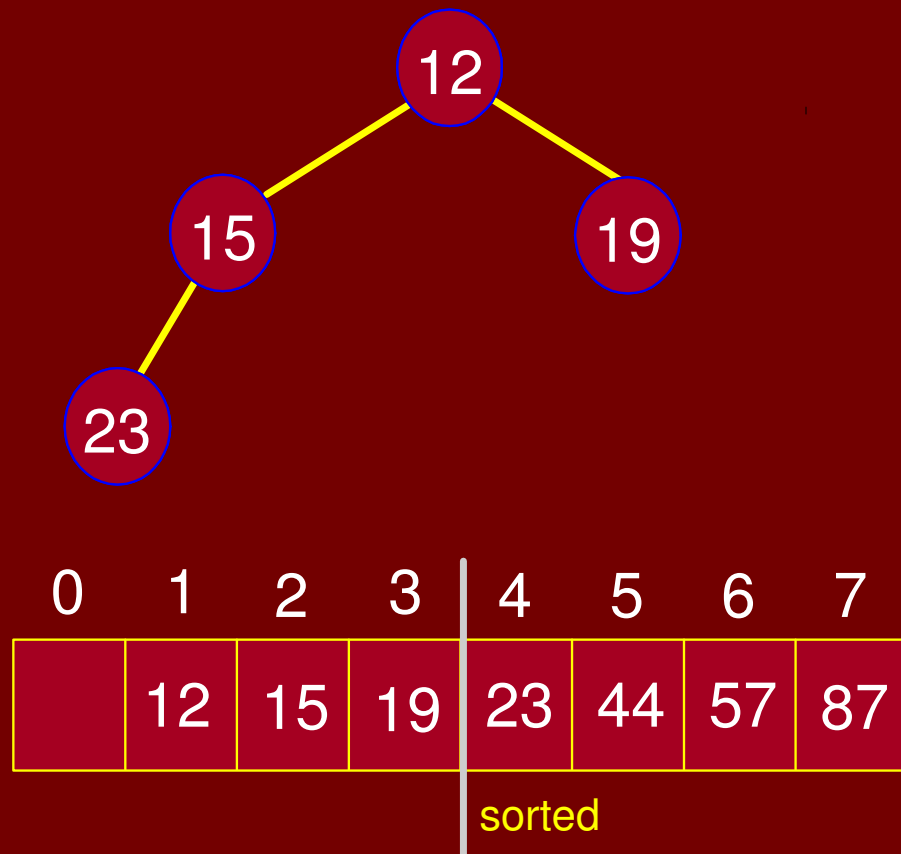
Heapsort Trace



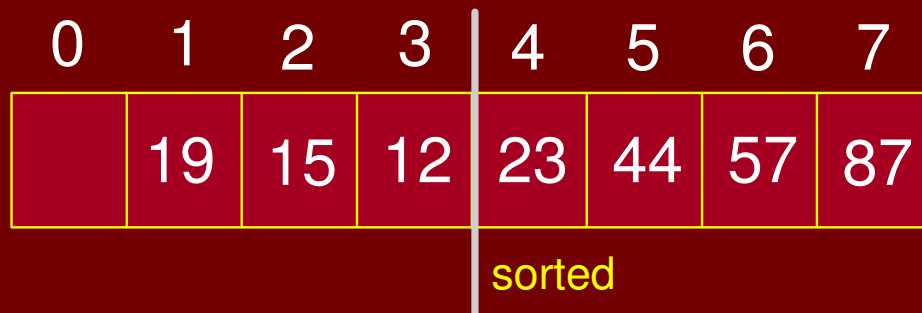
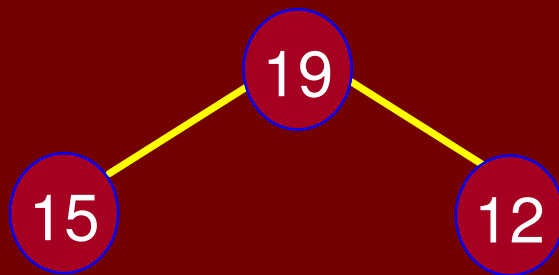
0	1	2	3	4	5	6	7
	23	15	19	12	44	57	87

sorted

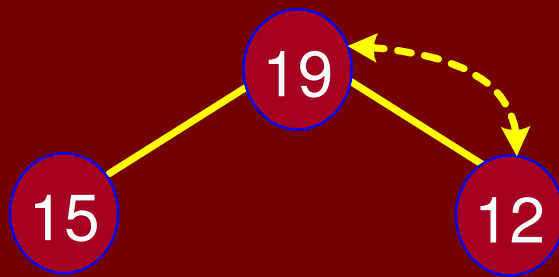
Heapsort Trace



Heapsort Trace



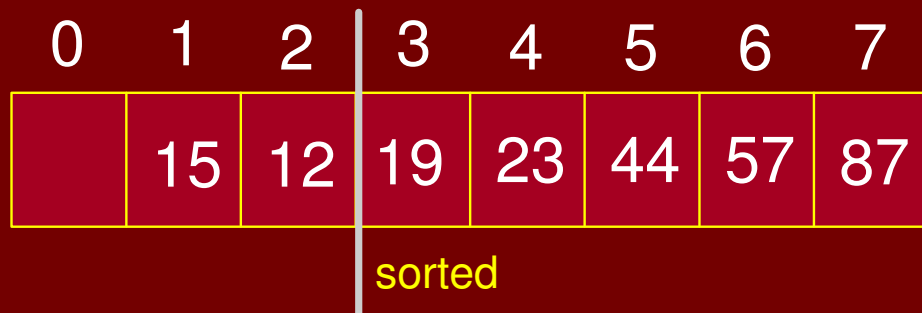
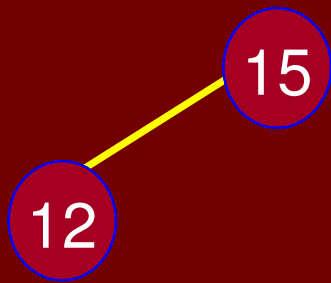
Heapsort Trace



0	1	2	3	4	5	6	7
	19	15	12	23	44	57	87

sorted

Heapsort Trace



Heapsort Trace

0	1	2	3	4	5	6	7
	12	15	19	23	44	57	87

sorted

Heapify

```
HEAPIFY( array A, int i, int n)
1  l ← LEFT(i)
2  r ← RIGHT(i)
3  max ← i
4  if (l ≤ m) and (A[l] > A[max])
5      then max ← l
6  if (r ≤ m) and (A[r] > A[max])
7      then max ← r
8  if (max ≠ i)
9      then SWAP(A[i], A[max])
10     HEAPIFY(A, max, m)
```

Analysis of Heapify

- We call heapify on the root of the tree.
- The maximum levels an element could move up is $\Theta(\log n)$ levels.
- At each level, we do simple comparison which $O(1)$.
- The total time for heapify is thus $O(\log n)$.
- Notice that it is not $\Theta(\log n)$ since, for example, if we call heapify on a leaf, it will terminate in $\Theta(1)$ time.

BuildHeap

BUILDHEAP(array A , int n)

1 for $i \leftarrow n/2$ downto 1

2 do

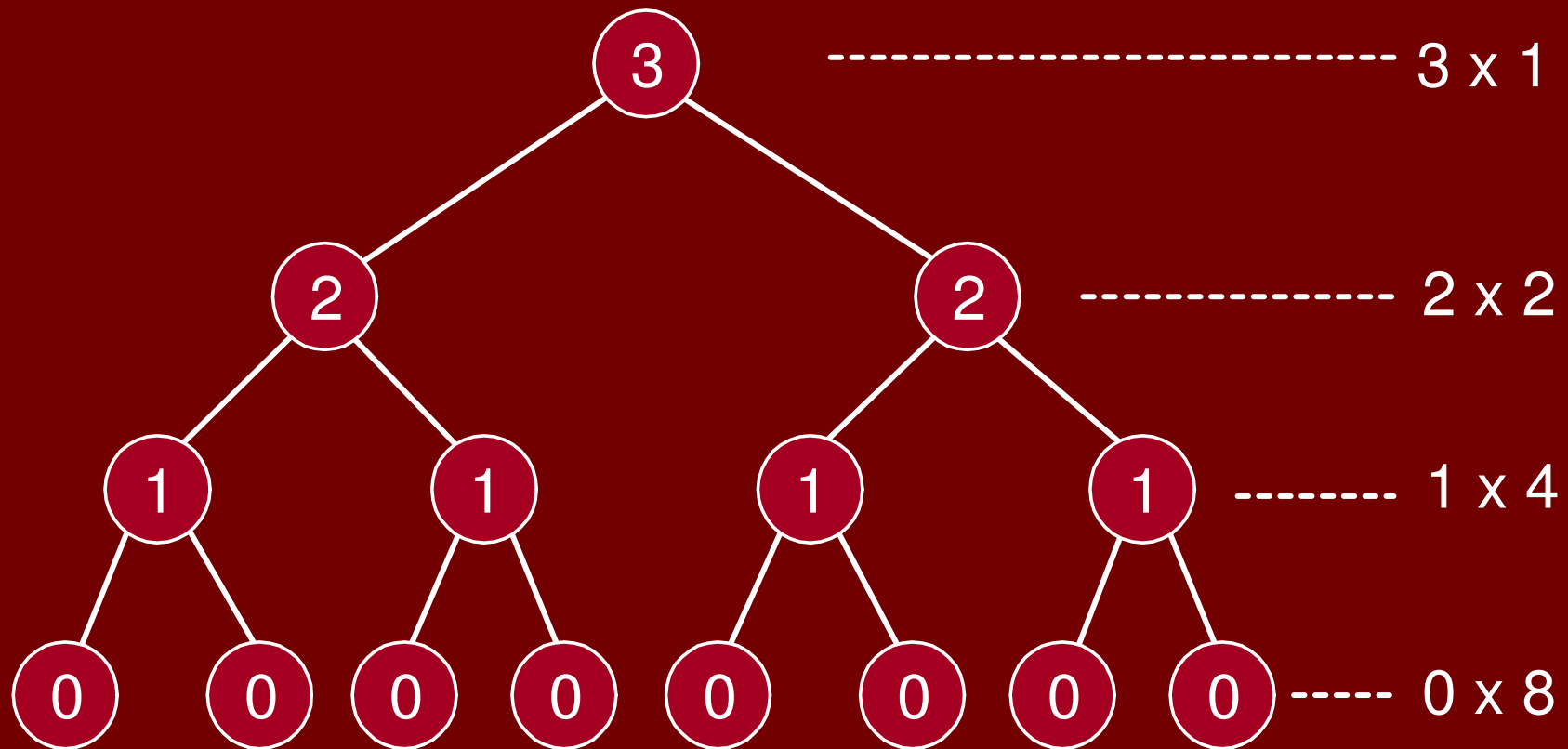
3 HEAPIFY(A, i, n)

Analysis of BuildHeap

- For convenience, we will assume $n = 2^{h+1} - 1$ where h is the height of tree.
- The heap is a left-complete binary tree.
- Thus at each level j , $j < h$, there are 2^j nodes in the tree.
- At level h , there will be 2^h or less nodes.
- How much work does buildHeap carry out?

BuildHeap

Total work performed



Analysis of BuildHeap

- At the bottom most level, there are 2^h nodes but we do not heapify these.
- At the next level up, there are 2^{h-1} nodes and each might shift down 1.
- In general, at level j , there are 2^{h-j} nodes and each may shift down j levels.

Analysis of BuildHeap

- So, if count from bottom to top, level-by-level, the total time is

$$T(n) = \sum_{j=0}^h j 2^{h-j} = \sum_{j=0}^h j \frac{2^h}{2^j}$$

- We can factor out the 2^h term:

$$T(n) = 2^h \sum_{j=0}^h \frac{j}{2^j}$$

Analysis of BuildHeap

- How do we solve this sum? Recall the geometric series, for any constant $x < 1$

$$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x}$$

- Take the derivative with respect to x and multiply by x

$$\sum_{j=0}^{\infty} jx^{j-1} = \frac{1}{(1-x)^2}$$

$$\sum_{j=0}^{\infty} jx^j = \frac{x}{(1-x)^2}$$

Analysis of BuildHeap

- We plug $x = 1/2$ and we have the desired formula:

$$\sum_{j=0}^{\infty} \frac{j}{2^j} = \frac{1/2}{(1 - (1/2))^2} = \frac{1/2}{1/4} = 2$$

Analysis of BuildHeap

- In our case, we have a bounded sum, but since the infinite series is bounded, we can use it as an easy approximation:

$$\begin{aligned} T(n) &= 2^h \sum_{j=0}^h \frac{j}{2^j} \\ &\leq 2^h \sum_{j=0}^{\infty} \frac{j}{2^j} \\ &\leq 2^h \cdot 2 = 2^{h+1} \end{aligned}$$

Analysis of BuildHeap

- Recall that $n = 2^{h+1} - 1$.
- Therefore

$$T(n) \leq n + 1 \in O(n)$$

- The algorithm takes at least $\Omega(n)$ time since it must access every element at once.
- So the total time for BuildHeap is $\Theta(n)$.

Analysis of BuildHeap

- BuildHeap is a relatively complex algorithm.
- Yet, the analysis yield that it takes $\Theta(n)$ time.
- An intuitive way to describe why it is so is to observe an important fact about binary trees
- The fact is that the vast majority of the nodes are at the lowest level of the tree.
- For example, in a complete binary tree of height h , there is a total of $n \approx 2^{h+1}$ nodes.

Analysis of BuildHeap

- The number of nodes at the bottom three levels alone is

$$2^h + 2^{h-1} + 2^{h-2} = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} = \frac{7n}{8} = 0.875n$$

- Almost 90% of the nodes of a complete binary tree reside in the 3 lowest levels.
- Thus, algorithms that operate on trees should be efficient (as BuildHeap is) on the bottom-most levels since that is where most of the weight of the tree resides.

Analysis of Heapsort

- Heapsort calls BuildHeap once. This takes $\Theta(n)$.
- Heapsort then extracts roughly n maximum elements from the heap.
- Each extract requires a constant amount of work (swap) and $O(\log n)$ heapify.
- Heapsort is thus $O(n \log n)$.

Analysis of Heapsort

- Is HeapSort $\Theta(n \log n)$?
- The answer is yes.
- In fact, later we will show that comparison based sorting algorithms can not run faster than $\Omega(n \log n)$.
- Heapsort is such an algorithm and so is Mergesort that we saw earlier.

Quicksort

- Our next sorting algorithm is Quicksort.
- It is one of the fastest sorting algorithms known and is the method of choice in most sorting libraries.
- Quicksort is based on the divide and conquer strategy.

Quicksort

```
QUICKSORT( array A, int p, int r)
1  if (r > p)
2      then
3          i ← a random index from [p..r]
4          swap A[i] with A[p]
5          q ← PARTITION(A, p, r)
6          QUICKSORT(A, p, q - 1)
7          QUICKSORT(A, q + 1, r)
```

Partition Algorithm

Recall that the partition algorithm partitions the array $A[p..r]$ into three sub arrays about a pivot element x .

- $A[p..q - 1]$ whose elements are less than or equal to x ,
- $A[q] = x$,
- $A[q + 1..r]$ whose elements are greater than x

Choosing the Pivot

- We will choose the first element of the array as the pivot, i.e. $x = A[p]$.
- If a different rule is used for selecting the pivot, we can swap the chosen element with the first element.
- We will choose the pivot randomly.

Partition Algorithm

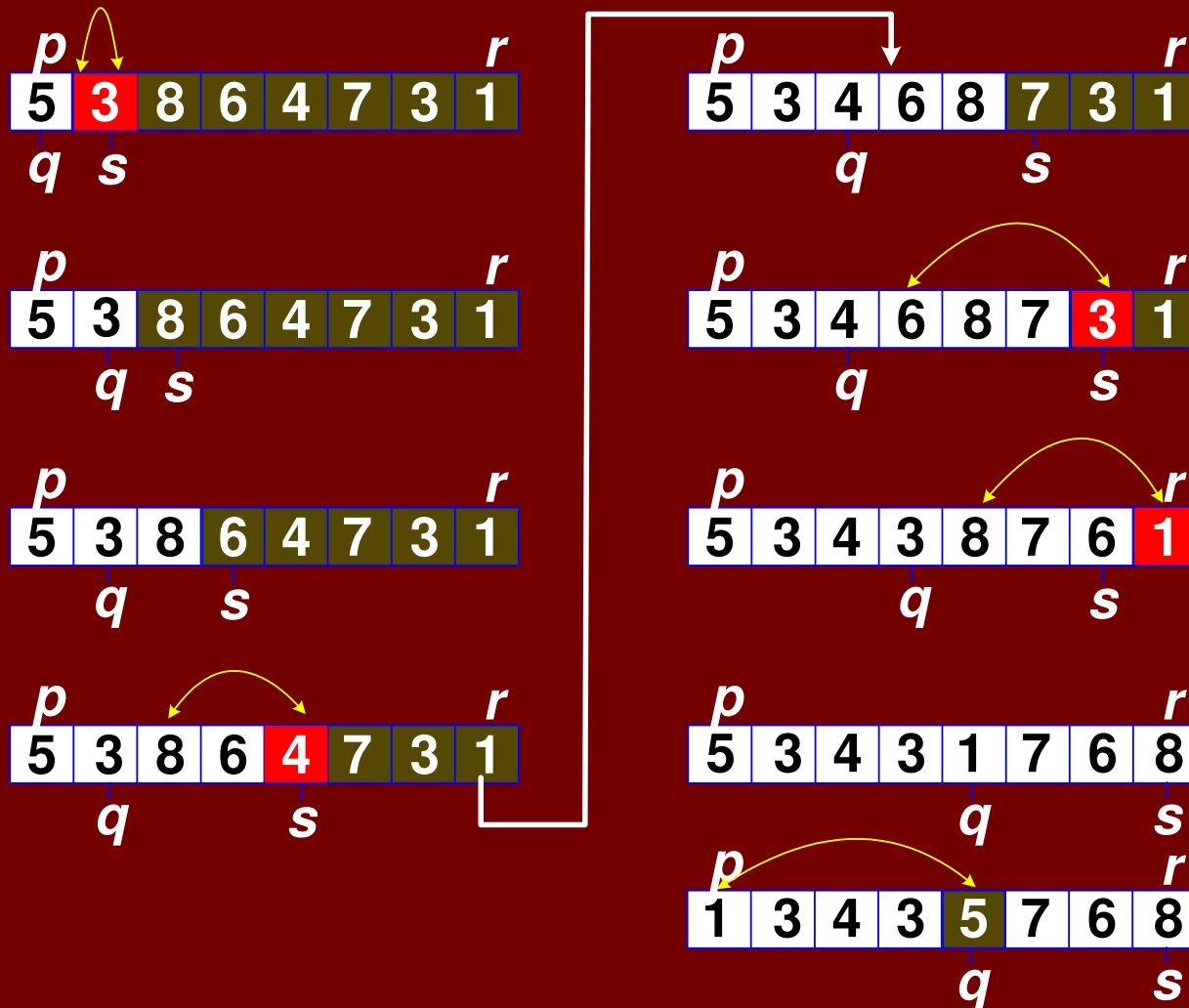
The algorithm works by maintaining the following *invariant condition*.

1. $A[p] = x$ is the pivot value.
2. $A[p..q - 1]$ contains elements that are less than x ,
3. $A[q + 1..s - 1]$ contains elements that are greater than or equal to x
4. $A[s..r]$ contains elements whose values are currently unknown.

Partition Algorithm

```
PARTITION( array A, int p, int r)
1  x ← A[p]
2  q ← p
3  for s ← p + 1 to r
4  do if (A[s] < x)
5      then q ← q + 1
6          swap A[q] with A[s]
7
8  swap A[p] with A[q]
9  return q
```

Partition Algorithm



Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
---	---	----	---	----	---	---	---	----	----	----	---	----	----	---	---	----

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17

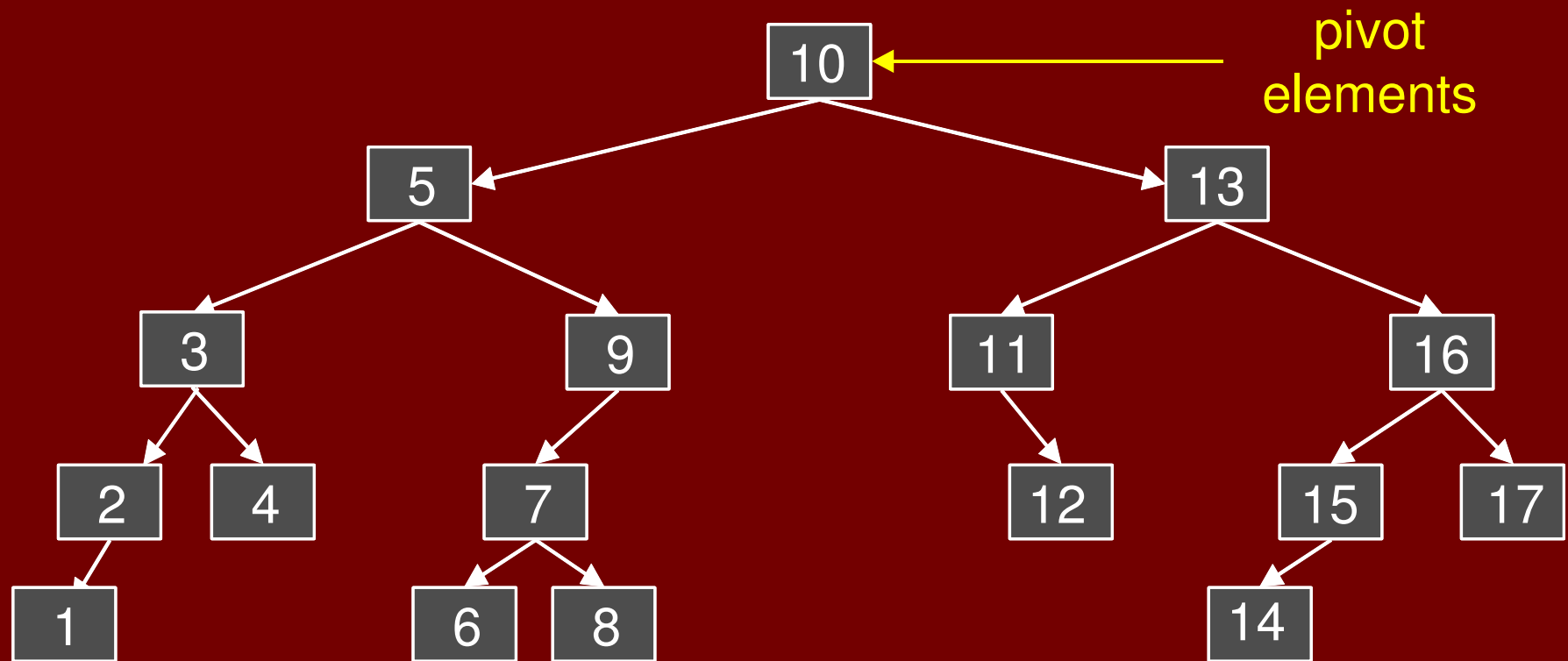
Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Quicksort BST



Analysis of Quicksort

- The running time of quicksort depends heavily on the selection of the pivot.
- If the rank of the pivot is very large or very small then the partition (BST) will be unbalanced.
- Since the pivot is chosen randomly in our algorithm, the expected running time is $O(n \log n)$.
- The worst case time, however, is $O(n^2)$. Luckily, this happens rarely.

Analysis of Quicksort

- It takes $\Theta(n)$ time to do the partitioning.
- The time taken for sorting array $A[1..n]$ is

$$T(n) = T(q - 1) + T(n - q) + n$$

- To get the worst case, we maximize over all possible values of q .

Analysis of Quicksort

Putting is together, we get the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \max_{1 \leq q \leq n} (T(q-1) + T(n-q) + n) & \text{otherwise} \end{cases}$$

The worse case happens at $q = 1$ or $q = n$. If we expand the recurrence for $q = 1$, we get:

Analysis of Quicksort

$$\begin{aligned}T(n) &\leq T(0) + T(n-1) + n \\&= 1 + T(n-1) + n \\&= T(n-1) + (n+1) \\&= T(n-2) + n + (n+1) \\&= T(n-3) + (n-1) + n + (n+1) \\&= T(n-4) + (n-2) + (n-1) + n + (n+1) \\&= T(n-k) + \sum_{i=1}^{k-2} (n-i)\end{aligned}$$

Analysis of Quicksort

For the basis $T(1) = 1$ we set $k = n - 1$ and get

$$\begin{aligned} T(n) &\leq T(1) + \sum_{i=-1}^{n-3} (n - i) \\ &= 1 + (3 + 4 + 5 + \dots + (n - 1) + n + (n + 1)) \\ &\leq \sum_{i=1}^{n+1} i = \frac{(n + 1)(n + 2)}{2} \in \Theta(n^2) \end{aligned}$$

Average-case Analysis of Quicksort

- We will now show that in the average case, quicksort runs in $\Theta(n \log n)$ time.
- Recall that when we talked about average case at the beginning of the semester, we said that it depends on some assumption about the distribution of inputs.

Average-case Analysis of Quicksort

- However, in the case of quicksort, the analysis does not depend on the distribution of input at all.
- It only depends upon the random choices of pivots that the algorithm makes.

Average-case Analysis of Quicksort

- The algorithm has n random choices for the pivot element.
- Each of these choices has an equal probability $1/n$ of occurring.
- The recurrence can be written as

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \frac{1}{n} \sum_{q=1}^n (T(q-1) + T(n-q) + n) & \text{otherwise} \end{cases}$$

Average-case Analysis of Quicksort

The time $T(n)$ is the weighted sum of the times taken for various choices of q . I.e.,

$$\begin{aligned} T(n) = & \left[\frac{1}{n}(T(0) + T(n-1) + n) \right. \\ & + \frac{1}{n}(T(1) + T(n-2) + n) \\ & + \frac{1}{n}(T(2) + T(n-3) + n) \\ & \left. + \dots + \frac{1}{n}(T(n-1) + T(0) + n) \right] \end{aligned}$$

Average-case Analysis of Quicksort

- We have not seen such a recurrence before.
- To solve it, expansion is possible but it is rather tricky.
- We will attempt a constructive induction to solve it.
- We know that we want a $\Theta(n \log n)$.
- Let us assume that $T(n) \leq cn \log n$ for $n \geq 2$ where c is a constant.

Average-case Analysis of Quicksort

For the base case $n = 2$ we have

$$\begin{aligned} T(n) &= \frac{1}{2} \sum_{q=1}^2 (T(q-1) + T(2-q) + 2) \\ &= \frac{1}{2} [(T(0) + T(1) + 2) + (T(1) + T(0) + 2)] \\ &= \frac{8}{2} = 4. \end{aligned}$$

Average-case Analysis of Quicksort

We want this to be at most $c2 \log 2$, i.e.,

$$T(2) \leq c2 \log 2$$

or

$$4 \leq c2 \log 2$$

therefore

$$c \geq 4/(2 \log 2) \approx 2.88.$$

Average-case Analysis of Quicksort

- For the induction step, we assume that $n \geq 3$ and
- The induction hypothesis is that for any $n' < n$, we have $T(n') \leq cn' \log n'$.
- We want to prove that it is true for $T(n)$.

Average-case Analysis of Quicksort

By expanding $T(n)$ and moving the factor of n outside the sum we have

$$\begin{aligned} T(n) &= \frac{1}{n} \sum_{q=1}^n (T(q-1) + T(n-q) + n) \\ &= \frac{1}{n} \sum_{q=1}^n (T(q-1) + T(n-q)) + n \\ &= \frac{1}{n} \sum_{q=1}^n T(q-1) + \frac{1}{n} \sum_{q=1}^n T(n-q) + n \end{aligned}$$

Average-case Analysis of Quicksort

$$T(n) = \frac{1}{n} \sum_{q=1}^n T(q-1) + \frac{1}{n} \sum_{q=1}^n T(n-q) + n$$

- Observe that the two sums add up the same values $T(0) + T(1) + \dots + T(n-1)$.
- One counts up and other counts down.
- Thus we can replace them with $2 \sum_{q=0}^{n-1} T(q)$.

Average-case Analysis of Quicksort

We will extract $T(0)$ and $T(1)$ and treat them specially. These two do not follow the formula.

$$\begin{aligned} T(n) &= \frac{2}{n} \left(\sum_{q=0}^{n-1} T(q) \right) + n \\ &= \frac{2}{n} \left(T(0) + T(1) + \sum_{q=2}^{n-1} T(q) \right) + n \end{aligned}$$

Average-case Analysis of Quicksort

We will apply the induction hypothesis for $q < n$
we have

$$\begin{aligned} T(n) &= \frac{2}{n} \left(T(0) + T(1) + \sum_{q=2}^{n-1} T(q) \right) + n \\ &\leq \frac{2}{n} \left(1 + 1 + \sum_{q=2}^{n-1} (cq \log q) \right) + n \\ &= \frac{2c}{n} \left(\sum_{q=2}^{n-1} (cq \ln q) \right) + n + \frac{4}{n} \end{aligned}$$

Average-case Analysis of Quicksort

We have never seen this sum before:

$$S(n) = \sum_{q=2}^{n-1} (cq \ln q)$$

Recall from calculus that for any monotonically increasing function $f(x)$

$$\sum_{i=a}^{b-1} f(i) \leq \int_a^b f(x) dx$$

Average-case Analysis of Quicksort

The function $f(x) = x \ln x$ is monotonically increasing, and so

$$S(n) = \sum_{q=2}^{n-1} (cq \ln q) \leq \int_2^n x \ln x \, dx \quad (1)$$

We can integrate this by parts:

$$\int_2^n x \ln x \, dx = \frac{x^2}{2} \ln x - \frac{x^2}{4} \Big|_{x=2}^n$$

Average-case Analysis of Quicksort

$$\begin{aligned}\int_2^n x \ln x \, dx &= \frac{x^2}{2} \ln x - \frac{x^2}{4} \Big|_{x=2}^n \\ &= \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) - (2 \ln 2 - 1) \\ &\leq \frac{n^2}{2} \ln n - \frac{n^2}{4}\end{aligned}$$

Average-case Analysis of Quicksort

We thus have

$$S(n) = \sum_{q=2}^{n-1} (cq \ln q) \leq \frac{n^2}{2} \ln n - \frac{n^2}{4}$$

Plug this back into the expression for $T(n)$ to get

$$T(n) = \frac{2c}{n} \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) + n + \frac{4}{n}$$

Average-case Analysis of Quicksort

$$\begin{aligned}T(n) &= \frac{2c}{n} \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) + n + \frac{4}{n} \\&= cn \ln n - \frac{cn}{2} + n + \frac{4}{n} \\&= cn \ln n + n \left(1 - \frac{c}{2} \right) + \frac{4}{n}\end{aligned}$$

Average-case Analysis of Quicksort

$$T(n) = cn \ln n + n\left(1 - \frac{c}{2}\right) + \frac{4}{n}$$

- To finish the proof, we want all of this to be at most $cn \ln n$.
- For this to happen, we will need to select c such that

$$n\left(1 - \frac{c}{2}\right) + \frac{4}{n} \leq 0$$

Average-case Analysis of Quicksort

- If we select $c = 3$, and use the fact that $n \geq 3$ we get

$$\begin{aligned} n\left(1 - \frac{c}{2}\right) + \frac{4}{n} &= \frac{3}{n} - \frac{n}{2} \\ &\leq 1 - \frac{3}{2} = -\frac{1}{2} \leq 0. \end{aligned}$$

- From the basis case we had $c \geq 2.88$.
Choosing $c = 3$ satisfies all the constraints.
- Thus $T(n) = 3n \ln n \in \Theta(n \log n)$.

In-place, Stable Sorting

- An *in-place* sorting algorithm is one that uses no additional array for storage.
- A sorting algorithm is *stable* if duplicate elements remain in the same relative position after sorting.

9	3	3'	5	6	5'	2	1	3''
---	---	----	---	---	----	---	---	-----

unsorted

1	2	3	3'	3''	5	5'	6	9
---	---	---	----	-----	---	----	---	---

stable sort

1	2	3'	3	3''	5'	5	6	9
---	---	----	---	-----	----	---	---	---

unstable

In-place, Stable Sorting

- Bubble sort, insertion sort and selection sort are in-place sorting algorithms.
- Bubble sort and insertion sort can be implemented as stable algorithms but selection sort cannot (without significant modifications).
- Mergesort is a stable algorithm but not an in-place algorithm. It requires extra array storage.
- Quicksort is not stable but is an in-place algorithm.
- Heapsort is an in-place algorithm but is not stable.

Lower Bounds for Sorting

- The best we have seen so far is $O(n \log n)$ algorithms for sorting.
- Is it possible to do better than $O(n \log n)$?
- If a sorting algorithm is solely based on comparison of keys in the array then it is *impossible* to sort more efficiently than $\Omega(n \log n)$ time.
- All algorithms we have seen so far are comparison-based sorting algorithms.

Lower Bounds for Sorting

- Consider sorting three numbers a_1, a_2, a_3 .
- There are $3! = 6$ possible combinations:

$(a_1, a_2, a_3), (a_1, a_3, a_2), (a_3, a_2, a_1)$

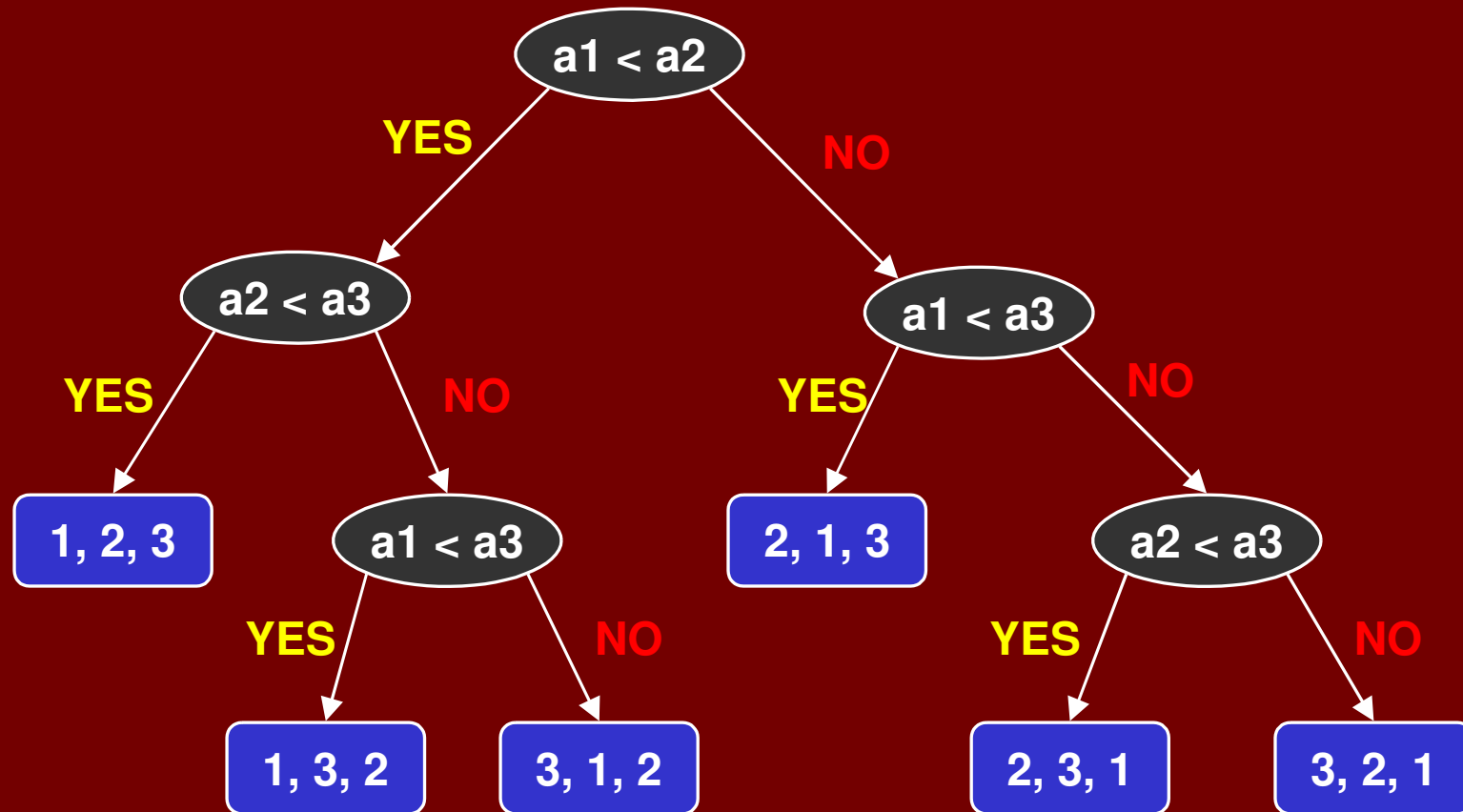
$(a_3, a_1, a_2), (a_2, a_1, a_3), (a_2, a_3, a_1)$

- One of these permutations leads to the numbers in sorted order.

Lower Bounds for Sorting

- The comparison based algorithm defines a *decision tree*.
- Here is the tree for the three numbers.

Decision Tree



Lower Bounds for Sorting

- For n elements, there will be $n!$ possible permutations.
- The height of the tree is exactly equal to $T(n)$, the running time of the algorithm.
- The height is $T(n)$ because any path from the root to a leaf corresponds to a sequence of comparisons made by the algorithm.

Lower Bounds for Sorting

- Any binary tree of height $T(n)$ has at most $2^{T(n)}$ leaves.
- Thus a comparison based sorting algorithm can distinguish between at most $2^{T(n)}$ different final outcomes.
- So we have

$$2^{T(n)} \geq n! \quad \text{and therefore}$$

$$T(n) \geq \log(n!)$$

Lower Bounds for Sorting

- We can use *Stirling's approximation* for $n!$:

$$n! \geq \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

- Therefore

$$\begin{aligned} T(n) &\geq \log \left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \right) \\ &= \log(\sqrt{2\pi n}) + n \log n - n \log e \\ &\in \Omega(n \log n) \end{aligned}$$

Lower Bounds for Sorting

We thus have the following theorem.

Theorem 1

Any comparison-based sorting algorithm has worst-case running time $\Omega(n \log n)$.

Linear Time Sorting

- The lower bound implies that if we hope to sort numbers faster than $O(n \log n)$, we cannot do it by making comparisons alone.
- Is it possible to sort without making comparisons?
- The answer is yes, but only under very restrictive circumstances.

Counting Sort

- We will consider three algorithms that are faster and work by not making comparisons.
- Counting sort assumes that the numbers to be sorted are in the range 1 to k where k is small.
- The basic idea is to determine the rank of each number in final sorted array.

Counting Sort

- Recall that the rank of an item is the number of elements that are less than or equal to it.
- Once we know the ranks, we simply copy numbers to their final position in an output array.
- The algorithm sorts in $\Theta(n + k)$.
- If k is $\Theta(n)$ then counting sort is an $\Theta(n)$ time algorithm.

Counting Sort

The algorithm uses three arrays.

1. $A[1..n]$: Holds the initial input.
2. $B[1..n]$: Array that holds the sorted output.
3. $C[1..k]$: Array of integers. $C[x]$ is the rank of x in A , where $x \in [1..k]$.

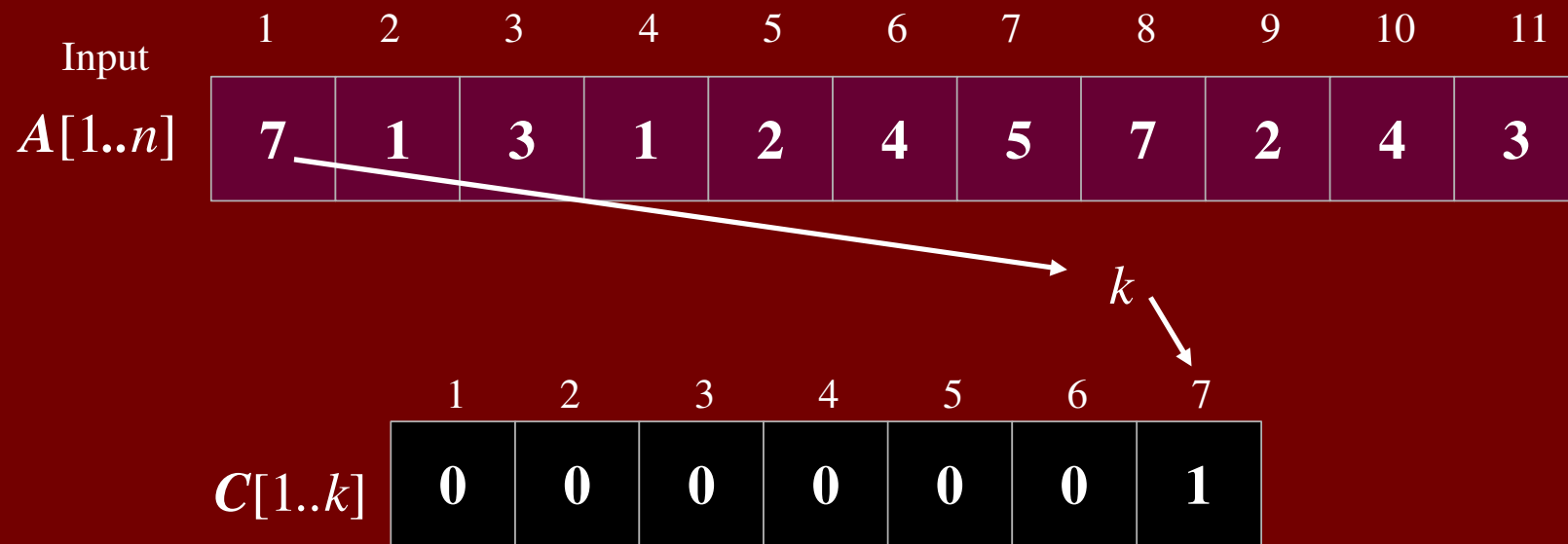
Counting Sort

Input	1	2	3	4	5	6	7	8	9	10	11
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

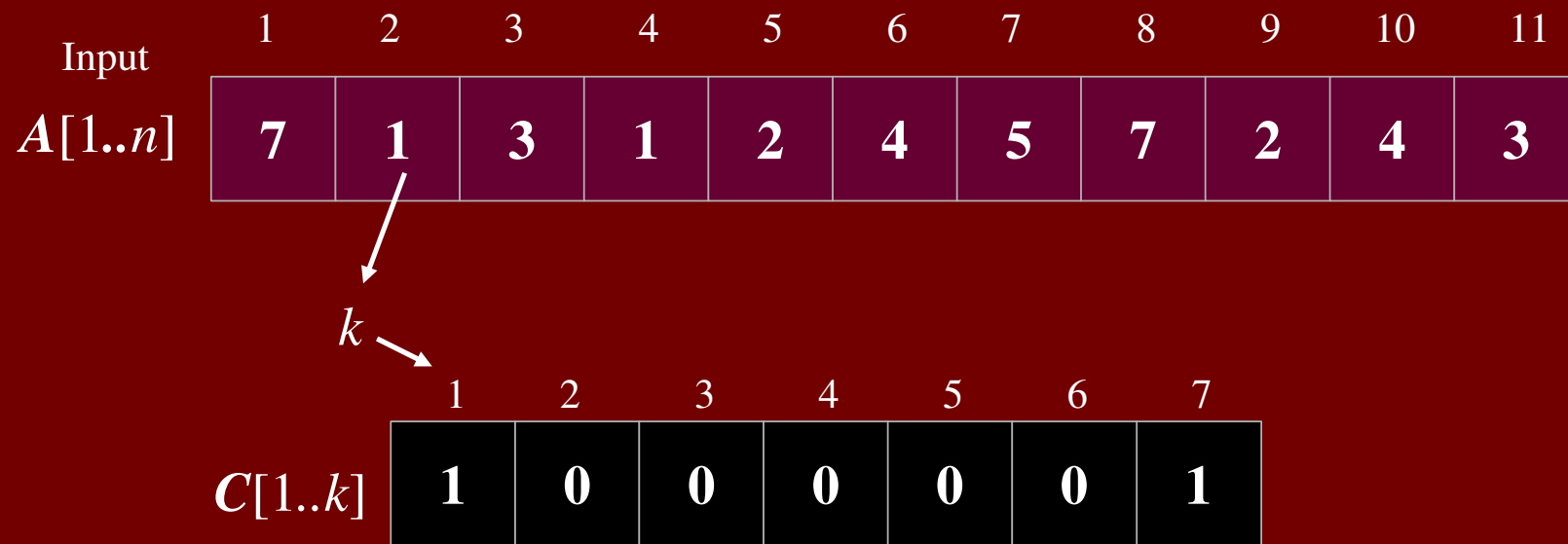
$k = 7$

	1	2	3	4	5	6	7
$C[1..k]$	0	0	0	0	0	0	0

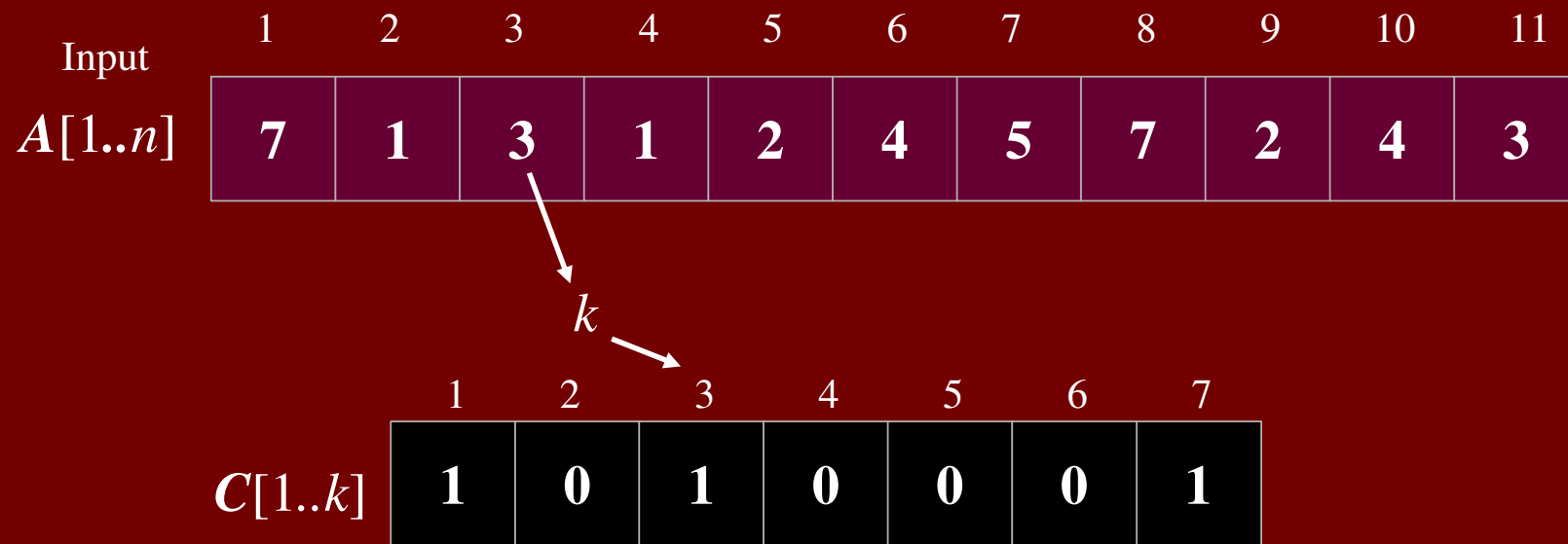
Counting Sort



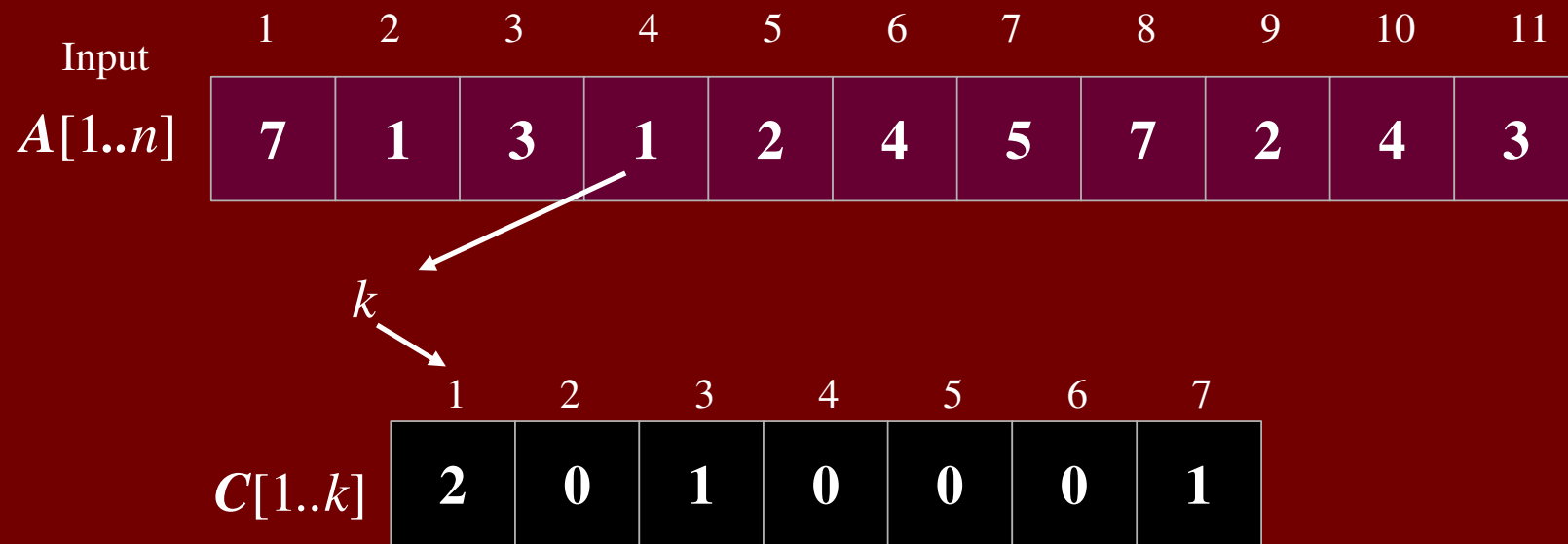
Counting Sort



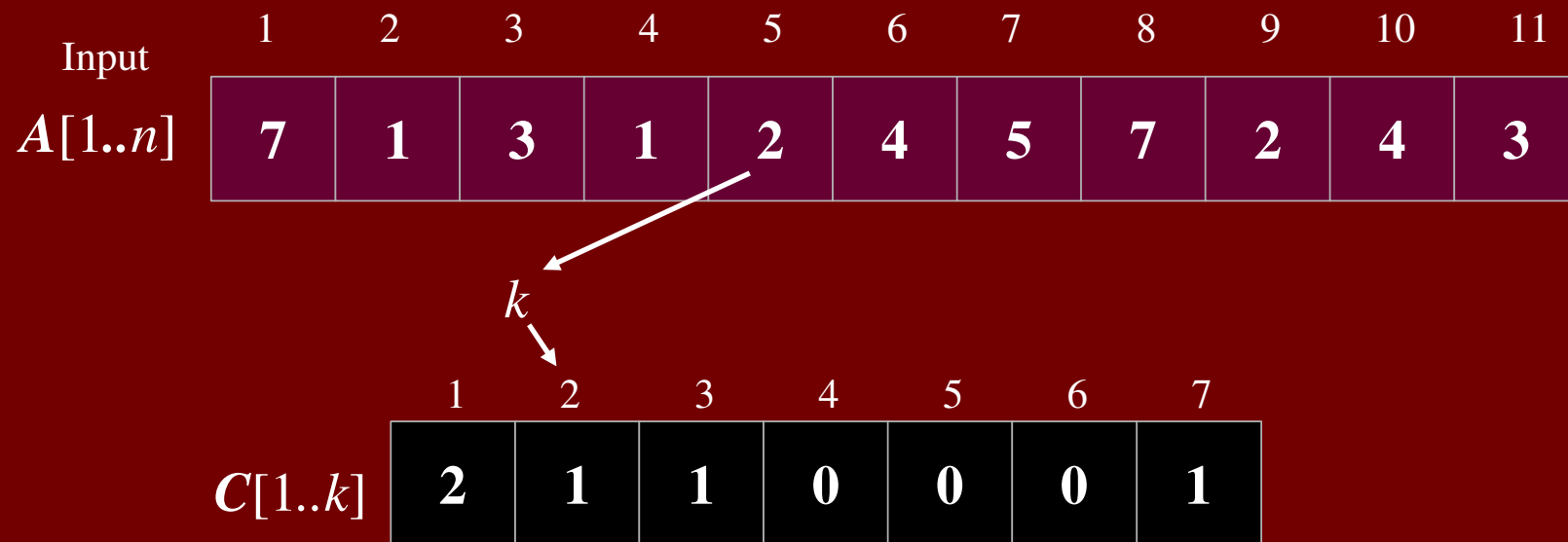
Counting Sort



Counting Sort



Counting Sort



Counting Sort

Input	1	2	3	4	5	6	7	8	9	10	11
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

finally

	1	2	3	4	5	6	7
$C[1..k]$	2	2	2	2	1	0	2

Counting Sort

Input	1	2	3	4	5	6	7	8	9	10	11
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7
$C[1..k]$	2	2	2	2	1	0	2

for $i = 2$ to 7
do $C[i] = C[i] + C[i-1]$

	1	2	3	4	5	6	7
C	2	4	6	8	9	9	11

↓
6 elements ≤ 3

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$						3					

$$B[6] = B[C[3]] = B[C[A[11]]] = A[11] = 3$$

	1	2	3	4	5	6	7
C	2	4	6	8	9	9	11

$$C[A[11]] = C[A[11]] - 1$$

C	2	4	5	8	9	9	11
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$						3		4			

$$B[8] = B[C[4]] = B[C[A[10]]] = A[10] = 4$$

	1	2	3	4	5	6	7
C	2	4	5	8	9	9	11

$$C[A[10]] = C[A[10]] - 1$$

C	2	4	5	7	9	9	11
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$				2		3		4			

$$B[4] = B[C[2]] = B[C[A[9]]] = A[9] = 2$$

	1	2	3	4	5	6	7
C	2	4	5	7	9	9	11

$$C[A[9]] = C[A[9]] - 1$$

C	2	3	5	7	9	9	11
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$				2		3		4			7

$$B[11] = B[C[7]] = B[C[A[8]]] = A[8] = 7$$

	1	2	3	4	5	6	7
C	2	3	5	7	9	9	11

$$C[A[8]] = C[A[8]] - 1$$

C	2	3	5	7	9	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$				2		3		4	5		7

$$B[9] = B[C[5]] = B[C[A[7]]] = A[7] = 5$$

	1	2	3	4	5	6	7
C	2	3	5	7	9	9	10

$$C[A[5]] = C[A[5]] - 1$$

C	2	3	5	7	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$				2		3	4	4	5		7

$$B[7] = B[C[4]] = B[C[A[6]]] = A[6] = 4$$

	1	2	3	4	5	6	7
C	2	3	5	7	8	9	10

$$C[A[6]] = C[A[6]] - 1$$

C	2	3	5	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$			2	2		3	4	4	5		7

$$B[3] = B[C[2]] = B[C[A[5]]] = A[5] = 2$$

	1	2	3	4	5	6	7
C	2	3	5	7	8	9	10

$$C[A[5]] = C[A[5]] - 1$$

C	2	2	5	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$		1	2	2		3	4	4	5		7

$$B[2] = B[C[1]] = B[C[A[4]]] = A[4] = 1$$

	1	2	3	4	5	6	7
C	2	2	5	7	8	9	10

$$C[A[4]] = C[A[4]] - 1$$

C	1	2	5	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$		1	2	2	3	3	4	4	5		7

$$B[5] = B[C[3]] = B[C[A[3]]] = A[3] = 3$$

	1	2	3	4	5	6	7
C	1	2	5	7	8	9	10

$$C[A[3]] = C[A[3]] - 1$$

C	1	2	4	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$	1	1	2	2	3	3	4	4	5		7

$$B[1] = B[C[1]] = B[C[A[2]]] = A[2] = 1$$

	1	2	3	4	5	6	7
C	1	2	4	7	8	9	10

$$C[A[3]] = C[A[3]] - 1$$

C	0	2	4	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4	5	7	2	4	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$	1	1	2	2	3	3	4	4	5	7	7

$$B[10] = B[C[7]] = B[C[A[1]]] = A[1] = 7$$

	1	2	3	4	5	6	7
C	0	2	4	7	8	9	10

$$C[A[1]] = C[A[1]] - 1$$

C	0	2	4	6	8	9	9
-----	---	---	---	---	---	---	---

Counting Sort

COUNTING-SORT(array A , array B , int k)

```
1  for  $i \leftarrow 1$  to  $k$ 
2  do  $C[i] \leftarrow 0$       k times
3  for  $j \leftarrow 1$  to  $\text{length}[A]$ 
4  do  $C[A[j]] \leftarrow C[A[j]] + 1$       n times
5  //  $C[i]$  now contains the number of elements  $= i$ 
6  for  $i \leftarrow 2$  to  $k$ 
7  do  $C[i] \leftarrow C[i] + C[i - 1]$       k times
8  //  $C[i]$  now contains the number of elements  $\leq i$ 
9  for  $j \leftarrow \text{length}[A]$  downto 1
10 do  $B[C[A[j]]] \leftarrow A[j]$ 
11    $C[A[j]] \leftarrow C[A[j]] - 1$       n times
```

Counting Sort: Stable

Counting sort is stable. Stability is important because

- data are often carried with the keys being sorted.
- radix sort (which uses counting sort as a subroutine) relies on it to work correctly.

Counting Sort: Stable

How is stability achieved?

COUNTING-SORT(array A , array B , int k)

```
1  ⋮  
2  for  $j \leftarrow \text{length}[A]$  downto 1  
3  do  $B[C[A[j]]] \leftarrow A[j]$   
4      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4*	5	7	2	4**	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$						3		4**			

$$B[8] = B[C[4]] = B[C[A[10]]] = A[10] = 4$$

	1	2	3	4	5	6	7
C	2	4	5	8	9	9	11

$$C[A[10]] = C[A[10]] - 1$$

C	2	4	5	7	9	9	11
-----	---	---	---	---	---	---	----

Counting Sort

Input											
$A[1..n]$	7	1	3	1	2	4*	5	7	2	4**	3
	1	2	3	4	5	6	7	8	9	10	11
Output											
$B[1..n]$				2		3	4*	4**	5		7

$$B[7] = B[C[4]] = B[C[A[6]]] = A[6] = 4$$

	1	2	3	4	5	6	7
C	2	3	5	7	8	9	10

$$C[A[6]] = C[A[6]] - 1$$

C	2	3	5	6	8	9	10
-----	---	---	---	---	---	---	----

Counting Sort

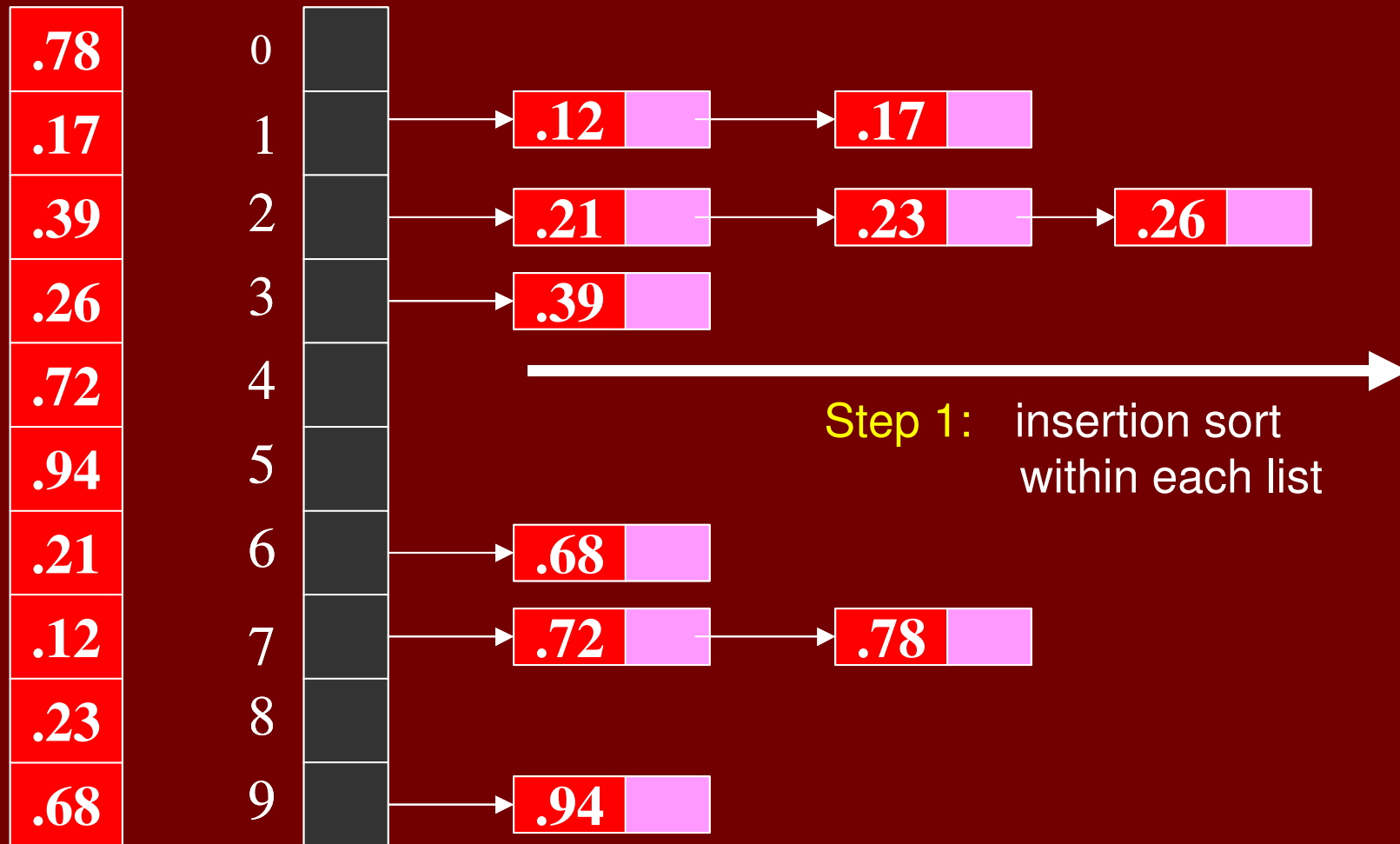
Input											
$A[1..n]$	7'	1^	3#	1^^	2+	4*	5	7''	2++	4**	3##
	1	2	3	4	5	6	7	8	9	10	11

Output											
$B[1..n]$	1^	1^^	2+	2++	3#	3##	4*	4**	5	7'	7''

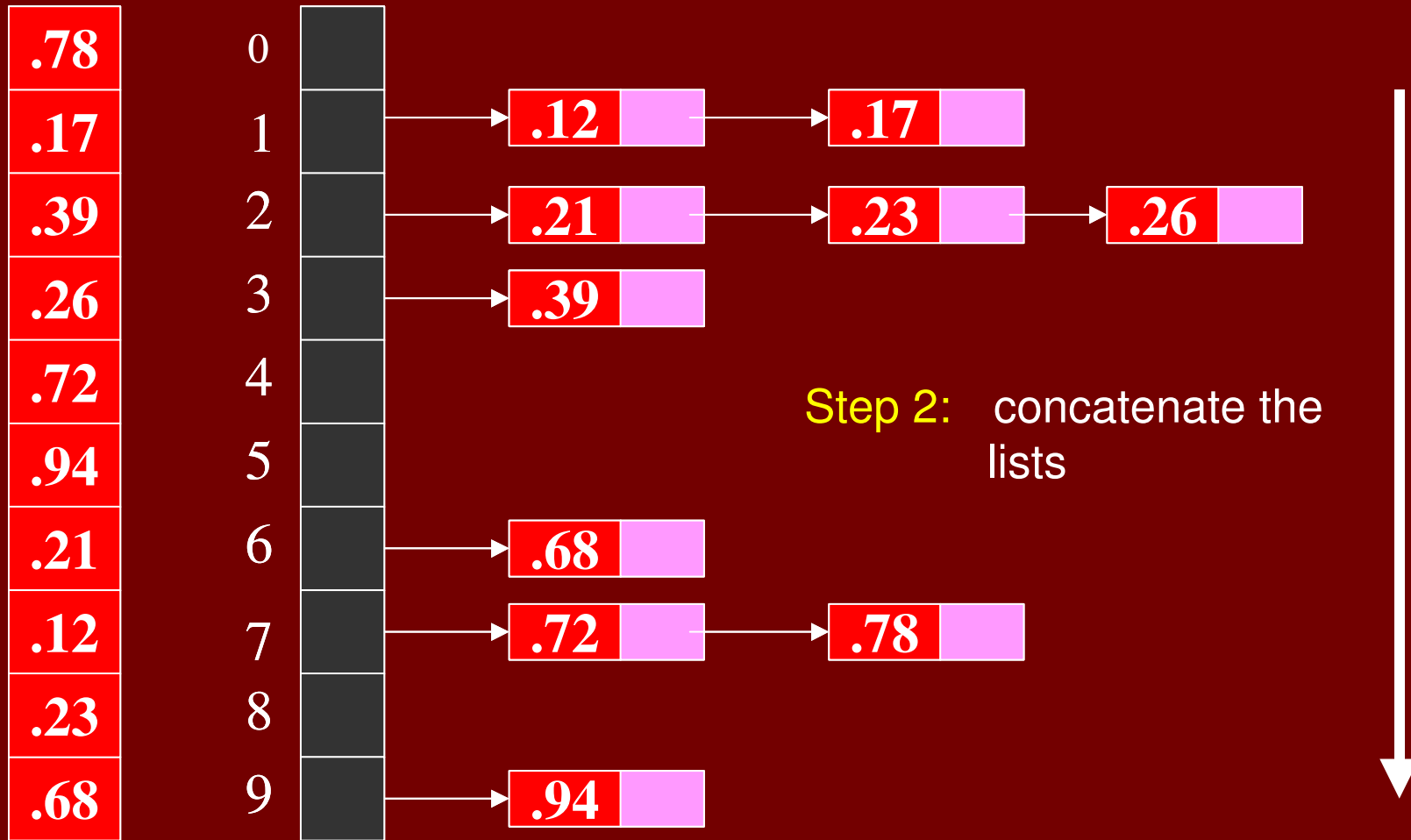
Bucket Sort

- Assumption: input elements are uniformly distributed over $[0, 1]$
- n inputs dropped into n equal-sized subintervals of $[0, 1]$.

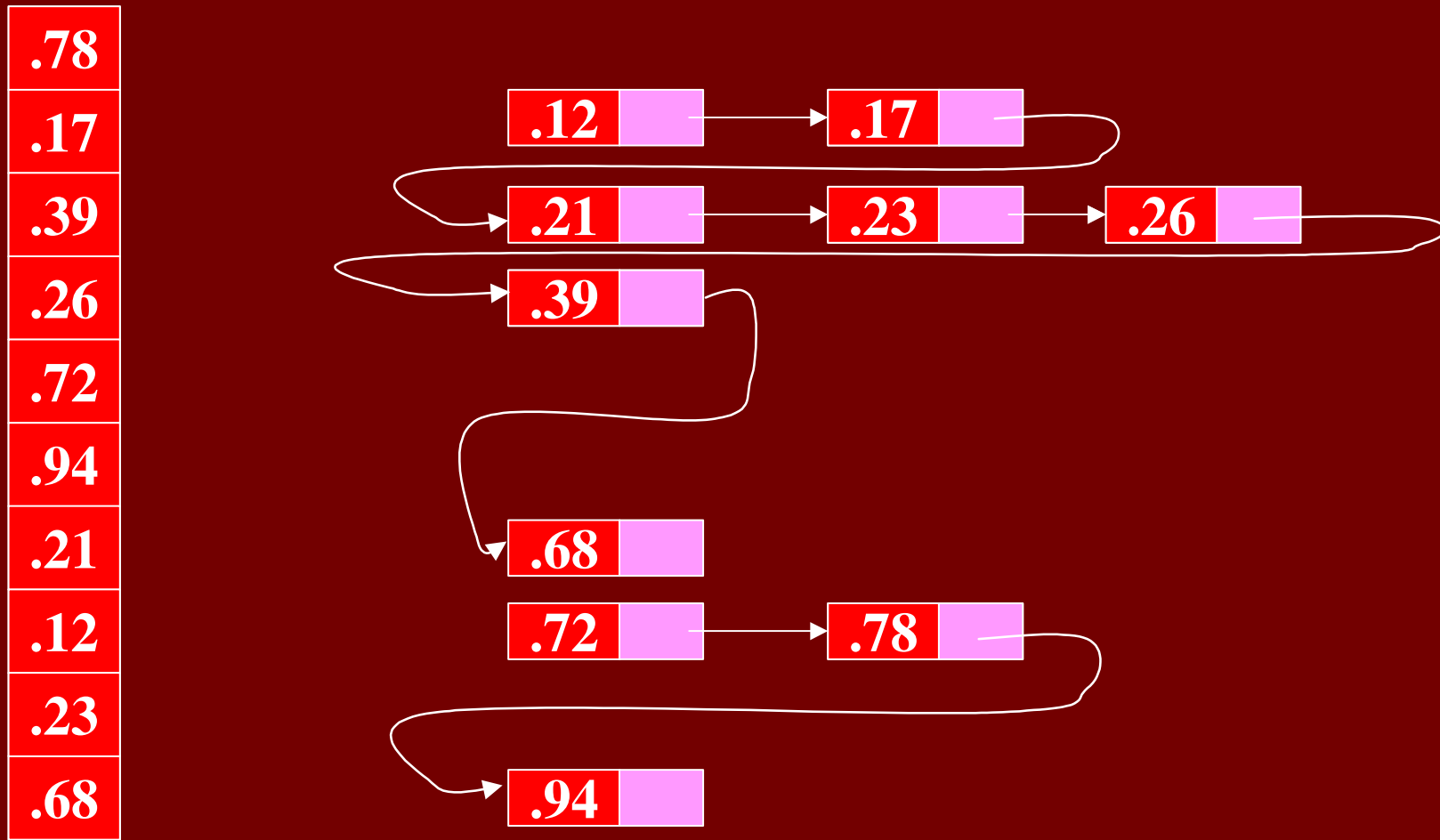
Bucket Sort



Bucket Sort



Bucket Sort



Radix Sort

- The main shortcoming of counting sort is that it is useful for small integers, i.e., $1..k$ where k is small.
- If k were a million or more, the size of the rank array would also be a million.
- Radix sort provides a nice work around this limitation by sorting numbers one digit at a time.

Radix Sort

576	49[4]	9[5]4	[1]76	176
494	19[4]	5[7]6	[1]94	194
194	95[4]	1[7]6	[2]78	278
296	$\Rightarrow$ 57[6]	$\Rightarrow$ 2[7]8	$\Rightarrow$ [2]96	$\Rightarrow$ 296
278	29[6]	4[9]4	[4]94	494
176	17[6]	1[9]4	[5]76	576
954	27[8]	2[9]6	[9]54	954

Radix Sort

Here is the algorithm that sorts $A[1..n]$ where each number is d digits long.

```
RADIX-SORT( array  $A$ , int  $n$ , int  $d$ )  
1  for  $i \leftarrow 1$  to  $d$   
2  do stably sort  $A$  w.r.t  $i^{\text{th}}$  lowest order digit
```



Dynamic Programming



Motivation

- Suppose we put a pair of rabbits in a place surrounded on all sides by a wall.
- How many pairs of rabbits can be produced from that pair in a year if it is supposed that every month each pair begets a new pair which from the second month on becomes productive?
- Resulting sequence is 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
- Each number is the sum of the two preceding numbers.

Leonardo Fibonacci Pisano

- This problem was posed by Leonardo Pisano, better known by his nickname Fibonacci (son of Bonacci, born 1170, died 1250).
- This problem and many others were in posed in his book *Liber abaci*, published in 1202.
- The book was based on the arithmetic and algebra that Fibonacci had accumulated during his travels.

Leonardo Fibonacci Pisano

- The book, which went on to be widely copied and imitated, introduced the Hindu-Arabic place-valued decimal system and the use of Arabic numerals into Europe.
- The rabbits problem in the third section of Liber abaci led to the introduction of the Fibonacci numbers and the Fibonacci sequence for which Fibonacci is best remembered today.

Leonardo Fibonacci Pisano

- This sequence, in which each number is the sum of the two preceding numbers, has proved extremely fruitful and appears in many different areas of mathematics and science.
- The *Fibonacci Quarterly* is a modern journal devoted to studying mathematics related to this sequence.

Fibonacci Sequence

- The Fibonacci numbers F_n are defined as follows:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Fibonacci Number

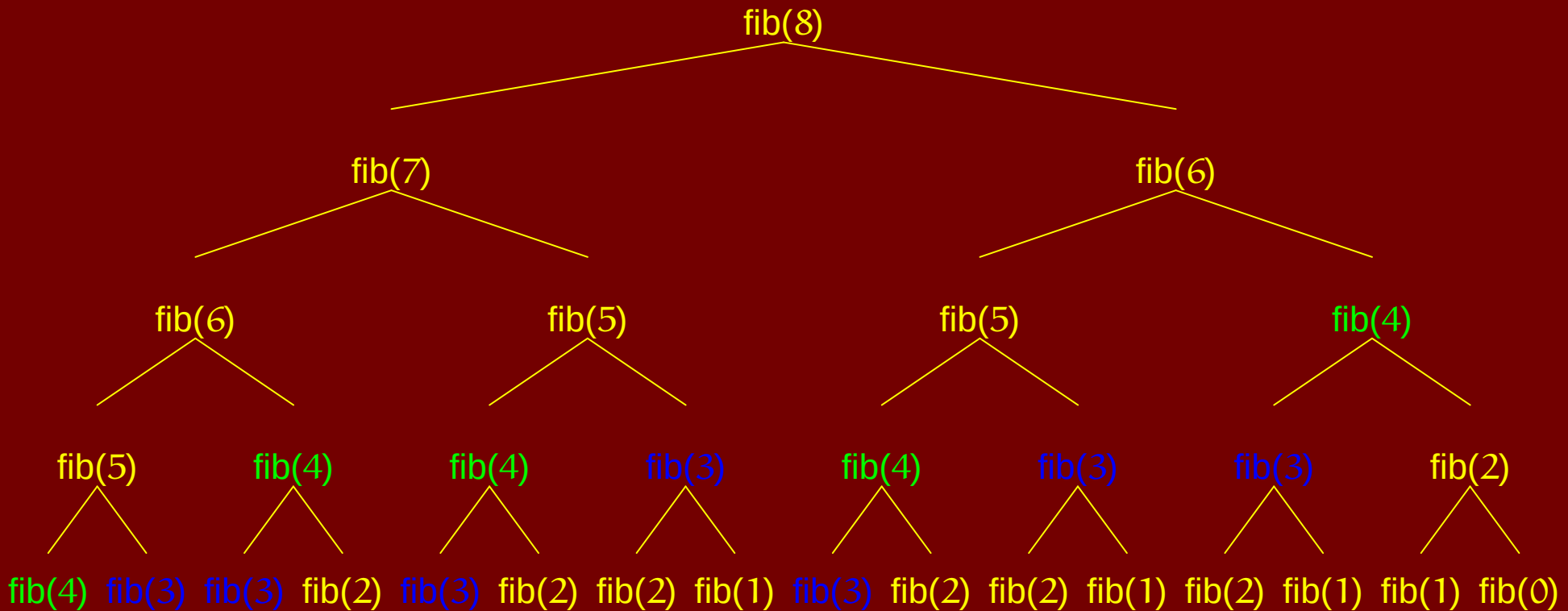
The recursive definition of Fibonacci numbers gives us a recursive algorithm for computing them:

$\text{FIB}(n)$

```
1  if ( $n < 2$ )
2      then return  $n$ 
3      else return  $\text{FIB}(n - 1) + \text{FIB}(n - 2)$ 
```

Fibonacci Number: Recursive Calls

Four levels of recursion for fib(8):



Fibonacci Number: Recursive Calls

- A single recursive call to $\text{fib}(n)$ results in one recursive call to $\text{fib}(n - 1)$, two recursive calls to $\text{fib}(n - 2)$, three recursive calls to $\text{fib}(n - 3)$, five recursive calls to $\text{fib}(n - 4)$ and, in general, F_{k-1} recursive calls to $\text{fib}(n - k)$
- For each call, we're recomputing the same fibonacci number from scratch.

Fibonacci Number: Recursive Calls

- We can avoid this unnecessary repetitions by writing down the results of recursive calls and looking them up again if we need them later.
- This process is called *memoization*.

Fibonacci Number: Memoization

MEMOFIB(n)

```
1  if ( $n < 2$ )
2      then return  $n$ 
3  if ( $F[n]$  is undefined)
4      then  $F[n] \leftarrow \text{MEMOFIB}(n - 1) + \text{MEMOFIB}(n - 2)$ 
5  return  $F[n]$ 
```

Fibonacci Number: Memoization

- If we trace through the recursive calls to MEMOFIB, we find that array $F[]$ gets filled from bottom up.
- I.e., first $F[2]$, then $F[3]$, and so on, up to $F[n]$.
- We can replace recursion with a simple for-loop that just fills up the array $F[]$ in that order.

```
MEMOFIB( $n$ )  
1  if ( $n < 2$ )  
2    then return  $n$   
3  if ( $F[n]$  is undefined)  
4    then  $F[n] \leftarrow \text{MEMOFIB}(n - 1)$   
5            $+ \text{MEMOFIB}(n - 2)$   
6  return  $F[n]$ 
```

Fibonacci Number: Iterative Algorithm

This gives us our first explicit dynamic programming algorithm.

ITERFIB(n)

1 $F[0] \leftarrow 0$

2 $F[1] \leftarrow 1$

3 **for** $i \leftarrow 2$ **to** n

4 **do**

5 $F[i] \leftarrow F[i - 1] + F[i - 2]$

6 **return** $F[n]$

Fibonacci Number: Iterative Algorithm

This gives us our first explicit dynamic programming algorithm.

ITERFIB(n)

1 $F[0] \leftarrow 0$

2 $F[1] \leftarrow 1$

3 **for** $i \leftarrow 2$ **to** n

4 **do**

5 $F[i] \leftarrow F[i - 1] + F[i - 2]$

6 **return** $F[n]$

Fibonacci Number: Iterative Algorithm

- This algorithm clearly takes only $O(n)$ time to compute F_n .
- By contrast, the original recursive algorithm takes $\Theta(\phi^n)$, $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$.
- ITERFIB achieves an exponential speedup over the original recursive algorithm.

Dynamic Programming

- Dynamic programming is essentially recursion without repetition.
- Developing a dynamic programming algorithm generally involves two separate steps:

Dynamic Programming

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller subproblems.
- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.

Dynamic Programming

- Dynamic programming algorithms need to store the results of intermediate subproblems.
- This is often *but not always* done with some kind of table.

Edit Distance

- Introduced by Levenshtein in 1966.
- Definition: Minimum number of edit operations to transform one string to another
- Possible edit operations
 - Symbol insertion (I)
 - Symbol deletion (D)
 - Symbol substitution (S)

Edit Distance

- For example, the edit distance between FOOD and MONEY is at most four:

FOOD $\longrightarrow$ MOOD $\longrightarrow$ MON_^D
 $\longrightarrow$ MONED $\longrightarrow$ MONEY

Edit Distance: Applications

Spelling Correction

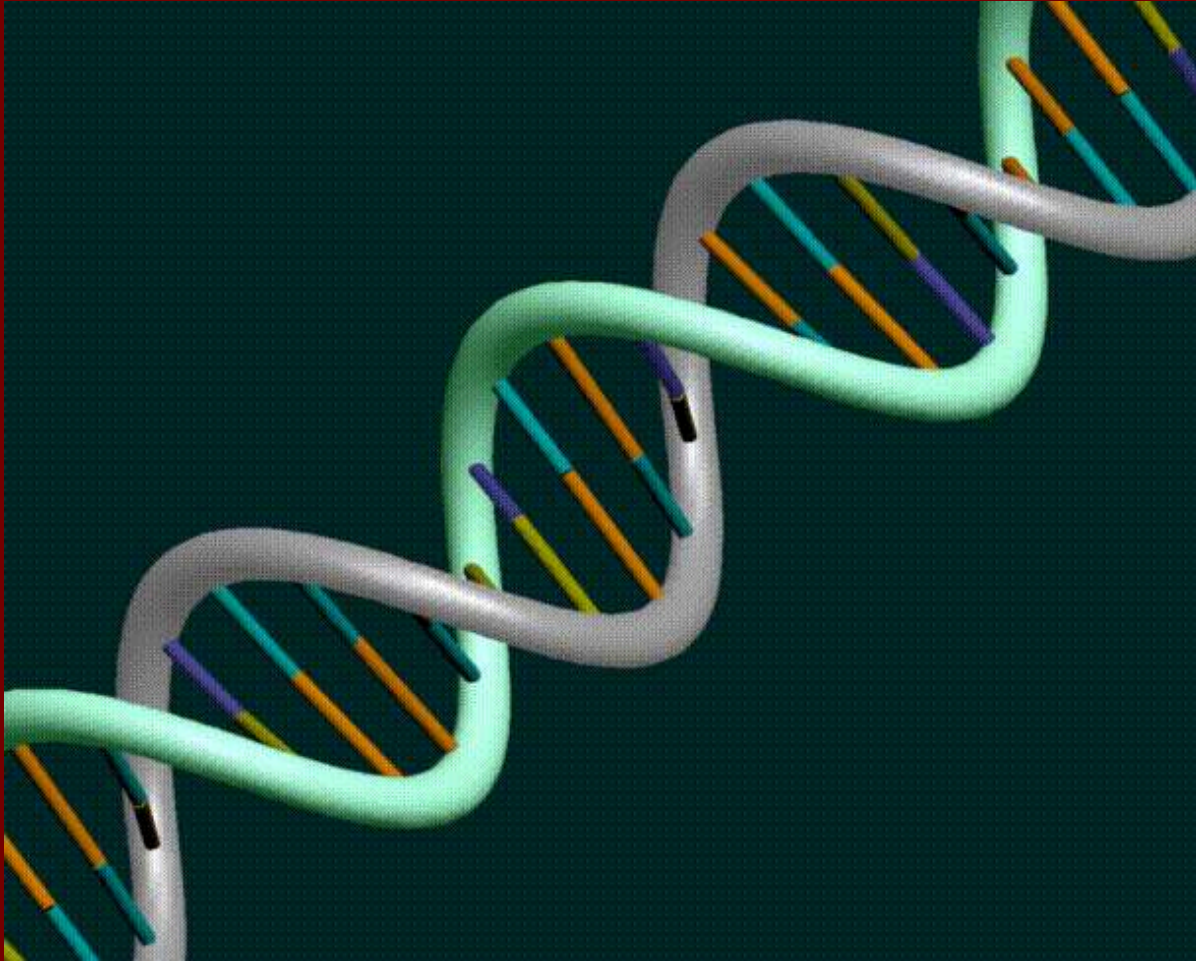
If a text contains a word that is not in the dictionary, a ‘close’ word, i.e. one with a small edit distance, may be suggested as a correction.

Edit Distance: Applications

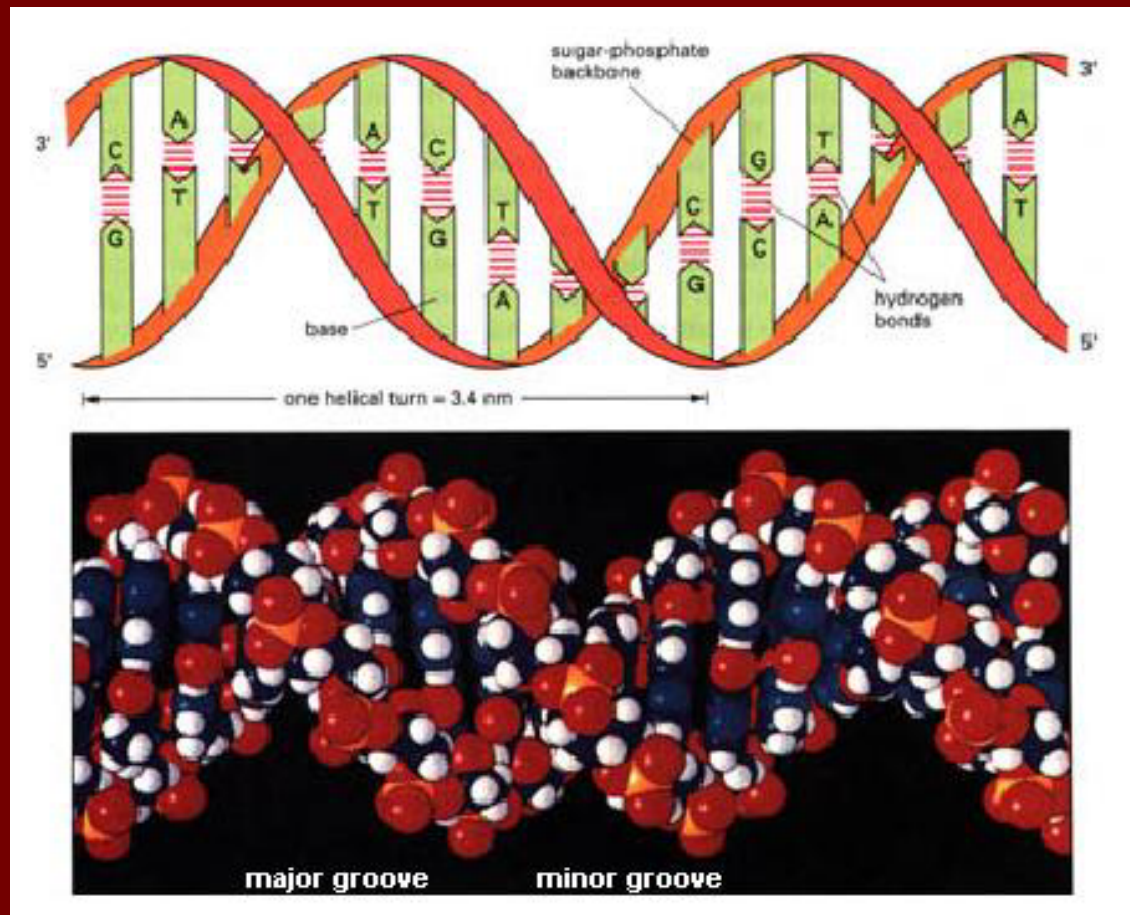
Plagiarism Detection

The edit distance provides an indication of similarity that might be too close in some situations.

Computational Molecular Biology

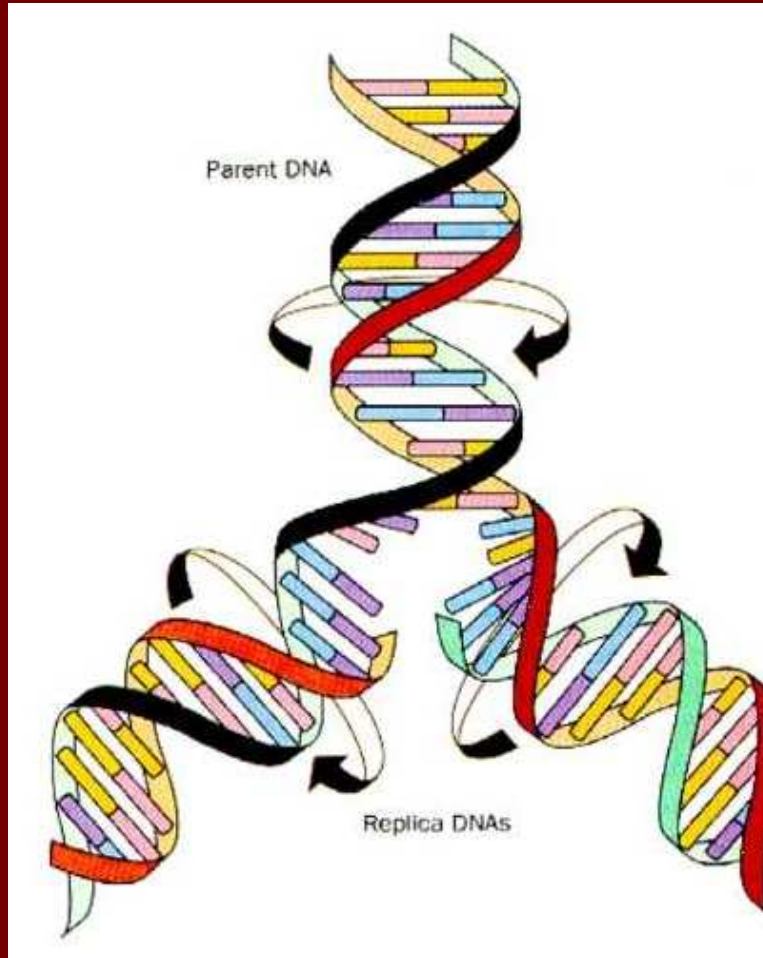


Computational Molecular Biology



- A-adenine
- G-guanine
- C-cytosine
- T-thymine

Computational Molecular Biology



Edit Distance: Applications

Computational Molecular Biology

DNA Replication errors lead to nucleotide:

- Substitutions
- Insertions
- Deletions

Edit Distance: Applications

Human beta globin region on chromosome 11

```
1  gaattctaatt  ctccctctca  accctacagt  caccattttg  gtatattaaa  gatgtgttgt
61  ctactgtcta  gtatccctca  agtagtgtca  ggaattagtc  atttaaatag  tctgcaagcc
121 aggagtgggtg  gctcatgtct  gtaattccag  cactggagag  gtagaagtgg  gaggactgct
181 tgagctcaag  agtttgatat  taccctggac  aacatagcaa  gacctcgtct  ctacttaaaa
241 aaaaaaaaaat  tagccaggca  tgtgatgtac  acctgtagtc  ccagctactc  aggaggccga
301 aatgggagga  tcccttgagc  tcaggaggtc  aaggctgcag  tgagacatga  tcttgccact
361 gcactccagc  ctggacagca  gagtgaaacc  ttgcctcacg  aaacagaata  caaaaacaaa
421 caaacacaaa  actgctccgc  aatgcgcttc  cttgatgctc  taccacatag  gtctgggtac
481 tttgtacaca  ttatctcatt  gctgttcgta  attgttagat  taattttgta  atattgatat
541 tattcctaga  aagctgaggc  ctcaagatga  taacttttat  tttctggact  tgtaatagct
601 ttctcttgta  ttcaccatgt  tgtaactttc  ttagagtagt  aacaatataa  agttattgtg
661 agtttttgca  aacacagcaa  acacaacgac  ccatatagac  attgatgtga  aattgtctat
721 tgtcaattta  tgggaaaaca  agtatgtact  ttttctacta  agccattgaa  acaggaataa
781 cagaacaaga  ttgaaagaat  acattttccg  aaattacttg  agtattatac  aaagacaagc
```

Edit Distance: Applications

```
841 acgtggacct gggaggaggg ttattgtcca tgactggtgt gtggagacaa atgcaggttt
901 ataatagatg ggatggcatc tagcgcaatg actttgccat cacttttaga gagctcttgg
961 ggaccccagt acacaagagg ggacgcaggg tatatgtaga catctcattc tttttcttag
1021 tgtgagaata agaatagcca tgacctgagt ttatagacaa tgagcccttt tctctctccc
1081 actcagcagc tatgagatgg cttgccctgc ctctctacta ggctgactca ctccaaggcc
1141 cagcaatggg cagggctctg tcagggcctt gatagcacta tctgcagagc cagggccgag
1201 aaggggtgga ctccagagac tctccctccc attcccgagc agggtttgct tatttatgca
1261 tttaaatagat atattttatt taaaagaaat aacaggagac tgcccagccc tggctgtgac
1321 atggaaacta tgtagaatat ttggggttcc attttttttt ccttctttca gttagaggaa
1381 aaggggctca ctgcacatac actagacaga aagtcaggag ctttgaatcc aagcctgatc
1441 atttccatgt catactgaga aagtccccac ctttctctga gcctcagttt ctctttttat
1501 aagtaggagt ctggagtaaa tgatttccaa tggctctcat ttcaatacaa aatttccggt
1561 tattaaatgc atgagcttct gttactccaa gactgagaag gaaattgaac ctgagactca
1621 ttgactggca agatgtcccc agaggctctc attcagcaat aaaattctca ccttcacca
1681 ggcccactga gtgtcagatt tgcatgcact agttcacgtg tgtaaaaagg aggatgcttc
```

Edit Distance: Applications

```
1741 tttcctttgt attctcacat accttttagga aagaacttag cacccttccc acacagccat
1801 cccaataact catttcagtg actcaaccct tgactttata aaagtcttgg gcagtataga
1861 gcagagatta agagtacaga tgctggagcc agaccacctg agtgattagt gactcagttt
1921 ctcttagtaa ttgtatgact cagttttcttc atctgtaaaa tggaggggtt ttttaattagt
1981 ttgtttttga gaaaggggtct cactctgtca cccaaatggg agtgtagtgg caaaatctcg
2041 gctcactgca acttgcactt cccagggtca agcgggtcctc ccacctcaac atcctgagta
2101 gctggaacca caggtacaca ccaccatacc tcgctaattt tttgtatttt tggtagagat
2161 ggggtttcac atgttacaca ggatgggtctc agactccgga gctcaagcaa tctgcccacc
2221 tcagccttcc aaagtgctgg gattataagc atgattacag gagttttaac aggctcataa
2281 gattgttctg cagcccgagt gagttaatac atgcaaagag tttaaagcag tgacttataa
2341 atgctaacta ctctagaaat gtttgctagt attttttggt taactgcaat cattcttgct
2401 gcagggtgaaa actagtgttc tgtactttat gccattcat ctttaactgt aataataaaa
2461 ataactgaca ttattgaag gctatcagag actgtaatta gtgctttgca taattaatca
2521 tatttaatac tcttggttcc tttcaggtag atactattat tatccccatt ttactacagt
2581 taaaaaaaact acctctcaac ttgctcaagc atacactctc acacacacaa acataaacta
2641 ctagcaaata gtagaattga gatttggtcc taattatgtc tttgctcact atccaataaa
2701 tattttattga catgtacttc ttggcagttc gtatgctgga tgctggggat acaagatgt
```

Edit Distance: Applications

```
72841 ggattaagaa aatgtggcac atatacacca tggaatacta tgcagccata aaaaatgatg
72901 agttcatgtc ctttgtaggg acatggatga agctggaaac tatcattctc agcaaactat
72961 cacaaggaca ataaaccaa caccgcatgt tctcactcat aggtgggaat tgaacaatga
73021 gaacacatgg acacatgaag aggaacatca cactctgggg actgttatgg ggtggggggc
73081 aggggcaggg atagcactag gagatatacc taatgctaaa tgacgagtta atgggtgcag
73141 cacaccaaca tggcacatgt atacatatat aacaaacctg ccgttgtgca catgtaccct
73201 aaaacttgaa gtataataat aaaaaaaagt tadcctatta aaactgatct cacacatccg
73261 tagagccatt atcaagtctt tctctttgaa acagacagaa atttagtggt ttctcagtca
73321 gttaac
```

Edit Distance: Applications

Computational Molecular Biology

Similarities in DNA Sequences can provide

- Clue to common evolutionary origin
- Clue to common function

Edit Distance: Applications

Speech Recognition

- Algorithms similar to those for the edit-distance problem are used in some speech recognition systems.
- Find a close match between a new utterance and one in a library of classified utterances.

Edit Distance

A better way to display this editing process is to place the words above the other:

M A _ T H S
A _ R T _ S

Edit Distance

<i>S</i>	<i>D</i>	<i>I</i>	<i>M</i>	<i>D</i>	<i>M</i>
M	A	_	T	H	S
A	_	R	T	_	S

- The first word has a gap for every insertion (I) and the second word has a gap for every deletion (D).
- Columns with two different characters correspond to substitutions (S).
- Matches (M) do not count.

Edit Distance

<i>S</i>	<i>D</i>	<i>I</i>	<i>M</i>	<i>D</i>	<i>M</i>
M	A	_	T	H	S
A	_	R	T	_	S

Edit transcript: A string over the alphabet M, S, I, D that describes a transformation of one string into another. Example

$$\begin{array}{cccccc} S & D & I & M & D & M \\ 1+ & 1+ & 1+ & 0+ & 1+ & 0+ & = 4 \end{array}$$

Edit Distance

<i>S</i>	<i>D</i>	<i>I</i>	<i>M</i>	<i>D</i>	<i>M</i>
M	A	_	T	H	S
A	_	R	T	_	S

Edit transcript: A string over the alphabet M, S, I, D that describes a transformation of one string into another. Example

$$\begin{array}{cccccc} S & D & I & M & D & M \\ 1+ & 1+ & 1+ & 0+ & 1+ & 0+ & = 4 \end{array}$$

Edit Distance

- In general, it is not easy to determine the optimal edit distance.
- For example, the distance between ALGORITHM and ALTRUISTIC is at most 6.

A	L	G	O	R	_	I	_	T	H	M
A	L	_	T	R	U	I	S	T	I	C

- Is this optimal?

Edit Distance: DP Algorithm

Recursive definition:

- Suppose we have an m -character string A and an n -character string B .
- Define $E(i, j)$ to be the edit distance between the first i characters of A and the first j characters of B .

Edit Distance: DP Algorithm

- $E(i, j)$:

$$\begin{array}{cccccccccccc} & \underbrace{\hspace{1.5cm}}_{i} & & & & & & & & & & & \\ A & L & G & O & R & I & T & H & M & & & & \\ A & L & \text{—} & T & R & U & I & S & T & I & C & & \\ & \underbrace{\hspace{1.5cm}}_{j} & & & & & & & & & & & \end{array}$$

- The edit distance between entire strings A and B is $E(m, n)$.

Edit Distance

- The gap representation for the edit sequences has a crucial “optimal substructure” property.
- If we remove the last column, the remaining columns must represent the shortest edit sequence for the remaining substrings:

Edit Distance

Edit distance = 6

A	L	G	O	R	_	I	_	T	H	M
A	L	_	T	R	U	I	S	T	I	C

Remove last column, Edit distance = 5

A	L	G	O	R	_	I	_	T	H
A	L	_	T	R	U	I	S	T	I

Edit Distance

There are a couple of obvious base cases:

- The only way to convert an empty string into a string of j characters is by doing j insertions. Thus

$$E(0, j) = j$$

- The only way to convert a string of i characters into the empty string is with i deletions:

$$E(i, 0) = i$$

Edit Distance

Four possibilities for the last column in the shortest possible edit sequence:

Deletion: Last entry in bottom row is empty.

$i=3$									
A	L	G	O	R	I	T	H	M	
$j=2$									
A	L		T	R	U	I	S	T	I
			C						

In this case

$$E(i, j) = E(i - 1, j) + 1$$

Edit Distance

Insertion: The last entry in the top row is empty.

$i=5$											
A	L	G	O	R	_	I	T	H	M		
A	L	_	T	R	U	I	S	T	I	C	
					$j=5$						

In this case

$$E(i, j) = E(i, j - 1) + 1$$

Edit Distance

Substitution: Both rows have characters in the last column.

ALGO R I T H M
AL _ T R U I S T I C

If the characters are different, then

$$E(i, j) = E(i - 1, j - 1) + 1$$

Edit Distance

Substitution: Both rows have characters in the last column.

$i=5$										
A	L	G	O	R	I	T	H	M		
A	L		T	R	U	I	S	T	I	C
					$j=4$					

If characters are same, no substitution is needed:

$$E(i, j) = E(i - 1, j - 1)$$

Edit Distance

Thus the edit distance $E(i, j)$ is the smallest of the four possibilities:

$$E(i, j) = \min \begin{pmatrix} E(i-1, j) + 1 \\ E(i, j-1) + 1 \\ E(i-1, j-1) + 1 & \text{if } A[i] \neq B[j] \\ E(i-1, j-1) & \text{if } A[i] = B[j] \end{pmatrix}$$

Edit Distance

Consider the example:

A	R	T	S	
M	A	T	H	S

The edit distance would be $E(4, 5)$.

Edit Distance

If we recursion to compute, we will have

$$E(4, 5) = \min \begin{pmatrix} E(3, 5) + 1 \\ E(4, 4) + 1 \\ E(3, 4) + 1 & \text{if } A[4] \neq B[5] \\ E(3, 4) & \text{if } A[4] = B[5] \end{pmatrix}$$

Recursion clearly leads to the same repetitive call pattern that we saw in Fibonacci sequence.

Edit Distance

If we recursion to compute, we will have

$$E(4, 5) = \min \begin{pmatrix} E(3, 5) + 1 \\ E(4, 4) + 1 \\ E(3, 4) + 1 & \text{if } A[4] \neq B[5] \\ E(3, 4) & \text{if } A[4] = B[5] \end{pmatrix}$$

To avoid this, we will use the DP approach.

Edit Distance

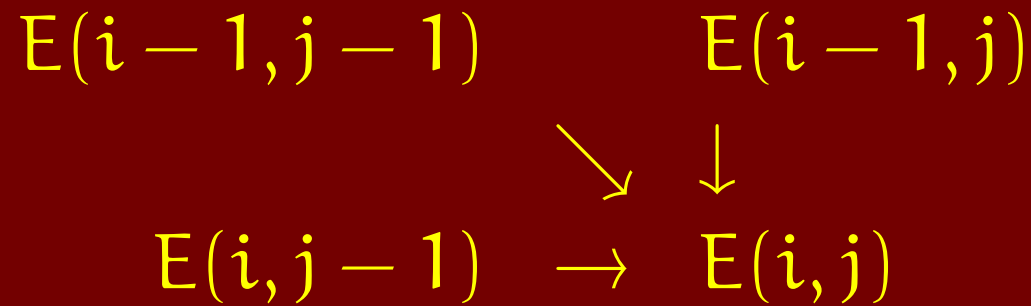
If we recursion to compute, we will have

$$E(4, 5) = \min \begin{pmatrix} E(3, 5) + 1 \\ E(4, 4) + 1 \\ E(3, 4) + 1 & \text{if } A[4] \neq B[5] \\ E(3, 4) & \text{if } A[4] = B[5] \end{pmatrix}$$

We will build the solution bottom-up.

Edit Distance

Pattern of building $E(i, j)$



Edit Distance

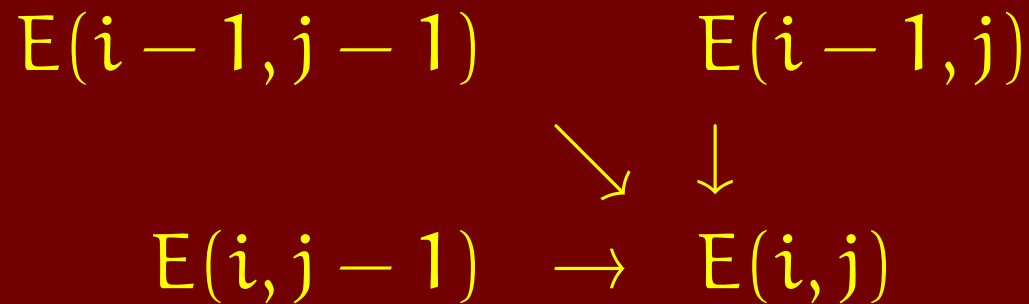
Pattern of building $E(i, j)$

$$\begin{array}{ccc} E(i-1, j-1) & & E(i-1, j) \\ & \searrow & \downarrow \\ E(i, j-1) & \rightarrow & E(i, j) \end{array}$$

Use the base case $E(0, j)$ to fill first row.

Edit Distance

Pattern of building $E(i, j)$



Use the base case $E(0, j)$ to fill first row.

Use the base case $E(i, 0)$ to fill first column.

Edit Distance

Pattern of building $E(i, j)$

$$\begin{array}{ccc} E(i-1, j-1) & & E(i-1, j) \\ & \searrow & \downarrow \\ E(i, j-1) & \rightarrow & E(i, j) \end{array}$$

Use the base case $E(0, j)$ to fill first row.

Use the base case $E(i, 0)$ to fill first column.

Fill the remaining E matrix row by row.

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M					
A					
T					
H					
S					

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓ 1				
A	↓ 2				
T	↓ 3				
H	↓ 4				
S	↓ 5				

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓ 1	↘ 1			
A	↓ 2				
T	↓ 3				
H	↓ 4				
S	↓ 5				

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓ 1	↘ 1	↘ →2		
A	↓ 2				
T	↓ 3				
H	↓ 4				
S	↓ 5				

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓ 1	↘ 1	↘ →2	↘ →3	
A	↓ 2				
T	↓ 3				
H	↓ 4				
S	↓ 5				

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓ 1	↘ 1	↘ →2	↘ →3	↘ →4
A	↓ 2				
T	↓ 3				
H	↓ 4				
S	↓ 5				

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓1	↘1	↘2	↘3	↘4
A	↓2	↘1	↘2	↘3	↘4
T	↓3	↓2	↘2	↘2	→3
H	↓4	↓3	↘↓3	↘↓3	↘3
S	↓5	↓4	↘↓4	↘↓4	↘3

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓1	↘1	↘→2	↘→3	↘→4
A	↓2	↘1	↘→2	↘→3	↘→4
T	↓3	↓2	↘2	↘2	→3
H	↓4	↓3	↘↓3	↘↓3	↘3
S	↓5	↓4	↘↓4	↘↓4	↘3

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓1	↘1	↘2	↘3	↘4
A	↓2	↖1	↘2	↘3	↘4
T	↓3	↓2	↖2	↘2	→3
H	↓4	↓3	↘↘3	↖↘3	↘3
S	↓5	↓4	↘↘4	↖↘4	↖3

Solution path 1:

$$1+ 0+ 1+ 1+ 0 = 3$$

D M S S M

M A T H S

– A R T S

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓1	↖1	↘2	↘3	↘4
A	↓2	↘1	↖2	↘3	↘4
T	↓3	↓2	↘2	↖2	→3
H	↓4	↓3	↘↘3	↘↖3	↘3
S	↓5	↓4	↘↘4	↘↘4	↖3

Solution path 2:

$$1+ \quad 1+ \quad 0+ \quad 1+ \quad 0 \quad = 3$$

S S M D M

M A T H S

A R T _ S

Edit Distance

		A	R	T	S
	0	→1	→2	→3	→4
M	↓1	↘1	↘2	↘3	↘4
A	↓2	↖1	↘2	↘3	↘4
T	↓3	↓2	↘2	↖2	→3
H	↓4	↓3	↘↘3	↘↖3	↘3
S	↓5	↓4	↘↘4	↘↘4	↖3

Solution path 3:

1+	0+	1+	0+	1+	0	= 3
<i>D</i>	<i>M</i>	<i>I</i>	<i>M</i>	<i>D</i>	<i>M</i>	
M	A	—	T	H	S	
—	A	R	T	—	S	

Analysis: Edit Distance

- There are $\Theta(n^2)$ entries in the matrix.
- Each entry $E(i, j)$ takes $\Theta(1)$ time to compute.
- The total running time is $\Theta(n^2)$.